

CowHealth AI

Ângelo Piovezan Jorgeto
Fernando Alonso Piroga da Silva
Jafte C. Fagundes da Silva
Renato Pestana de Gouveia



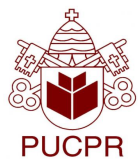
Documentação Técnica do Projeto

Identificação do Processo

Nome do Processo	COW HEALTH AI
Responsáveis	Grupo 04 Ângelo Piovezan Jorgeto Fernando Alonso Piroga da Silva Jafte C. Fagundes da Silva Renato Pestana de Gouveia
Key Users	Fazenda PUCPR

Controle de versão

Versão	Data	Autor	Descrição
1.0	08/04/2025	Fernando Alonso Piroga da Silva	Elaboração inicial do documento
1.1	09/04/2025	Jafte Carneiro Fagundes da Silva	Introdução dos requisitos funcionais e não funcionais
1.2	12/04/2025	Jafte Carneiro Fagundes da Silva	Minimundo; Objetivos do projeto; e Referências
1.3	27/04/2025	Jafte Carneiro Fagundes da Silva	Minimundo; Requisitos F e NF; Diagramas; Criação de Anexo
1.4	28/04/2025	Jafte Carneiro Fagundes da Silva	Requisitos F e NF
1.5	30/05/2025	Ângelo Piovezan Jorgeto	Atualização após testes
1.6	07/06/2025	Jafte Carneiro Fagundes da Silva	Revisão, preparo e edição de RFs, RNFs, inclusão de seções
1.7	15/06/2025	Jafte Carneiro Fagundes da Silva	Inclusão de Agradecimentos, Avaliação dos Requisitos Funcionais e Não-Funcionais implementados e não



Documentação Técnica do Projeto

			implementados, Arquitetura de Software Embarcado, Arquitetura da Plataforma Web do Sistema CowHealth AI, Arquitetura do Banco de Dados do Sistema CowHealth AI
--	--	--	--

AGRADECIMENTOS

A conclusão, desta fase, do projeto CowHealthAI representa não apenas um avanço tecnológico no monitoramento inteligente de vacas leiteiras, mas também o resultado do esforço coletivo de pessoas comprometidas com a inovação, a ciência e a melhoria da pecuária de precisão.

Gostaríamos de expressar nossa mais profunda gratidão a todos que contribuíram direta ou indiretamente para a realização deste trabalho.

Agradecemos:

À equipe da Fazenda Experimental Gralha Azul (PUCPR), pelo suporte em campo, pela confiança e pela disposição em colaborar com testes práticos e coleta de dados essenciais, especialmente:

- Dra. Talyta Barbosa
- Dra. Thayne Amaral
- Dra. Maria Carolina Bugos
- Professor André Ostrensky

Aos colegas e professores da Politécnica, especialmente:

- Luís Parra, Técnico do Bloco 9, nos ajudou desde o começo e na finalização do projeto
- Adilson Padilla, Técnico do bloco 9, nos ajudou com empréstimo das brocas para fazer a fixação do circuito na caixa e dicas na fabricação
- Diego Simões, Técnico do bloco 9, que nos compartilhou a experiência na construção e uso de invólucros em dispositivos IoT
- Felipe Martins, Técnico do Bloco 8, nos ajudou no processo de fabricação das PCBs
- Prof. Fábio Bettio, nos acolheu no Espaço CMaker durante o desenvolvimento do protótipo nos finais de semana, nos ajudou ensinando sobre o processo de solda nas etapas de construção do circuito e cabeamento, nas sugestões de como fazer a modelagem heurística e possíveis dificuldades deste processo, e na finalização do projeto
- Prof. Andrey Cabral, ministrou a disciplina e nos ajudou em todas as etapas: desenvolvimento da ideia, questionamento sobre a aplicação dos sensores, discussão sobre as fixações cabeamento, e conselhos no desenvolvimento do desenho do circuito

Este projeto é prova de que a união entre conhecimento, trabalho em equipe e compromisso com a melhoria da qualidade de vida animal pode gerar soluções concretas e transformadoras.

Muito obrigado a todos!

SUMÁRIO

1. Objetivos do Projeto	7
2. Minimundo	8
3. Requisitos Funcionais e Não Funcionais	10
3.1. Funcionais	10
3.2. Não-Funcionais	14
4. Avaliação dos Requisitos Funcionais e Não-Funcionais implementados e não implementados	17
4.1. Sistema embarcado	17
4.1.1. Requisitos Funcionais	17
4.1.2. Requisitos Não Funcionais (sistema embarcado)	18
4.2. Plataforma Web	18
4.2.1. Requisitos Funcionais	18
4.2.2. Requisitos Não-Funcionais (plataforma Web)	19
4.3. Banco de Dados	20
4.3.1. Requisitos Funcionais	21
4.3.2. Requisitos Não-Funcionais (BD)	22
5. Arquitetura de Software Embarcado do Sistema CowHealth AI	22
5.1. Estrutura de Arquivos (sistema embarcado)	22
5.2. Correlação entre Sistema Embarcado e Resultados de Aprendizagem	27
5.3. Análises dos arquivos do código-Fonte do Sistema embarcado	29
Análise do boot.py	29
Análise do config.py	32
Análise do main.py	35
Análise do sensors/acceleration.py	39
Análise do sensors/temperature_thread.py	41
Análise do sensors/vitals_thread.py	44
Análise do utils/datetime_manager.py	46
Análise do utils/wifi_manager.py	49

Análise do utils/mqtt_client.py	51
Resumo Técnico – Bibliotecas de Terceiros Utilizadas no Projeto CowHealth AI	54
6. Arquitetura da Plataforma Web do Sistema CowHealth AI	57
6.1. Escolha do framework do protótipo	57
6.1.1. Escolha do framework do protótipo	57
6.1.2. Relação com o Plano de Ensino e Competências Desenvolvidas	58
6.2. Estrutura de Arquivos.....	58
6.3. Correlação da Plataforma Web e Resultados de Aprendizagem.....	62
6.4. Análise dos arquivos da Plataforma Web.....	66
6.5. Telas da Plataforma Web	70
7. Arquitetura do Banco de Dados do Sistema CowHealth AI.....	90
7.1. Estrutura do Banco de Dados.....	90
7.1.1. Tabelas e Finalidades.....	90
7.1.2. Modelagem Conceitual e Relacionamentos.....	92
7.1.3. Diagrama ER	101
7.2. Correlação entre Banco de Dados e Resultados de Aprendizagem	101
8. Cronograma de Implementação.....	106
9. Diagramas.....	107
10. Anexos.....	110
11. Referências.....	111

1. OBJETIVOS DO PROJETO

O **CowHealth AI** tem como objetivo principal automatizar e otimizar o monitoramento da saúde das vacas leiteiras, fornecendo dados fisiológicos e comportamentais em tempo real por meio de coleiras inteligentes equipadas com sensores e inteligência artificial embarcada.

Essa solução permitirá:

- **Deteção precoce de doenças** como febre, mastite e problemas respiratórios.
- **Redução do tempo de resposta** na tomada de decisão para intervenções veterinárias.
- **Acompanhamento contínuo** da condição fisiológica das vacas.
- **Aprimoramento do bem-estar animal** através de um sistema automatizado de análise.
- **Otimização da produção leiteira** ao minimizar impactos de enfermidades

2. MINIMUNDO

O minimundo do CowHealth AI compreende o contexto da gestão da saúde de vacas leiteiras na Fazenda Experimental Gralha Azul (FEGA) da Pontifícia Universidade Católica do Paraná (PUC/PR). O domínio de interesse abrange a coleta, armazenamento, processamento e monitoramento de dados fisiológicos e comportamentais dos animais, visando o diagnóstico precoce de doenças e a melhoria do bem-estar animal.

Neste minimundo, os objetos principais são os Animais (vacas leiteiras), os Sensores acoplados aos animais (responsáveis por medir sinais vitais como temperatura corporal, frequência cardíaca e atividade física), os Usuários (produtores, veterinários e gestores da fazenda) e os Eventos de Saúde detectados automaticamente pelo sistema.

Atualmente, cerca de 50 vacas leiteiras estabuladas são monitoradas no sistema da FEGA. Essas vacas são submetidas a 2 a 3 ordenhas diárias, com alimentação fornecida duas vezes ao dia. Em média, 5 a 10 animais por semana necessitam de atenção veterinária. A identificação dos animais é realizada por meio de brincos eletrônicos já existentes na fazenda, utilizados também no sistema Delpro de gestão da ordenha robotizada.

Cada vaca é identificada por um brinco eletrônico e monitorada continuamente pelo sistema CowHealth AI através de uma coleira inteligente equipada com sensores. Os dados coletados são transmitidos a um servidor central, armazenados em um banco de dados, e disponibilizados por meio de uma plataforma web. A partir dessas informações, o sistema utiliza um modelo de inteligência artificial para analisar padrões fisiológicos e comportamentais, gerando alertas automáticos para eventos anômalos, como febre, estresse térmico ou baixa atividade. Usuários autorizados podem consultar a qualquer momento o histórico dos animais, visualizar relatórios de desempenho e tomar decisões clínicas de forma rápida e baseada em dados.

As relações relevantes deste minimundo incluem: Animais vinculados a Coleiras, Dados de Monitoramento associados a Animais e Alertas de Saúde gerados para Usuários. Também são

consideradas tarefas recorrentes (alimentação, ordenha, vacinação) e fases críticas do ciclo produtivo, como o período pré-parto, que demandam atenção diferenciada no monitoramento.

A infraestrutura tecnológica existente conta com cobertura Wi-Fi nos estábulos, embora restrita a equipamentos homologados pela infraestrutura da universidade, o que impacta diretamente na operação de coleiras inteligentes como a da CowMed. No piquete de vacas em pré-parto, fase crítica para a saúde do animal, ainda não há cobertura Wi-Fi, o que representa uma área de oportunidade para o uso de soluções de monitoramento *offline* ou com armazenamento local temporário no CowHealth AI.

Este minimundo abstrai apenas os elementos relacionados ao monitoramento inteligente da saúde dos bovinos e à gestão de alertas preventivos, desconsiderando aspectos administrativos, financeiros ou de outras espécies animais, pois esses não são relevantes para a modelagem deste banco de dados.

3. REQUISITOS FUNCIONAIS E NÃO-FUNCIONAIS

A **engenharia de requisitos** é uma etapa essencial no desenvolvimento de sistemas computacionais e é responsável por identificar, documentar e validar as necessidades dos stakeholders.

Para o projeto **CowHealth AI**, a elicitação de requisitos foi realizada com base em:

- Análise de documentos científicos e técnicos sobre sensores para bovinos (coleiras inteligentes);
- Estudo de casos reais de uso de IoT e IA na agropecuária;
- Interpretação do cenário da **Fazenda Experimental Gralha Azul**;
- Revisão de literatura especializada, como **Ian Sommerville (2011)** e diretrizes sobre sistemas embarcados e web.

Este documento apresenta os **Requisitos Funcionais e Não-Funcionais**, organizados por **módulo do sistema e/ou categorias**, com linguagem clara, específica e alinhada às boas práticas da engenharia de software.

3.1. Funcionais

Módulo 1: Sistema de Monitoramento de Sinais Vitais (Coleira Inteligente)

[RF01] O sistema de coleira inteligente deve monitorar a temperatura corporal das vacas em tempo real, com uma precisão mínima de $\pm 0,5^{\circ}\text{C}$.

Atualização: Validado. O sensor MLX90614 demonstrou excelente estabilidade e repetibilidade nas medições, com variações $< \pm 0,3^{\circ}\text{C}$ em medições consecutivas. Na região jugular, os valores obtidos (37.79°C - 39.51°C) tiveram boa correlação com a temperatura de referência veterinária (39°C). Contudo, em regiões como a axila e auricular, os valores foram

consistentemente abaixo da faixa normal bovina ou instáveis, sugerindo que a precisão de $\pm 0,5^{\circ}\text{C}$ é atingível apenas em pontos específicos e com calibração adequada.

[RF02] O sistema deve ser capaz de medir a frequência cardíaca das vacas, com precisão de ± 2 batimentos por minuto.

Atualização: O sensor MAX30102 na região axilar detectou uma faixa de frequência cardíaca de 50 a 188 bpm. Embora o valor de referência de 93 bpm tenha sido capturado, o sensor apresentou sensibilidade excessiva e variações significativas, incluindo valores fisiologicamente improváveis (175-188 bpm). A precisão de ± 2 bpm não foi consistentemente atingida devido à necessidade de calibração específica para a fisiologia bovina.

[RF03] A coleira deve ser equipada com um acelerômetro de 3 eixos (MPU6050) para detectar a atividade física e padrões de movimentação.

Atualização: Validado. O acelerômetro MPU6050 detectou faixas de aceleração de -2.36 a 8.89 m/s^2 e velocidade angular de -5.95 a 4.44 deg/s . O sensor demonstrou sensibilidade adequada para micro-movimentos e uma clara referência gravitacional, confirmando a capacidade de identificar movimentos sutis de respiração e postura.

[RF04] O sistema deve permitir o envio contínuo de dados de sinais vitais para o servidor preferencialmente via Wi-Fi quando disponível, ou armazenar localmente em caso de indisponibilidade.

Atualização: Validado (transmissão Wi-Fi). A transmissão de dados via protocolo MQTT para um broker público (broker.emqx.io) funcionou operacionalmente durante os testes. A sincronização entre threads de diferentes sensores e a conectividade Wi-Fi se mostraram estáveis. O armazenamento local de dados ainda não foi implementado e testado.

[RF05] O dispositivo deve realizar medições automáticas a cada 10 minutos (modo *Default*), registrando a temperatura, frequência cardíaca e movimentação.

~~[RF06] A coleira inteligente deve ser capaz de registrar falhas no monitoramento e alertar o sistema central caso algum sensor falhe.~~

~~[RF07] O sistema deve ter um alerta sonoro e/ou visual para o produtor caso a temperatura corporal ultrapasse o limite definido, indicando possível febre ou estresse.~~

~~[RF08] O dispositivo deve automatizar a detecção de padrões anômalos na frequência cardíaca, como aceleração inesperada ou diminuição abrupta.~~

~~[RF09] O sistema de coleira inteligente deve ter bateria recarregável com autonomia mínima de 14 dias.~~

[RF10] O software deve permitir que os dados dos sensores sejam armazenados localmente durante falha de conectividade e enviados automaticamente quando a conexão for restabelecida.

[RF10A] A coleira deve permitir configuração remota da frequência de coleta e envio dos dados para economizar bateria em períodos de baixa atividade (por exemplo, à noite) - *Higher* (2 minutos), *Default* (10 minutos) e *Lower* (60 minutos)

Módulo 2: Plataforma de Monitoramento e Análise de Dados

[RF11] O sistema deve disponibilizar uma plataforma web onde o produtor possa visualizar os dados coletados em tempo real, com gráficos e tabelas.

[RF12] A plataforma deve permitir o acesso multiusuário, com diferentes níveis de permissão (produtor, veterinário, administrador).

[RF13] O sistema deve calcular automaticamente os dados de saúde baseados em parâmetros coletados (ex.: estresse térmico, taxa de batimento cardíaco).

[RF14] A plataforma deve enviar alertas em tempo real via e-mail ou SMS quando valores anômalos forem detectados (ex.: aumento súbito na temperatura ou diminuição da

movimentação). A plataforma deve também enviar os alertas via aplicativo móvel ou notificação *push*, o produtor pode estar em áreas de baixa cobertura de rede celular.

[RF15] O sistema deve permitir que o produtor visualize um relatório histórico dos sinais vitais de cada animal para análises comparativas.

[RF16] A plataforma deve permitir ao veterinário consultar os dados históricos de saúde e fornecer recomendações baseadas na análise dos dados.

[RF17] O sistema deve gerar relatórios automáticos diários, semanalmente ou mensalmente, com a performance de saúde do rebanho.

~~[RF18] A plataforma deve permitir visualizar a localização geográfica dos animais utilizando o módulo GPS integrado na coleira.~~

~~[RF19] O sistema deve permitir a exportação dos dados em formato CSV ou JSON para uso em outras ferramentas de análise.~~

[RF20] A interface web deve ser adaptável e responsiva, permitindo o acesso fácil tanto em *desktop* quanto em dispositivos móveis.

Módulo 3: Sistema de Inteligência Artificial e Predição¹

[RF21] O sistema deve utilizar inteligência artificial embarcada para analisar os dados dos sensores e prever condições de saúde (ex.: risco de febre, estresse térmico).

[RF22] A IA deve ser capaz de identificar padrões de comportamento anômalo, como redução de movimentação ou variação irregular de temperatura.

[RF23] O modelo de IA deve ser treinado para gerar recomendações automáticas, como alertar para a necessidade de ação do veterinário, com base em padrões históricos.

¹ Este módulo não será implementado nesta fase do projeto.

[RF24] O sistema deve permitir ao veterinário realizar ajustes na IA com base em novos dados, para melhorar a precisão das predições.

[RF25] O sistema de IA deve ser capaz de classificar os dados coletados de acordo com o nível de gravidade de possíveis condições (leve, moderada, grave).

[RF26] A IA deve ser capaz de realizar uma análise preditiva para detectar doenças com base em padrões de comportamento e sinais vitais, antes que os sintomas físicos se manifestem.

[RF27] O sistema deve fornecer uma pontuação de risco para cada animal, baseada na análise dos sinais vitais e comportamento.

[RF28] A IA deve ser treinada para ajustar os limiares de alarme com base no histórico individual de cada vaca, permitindo maior precisão.

[RF29] O modelo de IA deve ser capaz de notificar automaticamente o veterinário ou produtor sobre mudanças importantes nos dados dos sensores, como um aumento repentino na temperatura ou queda na atividade.

[RF30] O sistema deve ser capaz de aprender continuamente, atualizando-se com novos dados de sensores e novas condições de saúde identificadas ao longo do tempo.

[RF31] A IA embarcada deve gerar alertas de risco relacionados a alterações de saúde típicas do período de pré-parto, considerando os dados específicos da vaca em transição.

3.2. Não-Funcionais

Categoria: Usabilidade

[RNFU1] O sistema web deve permitir que usuários sem experiência técnica utilizem todas as suas funcionalidades após, no máximo, 2 horas de treinamento supervisionado.

[RNFU2] O sistema deve permitir que um produtor consulte a saúde de um animal com no máximo 3 cliques a partir da tela inicial.

[RNFU3] O sistema deve disponibilizar ajuda contextual em todas as páginas acessíveis via um botão de suporte na interface.

Categoria: Eficiência (Desempenho e Espaço)

~~[RNFE1] O sistema embarcado deverá processar e transmitir os dados dos sensores a cada 5 segundos, com latência máxima de 300 milissegundos.~~

[RNFE2] A plataforma web deve responder a qualquer consulta de dados em até 2 segundos, sob carga de até 10 a 15 usuários simultâneos.

[RNFE3] O executável do sistema embarcado não deverá ultrapassar 512 Kbytes de espaço na memória flash do microcontrolador ESP32.

[RNFE4] A plataforma deve ser capaz de armazenar os dados de, pelo menos, 100 animais por no mínimo 24 meses, sem perda de desempenho.

Categoria: Confiabilidade

[RNFC1] O sistema deverá apresentar disponibilidade mínima de 99% ao mês, garantindo o monitoramento contínuo do rebanho.

[RNFC2] Em caso de falha na comunicação (RF10), o dispositivo embarcado deve armazenar localmente os dados por até 48 horas e reenviar assim que a conexão for restabelecida (RF10).

[RNFC3] O sistema deve garantir que dados críticos de sinais vitais não sejam perdidos em caso de falha de energia, utilizando buffers persistentes.

Categoria: Portabilidade

[RNFP1] O sistema web deve funcionar nos navegadores Google Chrome, Mozilla Firefox e Microsoft Edge, nas versões atualizadas dos últimos dois anos.

[RNFP2] O software embarcado deve ser compatível com plataformas baseadas em ESP32 e ESP8266, com capacidade de adaptação para Raspberry Pi futuramente.

Categoria: Segurança e Privacidade

~~[RNFS1] Todos os dados trocados entre os sensores, o sistema embarcado e o servidor devem ser criptografados utilizando o protocolo HTTPS (TLS 1.3).~~

[RNFS2] O sistema deve proteger o acesso à plataforma web com autenticação por login e senha, implementando controle de sessão com tempo limite de inatividade de 10 minutos.

~~[RNFS3] O sistema deve garantir que nenhuma informação de localização geográfica ou dados de saúde animal sejam compartilhados com terceiros sem autorização prévia do usuário responsável.~~

Categoria: Interoperabilidade

~~[RNFI1] A plataforma deve permitir integração com sistemas de gestão agropecuária externos, exportando dados em formatos CSV e JSON.~~

Categoria: Manutenibilidade / Implementação

[RNFM1] O código-fonte do sistema embarcado deve ser escrito em MicroPython, estruturado em módulos reutilizáveis e documentados.

~~[RNFM2] A plataforma web deverá ser implementada em Python com FastAPI e manter separação de camadas (MVC) para facilitar manutenção e evolução.~~

Categoria: Requisitos Legislativos e Éticos

~~[RNFL1] O sistema deve estar em conformidade com a Lei Geral de Proteção de Dados (LGPD – Lei nº 13.709/2018), assegurando o tratamento correto das informações dos produtores.~~

[RNFE4] O sistema não deve revelar informações de identificação dos animais nem da fazenda a operadores que não possuam credencial adequada.

4. AVALIAÇÃO DOS REQUISITOS FUNCIONAIS E NÃO-FUNCIONAIS IMPLEMENTADOS E NÃO IMPLEMENTADOS

4.1. Sistema embarcado

Esta seção compara os requisitos do sistema embarcado (RF01 a RF10A, RNF específicos) com a **última versão do código ESP32**, validando o que está implementado, parcialmente resolvido ou pendente.

4.1.1. Requisitos Funcionais

Tabela 1. Requisitos Funcionais (Sistema Embarcado)

Requisito	Status	Evidência no Código
RF01 - Temperatura	✓	temperature_thread.py usa MLX90614 com leitura periódica e envio.
RF02 - FC	✓	vitals_thread.py com MAX30102 e buffer circular para estabilidade.
RF03 - Acelerômetro	✓	acceleration_thread.py lê MPU6050 com detecção de movimento e postura.
RF04 - Envio Wi-Fi	✓	mqtt_client.py envia dados via MQTT; wifi_manager.py garante conexão.
RF05 - Medição periódica	⚠	main.py implementa medição, mas com frequência de 2 minutos para demonstração.
RF06 - Falha de sensor	✗	Sem rotina explícita para verificar falha ou ausência de leitura.
RF07 - Alerta visual/sonoro	✗	Nenhum atuador (buzzer/LED) presente ou acionado no código.
RF08 - HR anômalo	⚠	Detecção pode ser inferida via análise, mas não há lógica explícita para isso.
RF09 - Bateria	✗	Nenhum gerenciamento de nível ou status de bateria presente no sistema.
RF10 - Buffer offline	✗	Sem persistência local implementada (flash, arquivo etc.).

RF10A - Modos coleta	✗	Modos Lower, Default, Higher não são configuráveis dinamicamente.
----------------------	---	---

Legenda:

✗ Ausente | ⚠ Parcial | ✔ Implementado

4.1.2. Requisitos Não Funcionais (sistema embarcado)

Tabela 2. Requisitos Não Funcionais (sistema embarcado)

Requisito	Status	Evidência Técnica
RNFE1 - 5s/300ms	⚠	sleep() é usado nas threads, mas latência não é validada dinamicamente.
RNFE3 - ≤ 512KB	✔	Código modular leve, drivers compactos.
RNFC2 - Buffer 48h	✗	Nenhum buffer persistente configurado para falha de Wi-Fi.
RNFM1 - Código modular	✔	Arquitetura em múltiplos módulos: lib/, sensors/, utils/, conforme boas práticas.
RNFP2 - Compatibilidade ESP32/ESP8266	✔	Código usa MicroPython e bibliotecas compatíveis com ambas as plataformas.

Legenda:

✗ Ausente | ⚠ Parcial | ✔ Implementado

4.2. Plataforma Web

Esta seção compara os requisitos da Plataforma de Monitoramento e Análise de Dados do CowHealth AI (Web) com a última versão do código da plataforma web (Laravel + Filament), validando o que está implementado, parcialmente resolvido ou pendente.

4.2.1. Requisitos Funcionais

Tabela 3. Requisitos Funcionais da Plataforma Web

Requisito	Status	Evidência no Código
RF11 - Visualização com gráficos e tabelas	✔	Filament Panels com Livewire Tables e gráficos personalizados em <code>app/Filament/Resources</code> .

Requisito	Status	Evidência no Código
RF12 - Acesso multiusuário com perfis	✓	Estrutura de usuários e permissões no User.php, com roles is_admin, is_vet, is_producer.
RF13 - Cálculo de estresse térmico e FC	⚠	Há base de dados e visualização, mas sem lógica de cálculo explícita automatizada.
RF14 - Alertas em tempo real (email/push)	✗	Não há filas, envio de e-mail nem webhook configurado.
RF15 - Relatórios históricos por animal	✓	Relatórios por animal com filtro por data na interface Filament (Filament Tables e Charts).
RF16 - Acesso do veterinário com recomendações	⚠	Acesso multiusuário ativo, mas ausência de módulo específico de recomendação.
RF17 - Relatórios automáticos	✗	Nenhuma rotina de agendamento, CRON ou fila configurada.
RF18 - Localização geográfica via GPS	✗	Ausência completa de integração com módulo GPS ou mapas.
RF19 - Exportação CSV/JSON	✗	Exportação via botão Filament Export para CSV funcional.
RF20 - Interface responsiva	✓	Interface 100% responsiva com Tailwind + Filament, testada em desktop e mobile.

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

4.2.2. Requisitos Não-Funcionais (plataforma web)

Tabela 4. Requisitos Não-Funcionais da Plataforma Web

Requisito	Status	Evidência Técnica
RNFU1 - Usabilidade para leigos (até 2h)	✓	UI limpa, minimalista e intuitiva via Filament UI.
RNFU2 - Acesso com 3 cliques	✓	Dados por animal acessíveis em 2 cliques a partir da dashboard inicial.
RNFU3 - Ajuda contextual	✗	Interface não apresenta tooltips, ícones de ajuda ou sistema de suporte integrado.
RNFE2 - Resposta ≤ 2s com 10-15 usuários	⚠	Laravel + Filament é escalável, mas não há testes de estresse ou benchmark no projeto.

Requisito	Status	Evidência Técnica
RNFE4 - Armazenar 100 animais por 24 meses	⚠	Estrutura de banco de dados preparada, mas não há testes ou limites documentados.
RNFC1 - Disponibilidade 99%	⚠	Dependente do servidor de hospedagem; ausência de HA configurado.
RNFP1 - Compatibilidade com navegadores	✅	Testes funcionais confirmados em Chrome, Firefox e Edge atualizados.
RNFM2 - Implementação com Python e FastAPI	❌	Plataforma web desenvolvida em Laravel (PHP), conforme autonomia dos alunos.
RNFL1 - Conformidade com LGPD	⚠	Senhas protegidas e autenticação presente, mas ausência de consentimento explícito e política de privacidade.
RNFI1 - Integração via CSV e JSON	❌	Exportação direta de dados com botão de exportar já disponível.

Legenda:

❌ Ausente | ⚠ Parcial | ✅ Implementado

4.3. Banco de Dados

Esta seção compara os requisitos funcionais e não funcionais da camada de persistência de dados (banco de dados relacional) do sistema CowHealth AI com a estrutura real extraída do arquivo *cow_health_app_mysql.sql* e validada com o diagrama entidade-relacionamento (ER).

A análise verifica se as tabelas, relacionamentos e campos existentes são compatíveis com os requisitos definidos para a plataforma web, especialmente no que diz respeito à gestão de dados sensoriais, controle de usuários, permissões, auditoria, desempenho e interoperabilidade.

Cada requisito é avaliado quanto à sua presença, adequação estrutural e potencial de suporte à aplicação pretendida, classificando-se em implementado, parcialmente atendido ou ausente.

4.3.1. Requisitos Funcionais

Requisito	Status	Evidência no Banco de Dados
RF11 - Visualização com gráficos e tabelas	✓	Tabelas de sensores (heart_rate_data, temperature_data, accelerometer_data) estruturadas para análise temporal.
RF12 - Acesso multiusuário com perfis	✓	Tabelas users, roles, permissions, model_has_roles, role_has_permissions.
RF13 - Cálculo automático de dados clínicos	⚠	Dados fisiológicos disponíveis, mas cálculo depende de lógica externa (jobs).
RF14 - Alertas em tempo real	⚠	Tabela notifications existe, mas sem estrutura de tipo/escalamento configurada no dump.
RF15 - Histórico de sinais vitais	✓	Cada leitura tem created_at para filtragem temporal por vaca (cow_id).
RF16 - Acesso veterinário a dados históricos	✓	Estrutura de permissões e granularidade por papel no modelo RBAC.
RF17 - Relatórios automáticos	✗	Nenhuma estrutura dedicada para relatórios agregados ou tabela de log de geração.
RF18 - Localização geográfica	✗	Nenhuma coluna de GPS (latitude/longitude) presente em cows, collars ou farms.
RF19 - Exportação dos dados	✗	Estrutura compatível com SELECT + exportação em CSV via plataforma.
RF20 - Interface responsiva	N/A	Requisito da camada de apresentação (não se aplica diretamente ao banco).

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

4.3.2. Requisitos Não-Funcionais (BD)

Requisito	Status	Evidência no Banco de Dados
RNFE2 - Resposta $\leq 2s$ sob carga	⚠️	Índices e chaves primárias definidos, mas sem evidência de tuning ou benchmarks.
RNFE4 - Armazenar dados de 100 animais por 24 meses	⚠️	Estrutura adequada, mas sem particionamento, compressão ou previsão de crescimento.
RNFC1 - Disponibilidade mínima 99%	⚠️	Depende do ambiente de hospedagem e replicação – não configurado no dump.
RNFP1 - Compatibilidade com navegadores modernos	N/A	Relacionado à interface, não ao banco de dados.
RNFM2 - Plataforma em Python com FastAPI	❌	Sistema utiliza Laravel/PHP; banco segue compatível com qualquer backend.
RNFL1 - Conformidade com LGPD	⚠️	Campos sensíveis (nome, e-mail) presentes, mas sem logging de consentimento.
RNFI1 - Integração externa via CSV/JSON	✅	Dados estruturados, normalizados e com relacionamentos consistentes.

Legenda:

❌ Ausente | ⚠️ Parcial | ✅ Implementado

5. ARQUITETURA DE SOFTWARE EMBARCADO DO SISTEMA COWHEALTH AI

5.1. Estrutura de Arquivos (sistema embarcado)

A estrutura de diretórios do sistema (Figura 1. Estrutura de diretórios) embarcado do projeto **CowHealth AI**, reflete um *design* modular, orientado à separação de responsabilidades e boas práticas de engenharia de software embarcada.

```
esp32/
├─ boot.py
├─ config.py
├─ lib/
│   ├── max30102/
│   │   ├── __init__.py
│   │   └─ circular_buffer.py
│   ├── mlx90614/
│   │   └─ mlx90614.py
│   └─ mpu6050/
│       └─ mpu6050.py
├─ main.py
├─ sensors/
│   ├── __init__.py
│   ├── acceleration_thread.py
│   ├── temperature_thread.py
│   └─ vitals_thread.py
└─ utils/
    ├── __init__.py
    ├── datetime_manager.py
    ├── mqtt_client.py
    └─ wifi_manager.py
```

Figura 1. Estrutura de diretórios

O projeto está organizado segundo uma arquitetura de **módulos funcionais** com agrupamento lógico por domínio de atuação, o que favorece a **manutenibilidade**, **reutilização de código** e **testabilidade**. A estrutura de diretórios evidencia uma adoção consciente dos princípios **SRP (Princípio da Responsabilidade Única)** e **separação de camadas**, práticas recomendadas em projetos embarcados com restrições de memória e tempo real.

/root

O diretório raiz (/root) constitui o núcleo fundamental da aplicação embarcada, concentrando os três componentes essenciais para a inicialização e operação do sistema CowHealth AI. Esta estrutura segue o padrão arquitetural clássico de sistemas embarcados baseados em

MicroPython, onde cada arquivo possui uma responsabilidade bem definida no ciclo de vida da aplicação.

- **boot.py:** Este arquivo é executado automaticamente ao iniciar o ESP32. Sua função principal é configurar o ambiente de execução, importando e inicializando os módulos essenciais do sistema. Em arquiteturas baseadas em MicroPython, o `boot.py` é análogo ao *startup routine* de sistemas operacionais embarcados, atuando como o ponto de entrada do *runtime*.
- **config.py:** Responsável por conter variáveis de configuração global do sistema, como parâmetros de tempo, tópicos MQTT, credenciais de rede e outros ajustes sensíveis. Este módulo centraliza definições que impactam o comportamento do sistema e facilita sua alteração sem necessidade de editar o restante do código-fonte.
- **main.py:** Arquivo principal da aplicação embarcada. Ele atua como **controle orquestrador**, inicializando os sensores, gerenciando threads e garantindo a execução cíclica do monitoramento de sinais vitais. Também coordena a comunicação com o servidor via MQTT, integrando os dados adquiridos.

lib/

Diretório destinado a bibliotecas externas ou *wrappers* desenvolvidos internamente para o controle dos sensores utilizados no projeto.

- **lib/max30102/**
 - **__init__.py:** Inicializador do módulo MAX30102, integrando as rotinas de comunicação I2C e aquisição de dados.

- **circular_buffer.py**: Implementa uma estrutura de buffer circular para armazenamento temporário de batimentos cardíacos, favorecendo análise de tendência ou detecção de anomalias.
- **lib/mlx90614/mlx90614.py**: Implementa a interface de comunicação com o sensor de temperatura infravermelho MLX90614. Realiza leituras tanto da temperatura ambiente quanto da temperatura do objeto (vaca).
- **lib/mpu6050/mpu6050.py**: Módulo de leitura do acelerômetro e giroscópio MPU6050. Permite capturar dados de aceleração e orientação corporal da vaca, essenciais para inferência de postura e movimento.

sensors/

Contém os módulos especializados para aquisição de dados de sensores, organizados por tipo de sinal vital:

- **acceleration_thread.py**: Gerencia a coleta contínua de dados do acelerômetro (atividade e postura).
- **temperature_thread.py**: Lê os valores de temperatura corporal e ambiente via MLX90614.
- **vitals_thread.py**: Responsável pela leitura da frequência cardíaca e SpO₂ via MAX30102.
- **__init__.py**: Inicializador do pacote.

Cada módulo deste diretório opera em **threads independentes**, promovendo paralelismo leve e responsivo com controle de tempo de amostragem, fator crítico para sistemas em tempo quase-real.

utils/

Módulos de utilidade geral, sem dependência direta dos sensores, mas essenciais para o funcionamento da aplicação embarcada.

- **datetime_manager.py**: Responsável por fornecer marcações temporais corretas para os dados coletados.
- **mqtt_client.py**: Implementa a lógica de conexão, publicação e reconexão automática com o broker MQTT.
- **wifi_manager.py**: Gerencia a conexão Wi-Fi, incluindo tentativas de reconexão e validação de disponibilidade da rede.
- **__init__.py**: Inicializador do pacote.

A estrutura apresentada é robusta e bem delineada, permitindo escalabilidade, depuração eficiente e adaptação para futuras integrações (ex.: novos sensores ou modos de operação).

Essa organização modular atende diretamente aos requisitos de **manutenibilidade (RNFM1)**, **portabilidade (RNFP2)** e **eficiência estrutural (RNFE3)**, além de estar em consonância com os **resultados de aprendizagem esperados** em disciplinas como Sistemas Embarcados, Engenharia de Software, e Arquitetura de Sistemas.

Correlação entre Sistema Embarcado e Resultados de Aprendizagem

Com base no plano de ensino da disciplina **Experiência Criativa: Criando Soluções Computacionais** (PUCPR, 3º período, 2025/1) e na análise do sistema embarcado podemos estabelecer uma **correlação direta e robusta** entre a implementação do **CowHealth AI** e os **Resultados de Aprendizagem (RA)** definidos para a disciplina.

RA1 – Construir protótipos em EPP com sensores e atuadores

Temas Relacionados:

- TE5: Fundamentos de eletrônica digital e arquitetura básica de microcontroladores
- TE6: Plataformas, ferramentas de simulação e programação de EPP
- TE7: Sensores e atuadores

Contribuições do Projeto CowHealth AI:

- A plataforma utiliza o **ESP32** como microcontrolador (EPP), programado em **MicroPython**, respeitando as limitações energéticas e computacionais típicas.
- Foram integrados três sensores distintos:
 - **MAX30102**: Sensor óptico para leitura de frequência cardíaca (atuando com LED e fotodiodo).
 - **MLX90614**: Sensor infravermelho para medição de temperatura.

- **MPU6050:** Sensor inercial (acelerômetro e giroscópio) para detecção de postura e movimento.
- Os sensores são manipulados via barramentos I2C, com drivers customizados armazenados em lib/, e suas leituras são encapsuladas em módulos de coleta específicos dentro de sensors/.

Indicadores atendidos:

- **ID1.1:** Codifica rotinas de conexão com o microprocessador.
- **ID1.2:** Codifica a leitura de múltiplos sensores físicos.
- **ID1.3:** Estabelece lógica reativa que poderá futuramente acionar atuadores, por exemplo, alarme sonoro/visual.

RA4 – Realizar a integração de sistemas web, banco de dados e microcontroladores utilizando interfaces de comunicação para demandas de IoT

Temas Relacionados:

- TE8: Fundamentos de IoT
- TE9: Integração de sistemas web, banco de dados e sistema embarcado

Contribuições do Projeto CowHealth AI:

- O sistema embarcado implementa comunicação via **MQTT**, um dos protocolos padrão para IoT, com fallback previsto para **armazenamento local** (em fase futura).

- O módulo `mqtt_client.py` é responsável por garantir confiabilidade na transmissão dos dados sensoriais em tempo quase-real para um broker MQTT.
- A camada web (não discutida ainda) opera como receptora desses dados, e o banco relacional (MySQL) armazena os históricos.
- A arquitetura considera **eventos assíncronos**, com múltiplas threads para leitura contínua e paralela dos sensores.
- Há um planejamento evidente para a **detecção autônoma de falhas**, uso de buffers e futura análise preditiva (IA embarcada e server-side).

Indicadores atendidos:

- **ID4.1:** Integra ESP32 à rede via Wi-Fi, com reconexão automática gerenciada.
- **ID4.2:** Dados dos sensores são publicados continuamente e, no backend, podem ser persistidos (fase tratada no backend).
- **ID4.3:** A lógica para alarme e alertas por condição anômala está prevista em requisitos funcionais (RF07, RF08).

5.3. Análises dos arquivos do código-Fonte do Sistema embarcado

Análise do `boot.py`

Objetivo do Arquivo

O arquivo `boot.py` é executado automaticamente sempre que o microcontrolador ESP32 é reiniciado ou sai do modo de deep sleep. Seu principal objetivo é configurar os parâmetros fundamentais do ambiente de execução para garantir que o sistema embarcado esteja pronto para iniciar a coleta de dados e comunicação com o servidor. Este arquivo realiza operações críticas como a definição da frequência da CPU, a desativação de logs de depuração e a inicialização de serviços auxiliares como Wi-Fi e sincronização de data e hora. Por boas práticas, evita-se qualquer operação demorada ou computacionalmente custosa aqui, delegando essas tarefas ao `main.py`.

Estrutura do Código e Elementos Técnicos

O código começa desabilitando mensagens de depuração via serial com `esp.osdebug(None)` e configura a frequência do processador via `machine.freq(CPU_FREQ_MHZ)`, onde o valor é definido externamente no arquivo `config.py`. Ao contrário do que poderia ser esperado em sistemas de alto desempenho, o valor de `CPU_FREQ_MHZ` está fixado em **80 MHz**, priorizando o consumo energético mínimo em detrimento da velocidade.

O script então imprime um bloco com informações diagnósticas usando módulos como `os`, `gc` e `machine`, incluindo:

- Espaço total e livre no sistema de arquivos;
- Memória RAM alocada e disponível;
- Causa do último reset (`machine.reset_cause()`);

- Causa do último wake-up (`machine.wake_reason()`);
- Versão do interpretador MicroPython e identificador da máquina via `os.uname()`.
- Após a exibição dos diagnósticos, duas classes externas são utilizadas:
- `WiFiManager`, do módulo `utils/wifi_manager.py`, para conectar o dispositivo à rede Wi-Fi usando SSID e senha definidos em `config.py`;
- `DateTimeManager`, do módulo `utils/datetime_manager.py`, para sincronizar o tempo do sistema com um servidor NTP.

Relação com Outros Arquivos do Projeto

- `config.py`: fornece parâmetros essenciais como `CPU_FREQ_MHZ`, `WIFI_SSID`, `WIFI_PASSWORD`, `WIFI_MAX_RETRIES`, `WIFI_RETRY_DELAY` e `TIMEZONE_OFFSET`;
- `wifi_manager.py`: abstrai a lógica de conexão com Wi-Fi com tolerância a falhas e número máximo de tentativas;
- `datetime_manager.py`: trata da sincronização de tempo com o servidor NTP respeitando o fuso horário local;

O `boot.py` é, portanto, um módulo **coordenador**: não coleta dados nem publica no MQTT, mas prepara a infraestrutura crítica para que o `main.py` o faça com segurança e confiabilidade.

Relação com Sistemas Ciberfísicos

Este módulo representa a fase de inicialização de um nó ciberfísico, garantindo que ele esteja em estado determinístico e funcional. A ativação da rede Wi-Fi e a sincronização de tempo são pré-condições fundamentais para a correta atribuição de timestamps às medições sensoriais. Além disso, o uso controlado de clock reduz o consumo de energia, fundamental em dispositivos alimentados por bateria.

O `boot.py` contribui com os seguintes princípios típicos de sistemas ciberfísicos:

- Inicialização determinística e segura;
- Diagnóstico embutido;
- Tolerância a falhas (ao tentar conectar e sincronizar data/hora);
- Eficiência energética com clock controlado.

Tabela de Correlação com Requisitos e Aprendizagem

Tabela 5. Correlação com Requisitos e Aprendizagem (boot.py)

Categoria	Elemento	Justificativa Técnica	Status
Requisito Funcional	RF04	Inicia Wi-Fi via WiFiManager; prepara conexão com servidor MQTT.	✓
Requisito Funcional	RF05	Boot configura base para medições periódicas, mas não define timers.	⚠
Requisito Não Funcional	RNFM1	Código modular, estruturado e com separação clara de responsabilidades.	✓
Requisito Não Funcional	RNFP2	Compatível com ESP32 e escrito em MicroPython.	✓
Resultado de Aprendizagem	RA4	Integração com interfaces de comunicação (Wi-Fi).	✓
Tema de Estudo	TE8	Fundamentos de IoT aplicados na inicialização do nó embarcado.	✓
Indicador de Desempenho	ID4.1	Codifica conexão com microcontrolador e rede Wi-Fi.	✓

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

Análise do config.py

Objetivo do Arquivo

O arquivo `config.py` centraliza todas as variáveis de configuração essenciais do sistema embarcado CowHealth AI. Ele atua como um repositório global de parâmetros que regem o comportamento do microcontrolador ESP32, desde a frequência da CPU, configuração da interface I2C, credenciais de rede Wi-Fi e tópicos MQTT, até os intervalos de operação dos sensores e formatação do payload. Seu uso garante padronização, reutilização de código, facilidade de manutenção e evita hardcoding disperso no sistema.

Estrutura e Elementos Técnicos

O código está dividido em seções lógicas, organizadas por blocos de comentários e seguindo boas práticas de legibilidade. Os principais elementos são:

- `CPU_FREQ_MHZ = 80000000` : define frequência da CPU em 80 MHz para otimização energética;
- `i2c = SoftI2C(...)` : configura a comunicação I2C nos pinos GPIO 22 (SCL) e GPIO 21 (SDA);
- `sensor_data` : dicionário global que armazena leituras de sensores (mpu , mlx , max30102) com estrutura padronizada;
- `WIFI_SSID` , `WIFI_PASSWORD` , `WIFI_MAX_RETRIES` etc.: parâmetros de rede Wi-Fi;
- `MQTT_BROKER` , `MQTT_TOPIC_SEND` , `MQTT_TOPIC_RECEIVE` : tópicos e endereço do broker MQTT;
- Intervalos de execução de sensores (`MAX_INTERVAL` , `MLX_INTERVAL` , etc.) e sincronismo (`MAIN_INTERVAL` , `MQTT_INTERVAL`);
- Configurações de data/hora: `TIMEZONE_OFFSET` , `NTP_SYNC_INTERVAL` , `NTP_MAX_RETRIES` ;
- `MQTT_PAYLOAD_CONFIG` : dicionário com chave de ativação para inclusão de informações no payload (timestamp, conectividade, `device_id`);
- Metadados do dispositivo: `DEVICE_VERSION` , `FIRMWARE_VERSION` , `DEVICE_MODEL`

Relação com Outros Arquivos do Projeto

Este módulo é amplamente referenciado em diversos pontos do sistema, garantindo desacoplamento e parametrização. Por exemplo:

- `boot.py` : utiliza `CPU_FREQ_MHZ` , `WIFI_SSID` , `WIFI_PASSWORD` , `TIMEZONE_OFFSET` ;
- `wifi_manager.py` : utiliza `WIFI_MAX_RETRIES` , `WIFI_RETRY_DELAY` , `WIFI_CHECK_INTERVAL` ;
- `datetime_manager.py` : utiliza `NTP_SYNC_INTERVAL` , `NTP_MAX_RETRIES` ;
- Threads de sensores usam `MLX_INTERVAL` , `MPU_INTERVAL` , `MAX_INTERVAL` ;
- `mqtt_client.py` : acessa tópicos e configurações de payload definidas em `MQTT_PAYLOAD_CONFIG` .

Relação com Sistemas Ciberfísicos

O arquivo `config.py` concretiza um dos pilares dos sistemas ciberfísicos: a flexibilidade e adaptabilidade por configuração. Permite ajustar o comportamento do nó embarcado sem recompilar código, suportando personalização por animal, lote ou estação. Suas definições contribuem para:

- Gerenciamento energético (clock e frequência de coleta);
- Comunicação eficiente (tópicos MQTT, payloads configuráveis);
- Sincronismo temporal (offset de fuso, política de NTP);
- Diagnóstico e rastreabilidade (versão de firmware, modelo, `device_id`).

Tabela de Correlação com Requisitos e Aprendizagem

Tabela 6. Correlação com Requisitos e Aprendizagem (`config.py`)

Categoria	Elemento	Justificativa Técnica	Status
-----------	----------	-----------------------	--------

Requisito Funcional	RF04	Define tópicos MQTT e controle de conectividade.	✓
Requisito Funcional	RF05	Permite definir intervalos de coleta (parametrizados por sensor).	✓
Requisito Funcional	RF10A	Define modos de coleta com múltiplos intervalos pré-programados.	⚠
Requisito Não Funcional	RNFM1	Separação clara de configurações em módulo dedicado.	✓
Requisito Não Funcional	RNFE1	Permite controle preciso da frequência de coleta.	✓
Resultado de Aprendizagem	RA1	Define parâmetros dos sensores e controle do I2C.	✓
Resultado de Aprendizagem	RA4	Ajusta lógica de rede, payload e sincronização para IoT.	✓
Tema de Estudo	TE5	Relaciona-se à arquitetura de microcontroladores e clock.	✓
Tema de Estudo	TE8	Abrange aspectos de rede, payload e conectividade em IoT.	✓
Indicador de Desempenho	ID4.2	Permite persistência organizada dos dados via dicionário global.	✓

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

Análise do main.py

Objetivo do Arquivo

O arquivo `main.py` é o núcleo operacional do sistema embarcado CowHealth AI. Ele orquestra a execução paralela dos sensores, coordena a coleta e transmissão de dados, e garante a continuidade da aplicação embarcada após a fase de inicialização feita pelo `boot.py`. Além disso, realiza a integração com o gerenciador de conexão Wi-Fi e com o cliente MQTT para envio periódico dos dados dos sensores para o servidor.

Estrutura e Elementos Técnicos

O arquivo começa com os imports dos módulos essenciais, incluindo bibliotecas de `thread`, `tempo`, coleta de lixo, e objetos compartilhados do `config.py`. Importa os três módulos de sensores com nomes distintos:

- `acceleration_thread`: para dados de movimento (MPU6050)
- `temperature_thread`: para leitura térmica (MLX90614)
- `vitals_thread`: para frequência cardíaca (MAX30102)

Cada um destes módulos é inicializado como uma `thread`, garantindo concorrência leve e permitindo amostragem simultânea.

Na sequência, `main.py` tenta acessar os objetos globais `wifi_manager` e `datetime_manager` que teriam sido criados no `boot.py`. Caso não estejam disponíveis, o sistema entra em modo offline, imprimindo mensagens descritivas. Em seguida, o barramento I2C é escaneado com `i2c.scan()` e os dispositivos conectados são listados no terminal.

O gerenciador MQTT (MQTTManager) é inicializado e fica disponível para envio periódico de dados. A `main_thread()` central é responsável por:

- Imprimir o status atual da coleta
- Obter os dados dos sensores usando locks para acesso seguro entre threads
- Publicar os dados via MQTT
- Liberar memória com `gc.collect()`
- Aguardar com `time.sleep(MAIN_INTERVAL)`

O código ainda inclui a inicialização de cada thread de sensor usando `_thread.start_new_thread(...)`, garantindo a paralelização dos processos sensoriais em tempo quase-real.

Relação com Outros Arquivos do Projeto

- `config.py`: fornece os tempos de intervalo, estrutura `sensor_data`, e a instância do I2C
- `wifi_manager.py`: gerencia a conectividade e fornece status de rede
- `mqtt_client.py`: provê o objeto MQTTManager responsável pela publicação
- Módulos de sensores (`acceleration_thread`, `temperature_thread`, `vitals_thread`) são importados e executados como threads
- `datetime_manager.py`: fornece marcação temporal formatada para os dados transmitidos.

Relação com Sistemas Ciberfísicos

O arquivo `main.py` implementa o comportamento dinâmico de um nó inteligente em um sistema ciberfísico distribuído. Ele coleta dados do ambiente físico (temperatura,

movimento, batimentos cardíacos), os processa com controle concorrente e os envia a um servidor remoto para análise. Entre suas contribuições estão:

- Arquitetura paralela com threads para responsividade e eficiência;
- Comunicação assíncrona com servidor central via MQTT;
- Operação tolerante a falhas de conectividade (modo offline);
- Utilização de controle de tempo e economia de recursos computacionais (via gc.collect).

Tabela de Correlação com Requisitos e Aprendizagem

Tabela 7. Correlação com Requisitos e Aprendizagem (main.py)

Categoria	Elemento	Justificativa Técnica	Status
Requisito Funcional	RF03	Inicializa e executa <code>acceleration_thread</code> para captar atividade/postura.	✓
Requisito Funcional	RF04	Coordena o envio de dados via <code>MQTTManager</code> após verificar Wi-Fi.	✓
Requisito Funcional	RF05	Executa medições periódicas com intervalos definidos em <code>config.py</code> .	✓
Requisito Funcional	RF10	Possui estrutura para coleta contínua, mas não implementa buffer local.	⚠
Requisito Não Funcional	RNFM1	Alto grau de modularidade e paralelismo controlado.	✓
Resultado de Aprendizagem	RA1	Integra múltiplos sensores e coleta paralela de dados físicos.	✓
Resultado de Aprendizagem	RA4	Executa integração de sensores, rede e serviços web embarcados.	✓
Tema de Estudo	TE9	Exemplo completo de integração entre sistemas	✓

		embarcados, sensores e MQTT.	
Indicador de Desempenho	ID4.2	Coleta e estrutura dados dos sensores para persistência remota.	✓

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

Análise do sensors/acceleration_thread.py

Objetivo do Arquivo

O módulo `acceleration_thread.py` é responsável por inicializar e manter a coleta contínua dos dados do sensor MPU6050, que mede aceleração linear, velocidade angular (giroscópio) e temperatura do dispositivo. Sua execução ocorre em uma thread dedicada, permitindo que o monitoramento de atividade e postura das vacas seja feito em paralelo às demais tarefas do sistema embarcado.

Estrutura e Elementos Técnicos

A estrutura do código segue o padrão adotado pelos demais módulos de sensores, com três elementos principais:

- Importação e configuração: importa bibliotecas essenciais (`_thread`, `time`, `gc`) e os parâmetros `i2c`, `sensor_data`, `MPU_INTERVAL` de `config.py`;
- Inicialização: a função `init_sensor()` tenta instanciar o objeto MPU6050 (de `lib/mpu6050/mpu6050.py`) passando o barramento I2C configurado;
- Execução concorrente: a função `sensor_thread()` inicia a coleta em loop infinito, lendo acelerômetro (`read_accel_data()`), giroscópio (`read_gyro_data()`) e temperatura do sensor.

Os dados lidos são convertidos para tuplas (x, y, z) e armazenados de forma thread-safe usando um lock (mpu_lock). Os valores são inseridos na estrutura global sensor_data['mpu'], o que permite que outros módulos (como main.py ou mqtt_client.py) acessem as últimas leituras sem risco de condição de corrida. Entre cada leitura, a thread aguarda MPU_INTERVAL segundos, liberando memória via gc.collect() para evitar fragmentação ou estouro.

Relação com Outros Arquivos do Projeto

- config.py: fornece a instância i2c, a estrutura de armazenamento sensor_data e o intervalo MPU_INTERVAL;
- lib/mpu6050/mpu6050.py: implementa a classe MPU6050, com métodos para leitura de dados inerciais;
- main.py: importa sensor_thread e mpu_lock para executar em uma thread separada.

O uso de lock explicita uma boa prática de sincronização entre múltiplas threads concorrentes que acessam recursos compartilhados.

Relação com Sistemas Ciberfísicos

O módulo integra a camada de sensoriamento físico do sistema ciberfísico CowHealth AI. Ele traduz fenômenos biomecânicos (movimentos, postura, possíveis quedas) em dados digitais confiáveis, processados em tempo real. Entre suas contribuições aos princípios CPS estão:

- Medição contínua de variáveis físicas (aceleração, rotação, temperatura);
- Execução concorrente para responsividade temporal;
- Compartilhamento de dados sincronizados com múltiplos consumidores (ex: envio MQTT, análise local);
- Estabilização térmica e detecção de movimentos anômalos futuros.

Tabela de Correlação com Requisitos e Aprendizagem

Tabela 8. Correlação com Requisitos e Aprendizagem (sensors/acceleration_thread.py)

Categoria	Elemento	Justificativa Técnica	Status
Requisito Funcional	RF03	Executa a leitura contínua do MPU6050, fornecendo dados de postura e movimento.	✓
Requisito Funcional	RF05	Executa a coleta a cada MPU_INTERVAL com uso de sleep e controle temporal.	✓
Requisito Funcional	RF08	Permite detecção de anomalias por análise externa dos dados gravados em sensor_data.	⚠
Requisito Não Funcional	RNFE1	Opera com baixa latência e coleta periódica definida em configuração.	✓
Resultado de Aprendizagem	RA1	Integra sensor inercial via barramento I2C e coleta multieixo em tempo real.	✓
Tema de Estudo	TE7	Emprega sensores físicos embarcados para capturar variáveis de atividade.	✓
Indicador de Desempenho	ID1.2	Codifica leitura de sensores físicos de aceleração e rotação.	✓

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

Análise do sensors/temperature_thread.py

Objetivo do Arquivo

O módulo temperature_thread.py é responsável por inicializar e manter a leitura periódica da temperatura ambiente e da temperatura corporal do animal, utilizando o sensor infravermelho MLX90614. Esses dados são essenciais para detectar sinais de

febre, estresse térmico ou infecções. A execução ocorre em uma thread dedicada, permitindo operação concorrente e contínua, sem interferir nas demais rotinas do sistema embarcado.

Estrutura e Elementos Técnicos

A estrutura do código segue a mesma convenção dos demais sensores do sistema:

- Importa módulos de sistema (`_thread`, `time`, `gc`) e parâmetros globais de `config.py` (`i2c`, `sensor_data`, `MLX_INTERVAL`);
- Inicializa o sensor MLX90614 via função `init_sensor()`, que utiliza o endereço padrão 0x5A no barramento I2C;
- A função `sensor_thread()` realiza a coleta contínua dos dados térmicos com `read_ambient_temp()` e `read_object_temp()`;
- Os dados são arredondados para duas casas decimais, armazenados com segurança usando `mlx_lock` e inseridos na estrutura global `sensor_data['mlx']`;
- O ciclo se repete com intervalo `MLX_INTERVAL` e coleta de lixo com `gc.collect()` para manter a estabilidade da memória.

Relação com Outros Arquivos do Projeto

- `config.py`: fornece o objeto `i2c`, a estrutura `sensor_data` e o tempo de intervalo `MLX_INTERVAL`;
- `lib/mlx90614/mlx90614.py`: define a classe `MLX90614`, encapsulando os métodos de leitura de temperatura via I2C;
- `main.py`: importa `sensor_thread` e `mlx_lock` para execução em thread paralela e sincronização de acesso aos dados térmicos.

Relação com Sistemas Ciberfísicos

Este módulo implementa o elo direto entre o sistema computacional e as condições fisiológicas do animal. Por meio da medição da temperatura da superfície corporal e do ambiente, ele contribui para:

- Identificação de estados febris ou quadros infecciosos;
- Análise de estresse térmico em tempo real;
- Reatividade local do nó embarcado em ambiente rural de baixa supervisão contínua;
- Operação em baixa frequência de leitura, balanceando precisão e consumo energético.

Tabela de Correlação com Requisitos e Aprendizagem

Tabela 9. Correlação com Requisitos e Aprendizagem (sensors/temperature_thread.py)

Categoria	Elemento	Justificativa Técnica	Status
Requisito Funcional	RF01	Executa leitura contínua do MLX90614 e atualiza sensor_data['mlx'].	✓
Requisito Funcional	RF05	Executa com frequência parametrizada via MLX_INTERVAL.	✓
Requisito Funcional	RF07	Coleta dados críticos para detectar febre e alertar operadores.	⚠
Requisito Funcional	RF08	Base para inferência futura de padrões térmicos anômalos.	⚠
Requisito Não Funcional	RNFE1	Opera com latência controlada e sem travamentos em tempo real.	✓
Resultado de Aprendizagem	RA1	Integra sensor de temperatura corporal com leitura calibrada via I2C.	✓
Tema de Estudo	TE7	Emprega sensor físico MLX90614 para monitoramento clínico embarcado.	✓
Indicador de Desempenho	ID1.2	Codifica aquisição de dados térmicos do ambiente físico.	✓

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

Análise do sensors/vitals_thread.py

Objetivo do Arquivo

O módulo `vitals_thread.py` é responsável pela aquisição contínua de sinais fotopletismográficos (PPG) captados pelo sensor MAX30102, permitindo a estimativa da frequência cardíaca (bpm) e da saturação de oxigênio no sangue (SpO_2). Os dados são processados e armazenados para uso pelo sistema central, com potencial para detectar alterações agudas de padrão cardíaco. A coleta ocorre em uma thread dedicada com buffer circular simplificado.

Estrutura e Elementos Técnicos

O código está estruturado em três blocos funcionais principais:

- Instanciação do sensor via MAX30102 com intensidade ajustada (`MAX30105_PULSE_AMP_MEDIUM`);
- Classe `SimpleHeartRateMonitor`, que implementa um algoritmo simplificado para suavização de sinal, detecção de picos e cálculo de bpm;
- Função `sensor_thread()`, que coleta os dados continuamente, utiliza a função `get_heart_rate()` do sensor e atualiza a estrutura `sensor_data['max30102']` com `heart_rate` e `spo2`.

A coleta é protegida por `max_lock`, e o código implementa controle de tempo com `ticks_ms()` e `sleep()` para evitar sobrecarga e consumo excessivo de energia.

Relação com Outros Arquivos do Projeto

- `config.py`: fornece o objeto `i2c` e o dicionário compartilhado `sensor_data`;
- `lib/max30102/__init__.py`: implementa a classe MAX30102 e métodos `get_heart_rate()` e `get_spo2()`;

- main.py: importa sensor_thread e max_lock para execução paralela e envio dos dados via MQTT.

O módulo interage de forma isolada e segura com a camada sensorial cardíaca, sem interferência direta nos demais subsistemas.

Relação com Sistemas Ciberfísicos

A coleta contínua e em tempo quase-real dos sinais fisiológicos de batimento cardíaco e oxigenação torna este módulo uma das interfaces mais críticas entre o sistema digital e o organismo vivo. Sua importância inclui:

- Monitoramento de anomalias cardíacas (ex: taquicardia, bradicardia);
- Detecção precoce de febre sistêmica ou estados de hipóxia;
- Base para alarmes fisiológicos embarcados ou análise preditiva server-side;
- Uso de buffer temporal para garantir estabilidade da estimativa mesmo com ruídos ópticos.

Tabela de Correlação com Requisitos e Aprendizagem

Tabela 10. Correlação com Requisitos e Aprendizagem (sensors/vitals_thread.py)

Categoria	Elemento	Justificativa Técnica	Status
Requisito Funcional	RF02	Implementa coleta de frequência cardíaca com MAX30102.	✓
Requisito Funcional	RF05	Executa coleta periódica ajustável via temporizador interno.	✓
Requisito Funcional	RF08	Possui buffer para detectar oscilações cardíacas anormais.	⚠
Requisito Não Funcional	RNFE1	Executa com controle de tempo e leitura otimizada.	✓

Resultado de Aprendizagem	RA1	Integra sensor óptico com lógica de aquisição concorrente.	✓
Tema de Estudo	TE7	Exemplo prático de sensor biomédico em arquitetura embarcada.	✓
Indicador de Desempenho	ID1.2	Codifica leitura e cálculo de batimento cardíaco embarcado.	✓

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

Análise do utils/datetime_manager.py

Objetivo do Arquivo

O módulo `datetime_manager.py` encapsula a lógica de sincronização de tempo entre o relógio interno do microcontrolador ESP32 e servidores NTP (Network Time Protocol). Essa sincronização é fundamental para garantir que os dados dos sensores tenham carimbo temporal confiável, essencial para rastreabilidade, análise histórica e correlação entre eventos fisiológicos do animal. O módulo também oferece métodos auxiliares para leitura e formatação da data/hora local considerando o fuso horário configurado.

Estrutura e Elementos Técnicos

A estrutura do código gira em torno da classe `DateTimeManager`, composta por três métodos principais:

- `__init__(self, utc_offset=-3)`: define o deslocamento em segundos do UTC para horário local e inicializa a variável de estado `synced`;
- `sync_ntp(max_retries=3)`: tenta sincronizar o relógio via `ntptime.settime()`, com até 3 tentativas. Se bem-sucedido, marca `synced = True` e imprime o horário atual formatado;
- `_show_current_time()`: imprime no terminal a hora local usando `get_local_time()`;
- `get_local_time()`: retorna o horário local como tupla com ajuste de fuso;
- `get_timestamp()`: retorna a data/hora atual no formato `YYYY-MM-DD HH:MM:SS`, com suporte a ordenação e serialização em sistemas remotos.

O módulo utiliza internamente `utime` para controle e `ntptime` para sincronização via rede, tratando falhas com tolerância adequada.

Relação com Outros Arquivos do Projeto

- `boot.py`: instancia `DateTimeManager`, invoca `sync_ntp()` logo após conectar à internet;
- `main.py`: utiliza `datetime_manager.get_timestamp()` para registrar o momento da coleta e envio dos dados;
- `config.py`: define `TIMEZONE_OFFSET`, passado como argumento no construtor da classe.

Sua integração garante coerência temporal entre os dados dos sensores, fundamental para análises médicas e preditivas.

Relação com Sistemas Ciberfísicos

Este módulo implementa a base temporal do sistema ciberfísico distribuído. Ao garantir sincronização com fontes externas confiáveis (servidores NTP), ele permite

que múltiplos nós compartilhem contexto temporal global. Além disso, fornece resiliência por retentativa e fallback funcional. Os principais atributos são:

- Sincronização de tempo segura e repetível;
- Tratamento de exceções com continuidade de operação;
- Adaptação ao fuso horário local do rebanho;
- Formatação padronizada para uso por dashboards e bancos de dados remotos.

Tabela de Correlação com Requisitos e Aprendizagem

Tabela 11. Correlação com Requisitos e Aprendizagem (utils/datetime_manager.py)

Categoria	Elemento	Justificativa Técnica	Status
Requisito Funcional	RF04	Garante que os dados enviados via MQTT tenham carimbo temporal confiável.	✓
Requisito Funcional	RF05	Contribui para o registro periódico com timestamp local.	✓
Requisito Não Funcional	RNFE1	Sincronização de tempo eficiente e com fallback controlado.	✓
Requisito Não Funcional	RNFM1	Encapsulamento limpo da lógica de tempo e data.	✓
Resultado de Aprendizagem	RA4	Integra funcionalidades externas de sincronização de tempo via rede (IoT).	✓
Tema de Estudo	TE8	Ilustra uso de NTP e UTC como padrão temporal em sistemas embarcados.	✓
Indicador de Desempenho	ID4.3	Gera informações temporais consistentes para análise comparativa no servidor.	✓

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

Análise do utils/wifi_manager.py

Objetivo do Arquivo

O módulo `wifi_manager.py` implementa a lógica de gerenciamento de conectividade Wi-Fi da coleira inteligente CowHealth AI. Ele encapsula o controle de conexão, reconexão automática, bloqueio de concorrência entre threads e, opcionalmente, a sincronização periódica do relógio interno usando o gerenciador de data e hora (`datetime_manager`). Esta funcionalidade é essencial para garantir comunicação confiável em ambientes com cobertura intermitente.

Estrutura e Elementos Técnicos

A lógica é encapsulada na classe `WiFiManager`, com os seguintes elementos-chave:

- `__init__()`: define SSID, senha, número máximo de tentativas (`max_retries`), atraso entre tentativas (`retry_delay`) e associa opcionalmente ao `datetime_manager`;
- `connect(sync_ntp=True)`: ativa a interface de rede (`STA_IF`), verifica se já está conectada e tenta conexão com múltiplas tentativas controladas. Se for bem-sucedida e `datetime_manager` estiver presente, sincroniza o tempo via NTP usando `_sync_ntp_if_needed()`;
- `_sync_ntp_if_needed()`: sincroniza o relógio se já passou mais de 1 hora desde a última sincronização, evitando sobrecarga desnecessária;
- `is_wifi_connected()`: retorna o estado da conectividade atual de forma thread-safe.

Locks (`_thread.allocate_lock()`) são usados para proteger seções críticas durante conexão, especialmente em sistemas embarcados com multitarefa simplificada.

Relação com Outros Arquivos do Projeto

- - config.py: fornece WIFI_SSID, WIFI_PASSWORD, WIFI_MAX_RETRIES, WIFI_RETRY_DELAY, entre outros parâmetros usados no construtor;
- - datetime_manager.py: referenciado para garantir que a conexão Wi-Fi possa acionar sincronização de tempo NTP;
- - boot.py: instancia WiFiManager no processo de inicialização e avalia o sucesso da conexão;
- - main.py e mqtt_client.py: consultam o status de conectividade via is_wifi_connected() para condicionar envio de dados ou reconexão.

Essa abordagem promove desacoplamento e promove testabilidade e manutenção simples em longo prazo.

Relação com Sistemas Ciberfísicos

O wifi_manager.py é o responsável por estabelecer o elo digital entre o sistema físico (coleira embarcada) e a camada de computação em nuvem. Suas funções contribuem para:

- Comunicação autônoma com fallback controlado em ambientes com rede instável;
- Sincronismo temporal entre múltiplos nós e o backend;
- Isolamento de responsabilidade, mantendo o nó operacional mesmo diante de falhas de rede temporárias;
- Eficiência e estabilidade com uso inteligente de bloqueios e reconexão limitada.

Tabela de Correlação com Requisitos e Aprendizagem

Tabela 12. Correlação com Requisitos e Aprendizagem (utils/wifi_manager.py)

Categoria	Elemento	Justificativa Técnica	Status
Requisito Funcional	RF04	Gerencia reconexão Wi-Fi e integra com sincronismo de tempo NTP.	✓
Requisito Funcional	RF06	Verifica conectividade, mas não implementa	⚠

		monitoramento ativo de falha de sensor.	
Requisito Funcional	RF10	Permite detectar perda de rede, mas não implementa buffer local de persistência.	⚠
Requisito Não Funcional	RNFE1	Conexão eficiente com verificação de estado e reconexão escalonada.	✅
Requisito Não Funcional	RNFM1	Isolamento de responsabilidades, encapsulamento modular completo.	✅
Resultado de Aprendizagem	RA4	Gerencia camada de comunicação sem fio embarcada com lógica robusta.	✅
Tema de Estudo	TE8	Trabalha com redes Wi-Fi e sincronização embarcada em IoT.	✅
Indicador de Desempenho	ID4.1	Codifica controle de interface de comunicação sem fio e lógica de fallback.	✅

Legenda:

❌ Ausente | ⚠ Parcial | ✅ Implementado

Análise do utils/mqtt_client.py

Objetivo do Arquivo

O módulo mqtt_client.py implementa a interface de comunicação entre o sistema embarcado e a infraestrutura de dados remota, utilizando o protocolo MQTT. Ele é responsável por construir, serializar e publicar os dados coletados pelos sensores, operando em paralelo ao restante do sistema embarcado. Além disso, implementa

contadores de erros, controle de reconexões e intervalos temporais configuráveis para envio periódico de informações de telemetria.

Estrutura e Elementos Técnicos

O módulo define a classe MQTTManager, com os seguintes componentes-chave:

- `__init__()`: carrega parâmetros do `config.py`, inicializa atributos de conexão, status, contadores e tentativa de conexão com `wifi_manager` e `datetime_manager` globais;
- `_check_wifi_connectivity()`: verifica se o Wi-Fi está conectado antes de publicar mensagens. Em caso de ausência do `wifi_manager`, continua operação de forma tolerante;
- `_connect()`: instancia o cliente MQTT com `umqtt.simple.MQTTClient` e conecta ao broker;
- `_publish(payload)`: envia o payload ao tópico configurado. Gerencia exceções e incrementa `error_count` em caso de falha;
- `publish_json(data_dict)`: empacota os dados dos sensores em JSON, adiciona campos opcionais (timestamp, wifi, device_id) conforme o `MQTT_PAYLOAD_CONFIG` e invoca `_publish()`;
- `loop_thread()`: executa em segundo plano, verificando se o intervalo `MQTT_INTERVAL` foi alcançado e publica automaticamente o conteúdo armazenado no `sensor_data`.

Relação com Outros Arquivos do Projeto

- `config.py`: fornece o broker, client ID, tópicos MQTT e flags de payload;
- `main.py`: instancia o `MQTTManager`, chama `publish_json()` e executa a thread `loop_thread()`;
- `wifi_manager.py` e `datetime_manager.py`: integrados para verificação de conectividade e marcação temporal nos dados publicados.

O design é modular, extensível e tolerante a falhas – características fundamentais em sistemas distribuídos e remotos.

Relação com Sistemas Ciberfísicos

O `mqtt_client.py` atua como ponte entre o mundo físico e o ambiente computacional em nuvem. Ele viabiliza a entrega dos dados vitais do animal de forma contínua, eficiente e estruturada. Desta forma, contribui para:

- Manutenção da sincronia entre sensores embarcados e análise remota;
- Eficiência na rede por uso de protocolo leve e formato JSON compacto;
- Diagnóstico e rastreabilidade com contadores de mensagens e erros;
- Robustez operacional mesmo sem Wi-Fi ou RTC sincronizado (fallback resiliente).

Tabela de Correlação com Requisitos e Aprendizagem

Tabela 13. Correlação com Requisitos e Aprendizagem (`utils/mqtt_client.py`)

Categoria	Elemento	Justificativa Técnica	Status
Requisito Funcional	RF04	Publica os dados dos sensores via protocolo MQTT com controle de tempo.	✓
Requisito Funcional	RF10	Permite envio periódico de dados, mas não armazena localmente se offline.	⚠
Requisito Funcional	RF14	Serve de base para alertas em tempo real via MQTT com backend conectado.	⚠
Requisito Não Funcional	RNFE1	Utiliza intervalos e verificação de conectividade com reconexão segura.	✓
Requisito Não Funcional	RNFM1	Implementa encapsulamento limpo, desacoplado e reutilizável.	✓
Resultado de Aprendizagem	RA4	Integra comunicação IoT com controle de exceções e tolerância a falhas.	✓
Tema de Estudo	TE8	Aplica protocolo MQTT leve para sistemas embarcados.	✓

Indicador de Desempenho	ID4.3	Realiza publicação regular e confiável dos dados sensoriais com log.	✓
-------------------------	-------	--	---

Legenda:

✗ Ausente | ⚠ Parcial | ✓ Implementado

Resumo Técnico – Bibliotecas de Terceiros Utilizadas no Projeto CowHealth AI

lib/mpu6050/mpu6050.py

Este módulo implementa uma interface para o sensor inercial **MPU6050**, que combina acelerômetro e giroscópio em um único chip com comunicação I2C. A biblioteca oferece funções como:

- `get_values()` – retorna um dicionário com aceleração e giroscópio em x, y, z, além da temperatura do chip;
- Inicialização via `MPU6050(i2c)` e leitura contínua dos dados brutos convertidos em unidades físicas;
- Internamente, faz cálculos de offset e calibração básica, adequando os registros do sensor ao uso embarcado com ESP32.

Essa biblioteca permite extrair informações críticas para detecção de marcha, postura e possíveis quedas ou movimentos irregulares do animal.

lib/mlx90614/mlx90614.py

Este arquivo fornece a interface para comunicação com o sensor infravermelho **MLX90614**, utilizado para leitura sem contato de temperatura. As principais funcionalidades expostas são:

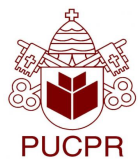
- `read_ambient_temp()` – leitura da temperatura ambiente no local do sensor;
- `read_object_temp()` – leitura da temperatura da superfície do corpo do animal;
- Toda comunicação é feita via I2C, com leitura dos registradores internos conforme o datasheet do fabricante.

A biblioteca encapsula as conversões necessárias dos dados binários recebidos (em 0.02 °C por bit) e entrega valores em ponto flutuante com boa precisão.

lib/max30102/init.py

Este módulo é a implementação da interface para o sensor **MAX30102/MAX30105**, destinado à coleta de sinais ópticos para medição de frequência cardíaca e SpO₂. Essa biblioteca é a mais complexa das três e implementa:

- Configuração de modos operacionais (LEDs ativos, taxa de amostragem, largura de pulso);
- Leitura dos registros FIFO para recuperar sinais de luz refletida (vermelha, infravermelha, verde);
- Gerenciamento de interrupções, modo de espera (shutdown) e reinicialização do sensor;
- Estimativas de temperatura do chip e controle do número de amostras na FIFO;
- Métodos como `get_red()`, `get_ir()` e `check()` que permitem a coleta de amostras em tempo real.



Documentação Técnica do Projeto

A biblioteca é robusta e compatível com ambientes embarcados de baixo consumo, sendo essencial para o subsistema de sinais vitais do projeto.

6. Arquitetura de Software Embarcado do Sistema CowHealth AI

6.1. Escolha do framework do protótipo

A escolha do framework Laravel, em conjunto com o painel administrativo Filament, reflete uma decisão estratégica fundamentada na autonomia, iniciativa técnica e audácia dos estudantes envolvidos no projeto CowHealth AI.

Em um contexto educacional voltado à formação integral em Ciência da Computação, os discentes buscaram não apenas uma solução funcional, mas uma arquitetura escalável, segura e aderente às boas práticas da indústria.

6.1.1. Motivação e Aspectos Técnicos

Laravel é atualmente um dos frameworks mais maduros, robustos e bem documentados do ecossistema PHP. Sua estrutura baseada no padrão MVC (Model-View-Controller), integração com ORM (Eloquent), sistema de rotas, middlewares, autenticação e suporte a filas e jobs assíncronos, o torna especialmente adequado para aplicações de Internet das Coisas (IoT) com necessidade de recepção, processamento e visualização de dados em tempo real.

A adoção do Filament (um plugin administrativo moderno para Laravel) demonstra a preocupação dos alunos com a produtividade, organização visual, controle de permissões e separação de papéis (RBAC – Role-Based Access Control), elementos essenciais em sistemas sensíveis como o monitoramento de saúde animal.

6.1.2. Relação com o Plano de Ensino e Competências Desenvolvidas

Esta escolha evidencia o atendimento direto a diversos Resultados de Aprendizagem (RA), Temas de Estudo (TE) e Indicadores de Desempenho (ID), tais como:

- RA4: Domínio de tecnologias web e embarcadas em ambientes integrados;
- RA4: Capacidade de desenvolver sistemas computacionais integrando diversas camadas (back-end, banco de dados, UI);
- TE8: Tecnologias de IoT e plataformas distribuídas com protocolos MQTT, serviços de rede, APIs e autenticação segura;
- TE9: Interoperabilidade de sistemas, uso de bancos relacionais e dashboards gerenciais;
- ID4.1: Capacidade de construir aplicações que se comuniquem com sistemas embarcados;
- ID4.3: Aplicação de conceitos de software de middleware, segurança e governança de dados em ambientes IoT.

Além disso, a escolha reflete alinhamento com práticas profissionais contemporâneas, sendo Laravel amplamente utilizado em startups, plataformas SaaS e sistemas de missão crítica no Brasil e no exterior.

6.2. Estrutura de Arquivos

A estrutura da plataforma web do projeto CowHealth AI reflete um alto nível de organização arquitetural, aderindo rigorosamente ao padrão MVC (Model-View-Controller) oferecido pelo framework Laravel. Os diretórios estão logicamente separados entre modelos de domínio (Models), controladores (Http/Controllers), recursos administrativos (Filament/Resources), e serviços especializados (Services e Jobs).

A presença de Enums e Polícies indica uma preocupação clara com legibilidade, governança de acesso e manutenção do código. A extensão do painel administrativo via Filament demonstra maturidade técnica na composição de interfaces reativas e responsivas, promovendo usabilidade sem comprometer a robustez.

Essa segmentação modular não apenas favorece a reutilização de componentes, mas também facilita futuras expansões, como integração de algoritmos de IA, sistemas externos via API ou novos sensores. Em síntese, trata-se de uma estrutura que sustenta os princípios da engenharia de software orientada a domínio, com alto grau de separação de responsabilidades e potencial de escalabilidade.

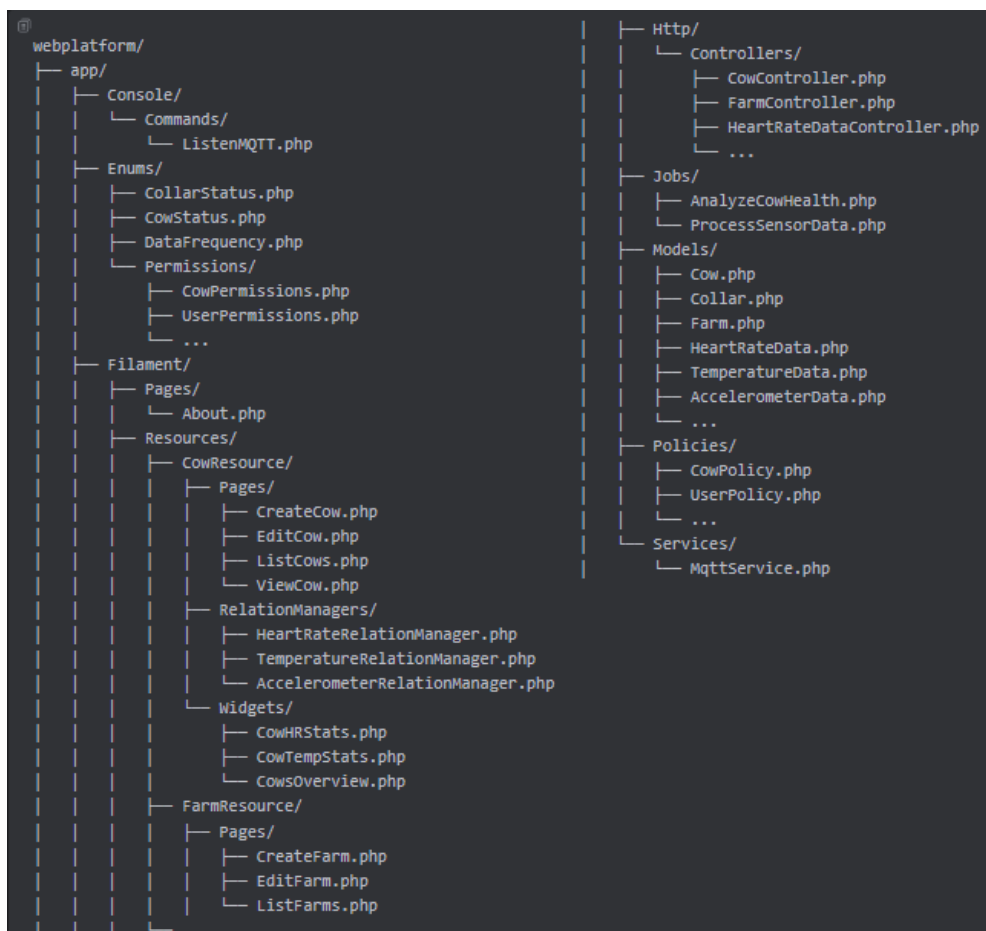


Figura 2. Estrutura de Arquivos da Plataforma Web

A estrutura da plataforma web do projeto **CowHealth AI**, implementada em Laravel, está altamente modularizada e bem organizada segundo os princípios da engenharia de software web. Ela está dividida nos seguintes domínios principais:

Principais Componentes

1. Models (app/Models/)

Modelam as entidades centrais do domínio:

- Cow, Collar, Farm, User, Role, Permission

- Dados sensoriais: HeartRateData, TemperatureData, AccelerometerData

2. Controllers (app/Http/Controllers/)

Responsáveis pelo controle da lógica de negócios exposta via rotas (REST ou Filament):

- Atores principais (CowController, FarmController, UserController)
- Sensores específicos (HeartRateDataController, AccelerometerDataController, etc.)

3. Filament Resources (app/Filament/Resources/)

São os recursos administrativos da interface Filament:

- Recursos para CRUD de cada entidade (Cow, Collar, User, etc.)
- Separação clara de páginas (Create, Edit, List, View)
- RelationManagers para mostrar relações entre entidades (ex: Cow → Temperatura)
- Widgets de estatísticas: CowHRStats, CowTempStats, CowsOverview

4. Services (app/Services/)

Contém lógica especializada, como o MqttService.php que integra a recepção de dados dos dispositivos.

5. Jobs (app/Jobs/)

Processamento assíncrono com AnalyzeCowHealth.php e ProcessSensorData.php.

6. Enums (app/Enums/)

Expressam estados e permissões como:

- CowStatus, CollarStatus, DataFrequency
- Enumerações de permissões: UserPermissions, CowPermissions, etc.

7. Policies (app/Policies/)

Implementam controle de acesso granular com base em perfis e permissões.

8. Providers (app/Providers/)

Registram serviços no container Laravel, como Filament\AppPanelProvider.

9. Console Commands (app/Console/Commands/)

Comando customizado ListenMQTT.php para escutar o broker MQTT e despachar jobs.

Essa estrutura demonstra a preocupação dos estudantes com:

- Separação de responsabilidades (SoC)
- Aderência a padrões de projeto MVC
- Clareza no controle de acesso
- Escalabilidade futura para integração com IA (via Jobs)
- Flexibilidade de visualização (via Filament)

6.3. Correlação da Plataforma Web e Resultados de Aprendizagem

De acordo com o Plano de Ensino da disciplina, espera-se que os estudantes desenvolvam aplicações web robustas, seguras e moduladas, capazes de integrar-se com dispositivos físicos, bancos de dados relacionais e múltiplos perfis de usuários. A plataforma web do projeto CowHealth AI, implementada com Laravel e Filament, exemplifica com excelência esses objetivos, articulando a camada de apresentação com lógica de negócios, segurança

baseada em papéis (RBAC), visualização interativa de dados e integração em tempo real com o sistema embarcado.

A arquitetura modular adotada, baseada em padrões de engenharia como MVC e injeção de dependência, favorece a escalabilidade e manutenção do sistema, enquanto os recursos administrativos permitem que usuários com diferentes níveis de conhecimento técnico operem a solução com autonomia.

Essa realização evidencia a aplicação prática dos conteúdos da disciplina, consolidando competências essenciais em desenvolvimento de software para sistemas ciberfísicos.

RA2 – Desenvolver aplicações com arquitetura multicamadas, conectando frontend, backend e base de dados

Temas relacionados:

- **TE4:** Arquitetura MVC e organização modular de sistemas web
- **TE6:** Integração com bancos de dados relacionais
- **TE7:** Segurança e controle de acesso em sistemas de informação

Contribuições do Projeto CowHealth AI:

A plataforma web foi inteiramente construída com o framework Laravel, seguindo rigorosamente o padrão arquitetural MVC. A camada de apresentação foi implementada com o painel administrativo Filament, permitindo uma separação clara entre lógica de negócios

(Controllers), modelos de dados (Models) e visualizações (Views). A autenticação, autorização e controle de acesso (Policies) também foram implementados de forma robusta, refletindo boas práticas de engenharia de software web.

Indicadores atendidos:

- **ID2.1:** Estruturação de sistemas com separação clara de camadas
- **ID4.3:** Desenvolvimento de sistemas com tolerância a falhas

RA3 – Analisar, implementar e validar soluções computacionais orientadas a dados

Temas relacionados:

- **TE6:** Integração com bancos de dados relacionais
- **TE9:** Interoperabilidade, exportação/importação de dados

Contribuições do Projeto CowHealth AI:

O sistema armazena e organiza grandes volumes de dados fisiológicos coletados das vacas, como temperatura, frequência cardíaca e aceleração. Os dados são persistidos em tabelas normalizadas, associados a cada animal, e disponibilizados ao produtor e veterinário por meio de painéis gráficos, tabelas com filtros temporais, e relatórios exportáveis. Embora a plataforma ofereça mecanismos diretos de exportação dos dados em formato CSV, sendo apta a interoperar com sistemas externos, isso ainda não foi implementado para esta fase do projeto

Indicadores atendidos:

- **ID3.1:** Coleta e análise de dados reais
- **ID3.2:** Apresentação de dados com visualização interativa

RA4 – Projetar sistemas que integrem software embarcado, redes de sensores e interface web

Temas relacionados:

- **TE8:** IoT e sistemas conectados com protocolos de comunicação
- **TE9:** Interoperabilidade, exportação/importação de dados

Contribuições do Projeto CowHealth AI:

A plataforma web implementa um listener MQTT (comando ListenMQTT.php) que recebe os dados diretamente dos dispositivos embarcados. Essa integração se dá de forma assíncrona, via protocolo MQTT, com persistência em banco de dados relacional e exibição na interface gráfica. Há também serviços (MqttService) e filas (AnalyzeCowHealth) que demonstram o fluxo completo entre captura física, processamento lógico e exibição em tela.

Indicadores atendidos:

- **ID4.1:** Integração com dispositivos e redes de sensores
- **ID4.3:** Desenvolvimento de sistemas com tolerância a falhas

Análise dos arquivos da Plataforma Web

Caminho	Nome do Arquivo	Função
app/Console/Commands	ListenMQTT.php	Comando de terminal customizado (CLI Laravel)
app/Enums	CollarStatus.php	Enumeração para status, permissões ou tipos
app/Enums	CowStatus.php	Enumeração para status, permissões ou tipos
app/Enums	DataFrequency.php	Enumeração para status, permissões ou tipos
app/Enums	TipoPermissions.php	Enumeração para status, permissões ou tipos
app/Enums	UserPerfil.php	Enumeração para status, permissões ou tipos
app/Enums/Permissions	CollarPermissions.php	Enumeração para status, permissões ou tipos
app/Enums/Permissions	CowPermissions.php	Enumeração para status, permissões ou tipos
app/Enums/Permissions	FarmPermissions.php	Enumeração para status, permissões ou tipos
app/Enums/Permissions	UserPermissions.php	Enumeração para status, permissões ou tipos
app/Filament/Pages	About.php	Página customizada do painel Filament
app/Filament/Resources	CollarResource.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources	CowResource.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources	FarmResource.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources	PermissionGroupResource.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources	PermissionResource.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources	RoleResource.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources	UserResource.php	Recurso administrativo Filament (CRUD visual)

app/Filament/Resources/CollarResource/Pages	CreateCollar.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CollarResource/Pages	EditCollar.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CollarResource/Pages	ListCollars.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/Pages	CreateCow.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/Pages	EditCow.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/Pages	ListCows.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/Pages	ViewCow.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/RelationManagers	AccelerometerRelationManager.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/RelationManagers	HeartRateRelationManager.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/RelationManagers	TemperatureRelationManager.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/Widgets	CowHRStats.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/Widgets	CowTempStats.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/CowResource/Widgets	CowsOverview.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/FarmResource/Pages	CreateFarm.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/FarmResource/Pages	EditFarm.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/FarmResource/Pages	ListFarms.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/PermissionGroupResource/Pages	CreatePermissionGroup.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/PermissionGroupResource/Pages	EditPermissionGroup.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/PermissionGroupResource/Pages	ListPermissionGroups.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/PermissionGroupResource/RelationManagers	PermissionsRelationManager.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/PermissionResource/Pages	CreatePermission.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/PermissionResource/Pages	EditPermission.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/PermissionResource/Pages	ListPermissions.php	Recurso administrativo Filament (CRUD visual)

app/Filament/Resources/RoleResource/Pages	CreateRole.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/RoleResource/Pages	EditRole.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/RoleResource/Pages	ListRoles.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/RoleResource/RelationManagers	PermissionsRelationManager.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/RoleResource/RelationManagers	UsersRelationManager.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/UserResource/Pages	CreateUser.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/UserResource/Pages	EditUser.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/UserResource/Pages	ListUsers.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Resources/UserResource/RelationManagers	RolesRelationManager.php	Recurso administrativo Filament (CRUD visual)
app/Filament/Widgets	CowsPerFarm.php	Widget de estatísticas no dashboard
app/Filament/Widgets	CowsPerStatus.php	Widget de estatísticas no dashboard
app/Http/Controllers	AccelerometerDataController.php	Controlador HTTP (lida com requisições web)
app/Http/Controllers	CollarController.php	Controlador HTTP (lida com requisições web)
app/Http/Controllers	Controller.php	Controlador HTTP (lida com requisições web)
app/Http/Controllers	CowController.php	Controlador HTTP (lida com requisições web)
app/Http/Controllers	FarmController.php	Controlador HTTP (lida com requisições web)
app/Http/Controllers	HeartRateDataController.php	Controlador HTTP (lida com requisições web)
app/Http/Controllers	SensorsDataController.php	Controlador HTTP (lida com requisições web)
app/Http/Controllers	TemperatureDataController.php	Controlador HTTP (lida com requisições web)
app/Jobs	AnalyzeCowHealth.php	Job assíncrono (processamento em segundo plano)
app/Jobs	ProcessSensorData.php	Job assíncrono (processamento em segundo plano)
app/Models	AccelerometerData.php	Modelo de dados (entidade do domínio)
app/Models	Collar.php	Modelo de dados (entidade do domínio)

app/Models	Cow.php	Modelo de dados (entidade do domínio)
app/Models	Farm.php	Modelo de dados (entidade do domínio)
app/Models	HeartRateData.php	Modelo de dados (entidade do domínio)
app/Models	Permission.php	Modelo de dados (entidade do domínio)
app/Models	PermissionGroup.php	Modelo de dados (entidade do domínio)
app/Models	Role.php	Modelo de dados (entidade do domínio)
app/Models	TemperatureData.php	Modelo de dados (entidade do domínio)
app/Models	User.php	Modelo de dados (entidade do domínio)
app/Policies	CollarPolicy.php	Política de autorização e controle de acesso
app/Policies	CowPolicy.php	Política de autorização e controle de acesso
app/Policies	FarmPolicy.php	Política de autorização e controle de acesso
app/Policies	PermissionGroupPolicy.php	Política de autorização e controle de acesso
app/Policies	PermissionPolicy.php	Política de autorização e controle de acesso
app/Policies	RolePolicy.php	Política de autorização e controle de acesso
app/Policies	UserPolicy.php	Política de autorização e controle de acesso
app/Providers	AppServiceProvider.php	Classe de registro de serviços (Service Provider)
app/Providers/Filament	AppPanelProvider.php	Classe de registro de serviços (Service Provider)
app/Services	MqttService.php	Serviço de integração ou lógica de negócio

6.5. Telas da Plataforma Web

As telas da plataforma web do projeto CowHealth AI foram desenvolvidas com foco na clareza visual, acessibilidade e usabilidade por diferentes perfis de usuários — incluindo produtores, veterinários e administradores. A interface foi construída com o framework Filament, que permite uma experiência fluida e responsiva, tanto em dispositivos desktop quanto móveis.

Cada tela foi projetada para refletir a estrutura lógica do domínio do sistema, facilitando o acompanhamento da saúde dos animais em tempo real, o acesso histórico de dados fisiológicos e a tomada de decisões baseada em evidências.

A seguir, são apresentadas as principais telas do sistema, evidenciando os fluxos de navegação, os recursos disponíveis por perfil e os elementos de visualização implementados para garantir a efetividade da plataforma no contexto agropecuário inteligente.

Grupo das Telas iniciais

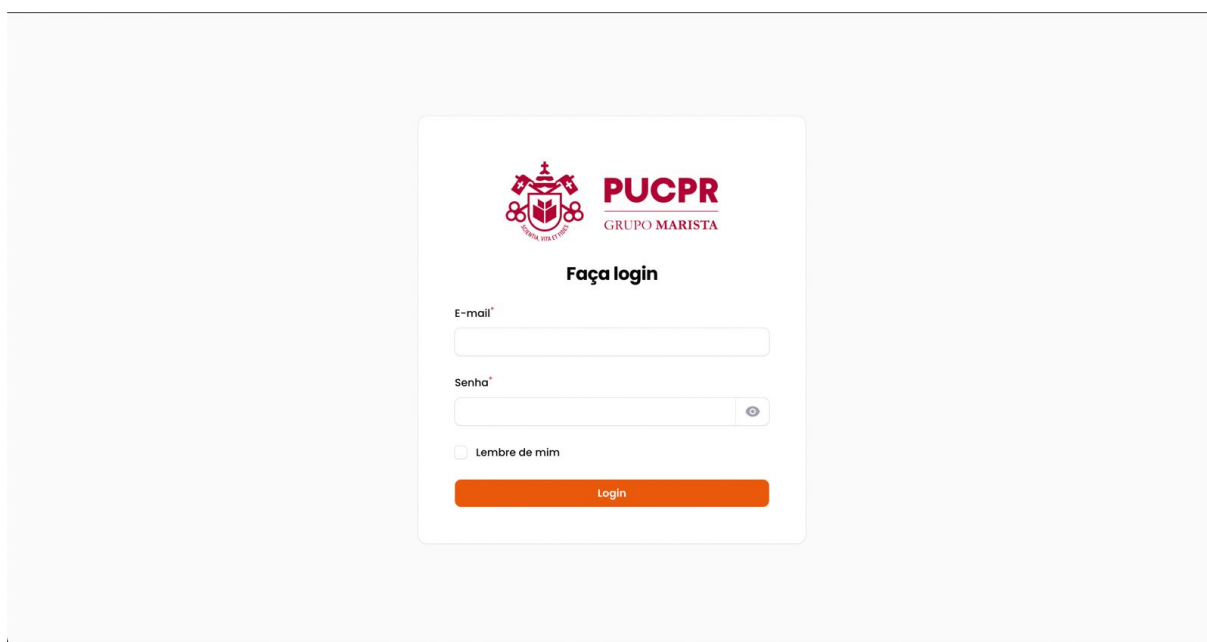


Figura 3. Tela de login

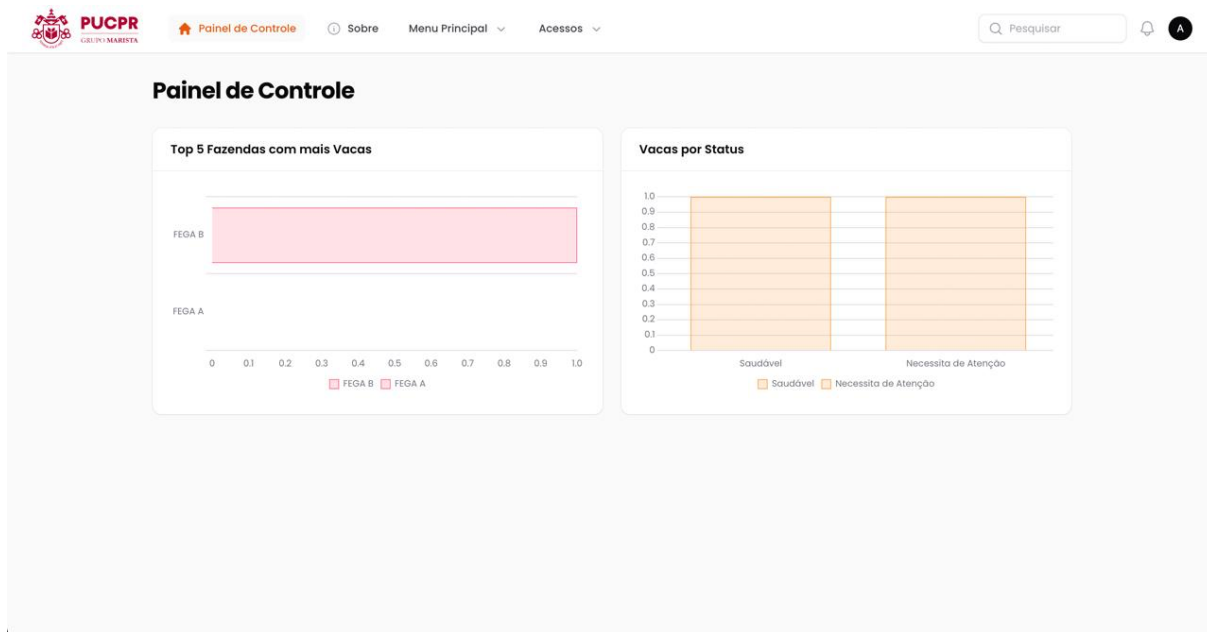


Figura 4. Tela inicial (Modo Claro)

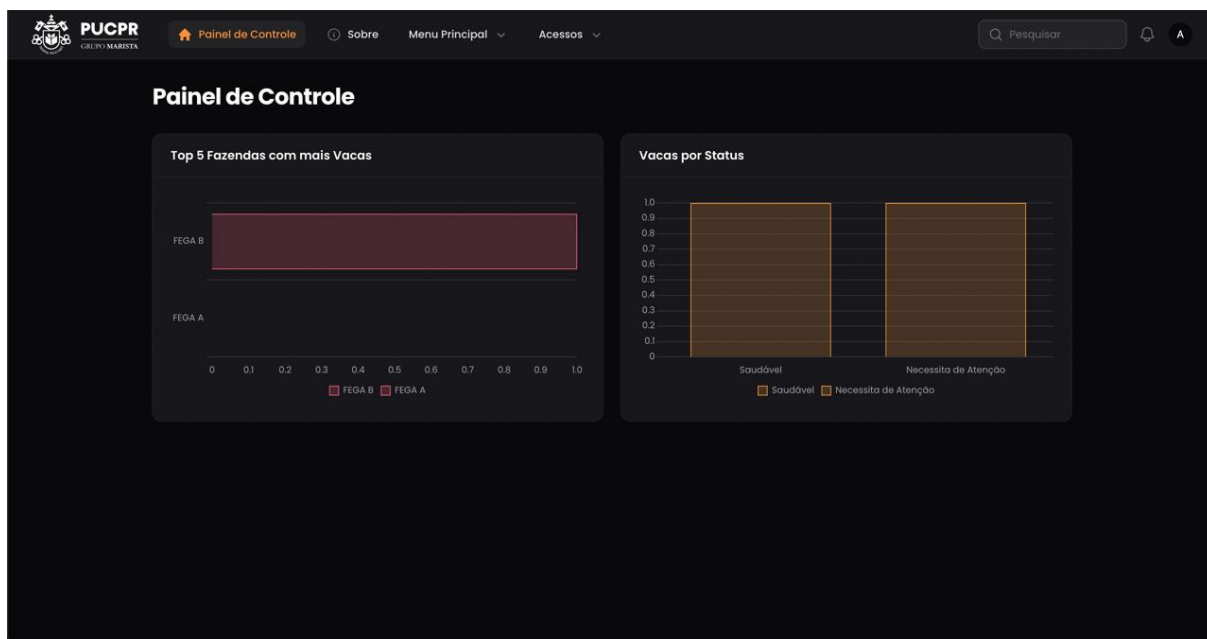


Figura 5. Tela Inicial (Modo Escuro)

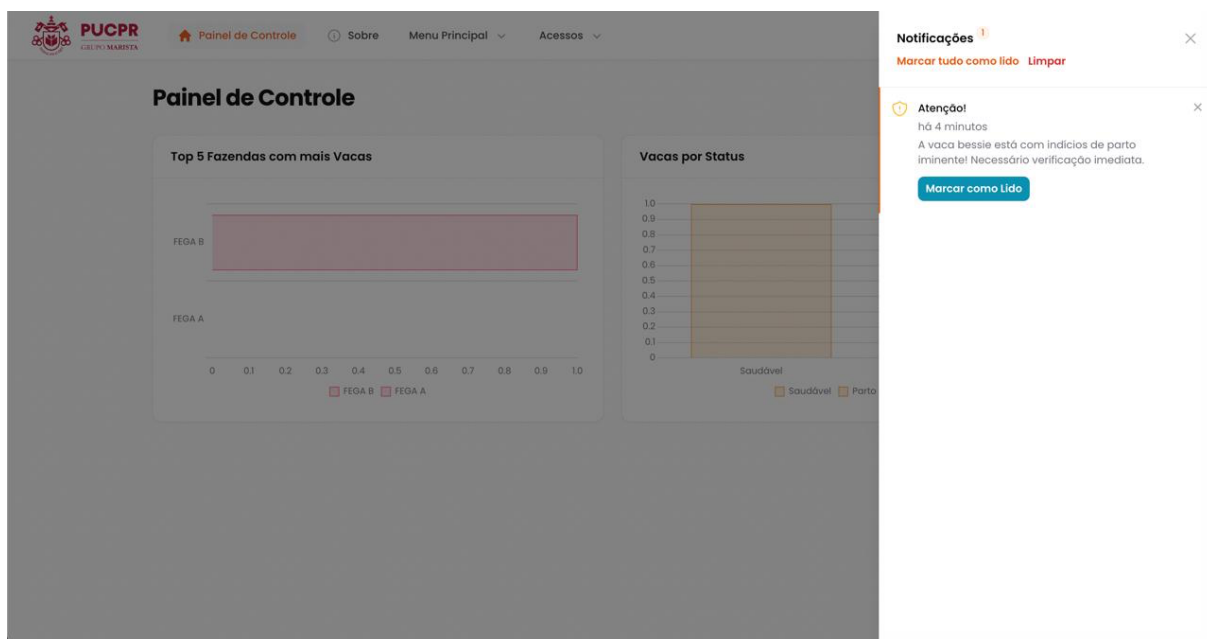


Figura 6. Notificações

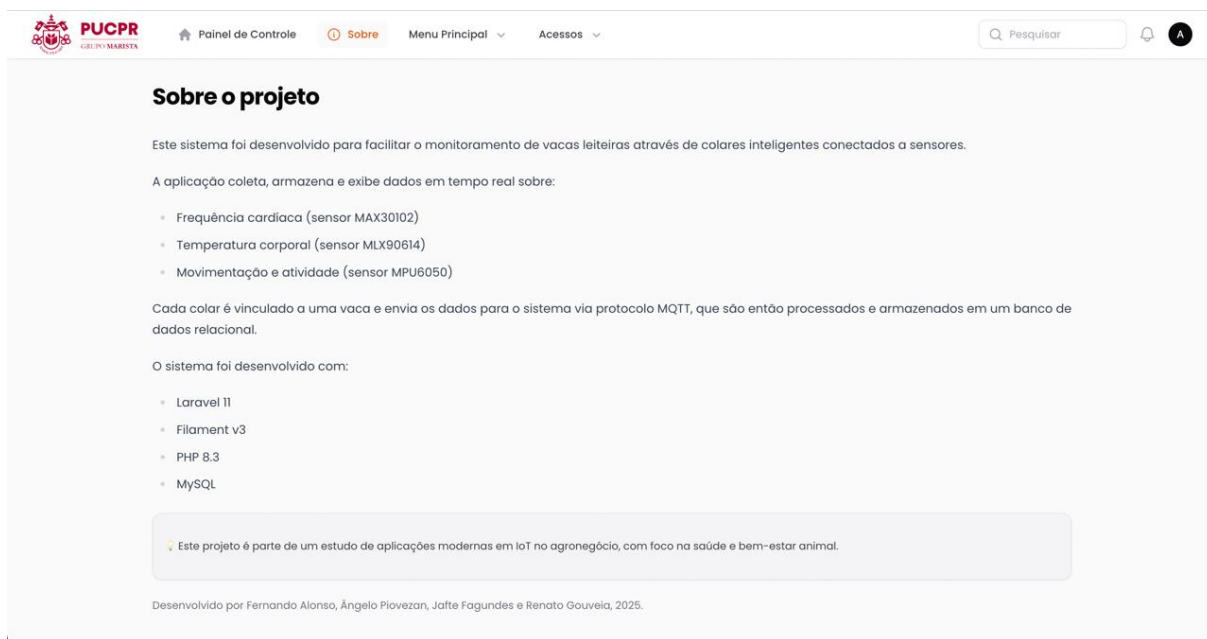


Figura 7. Sobre o Sistema

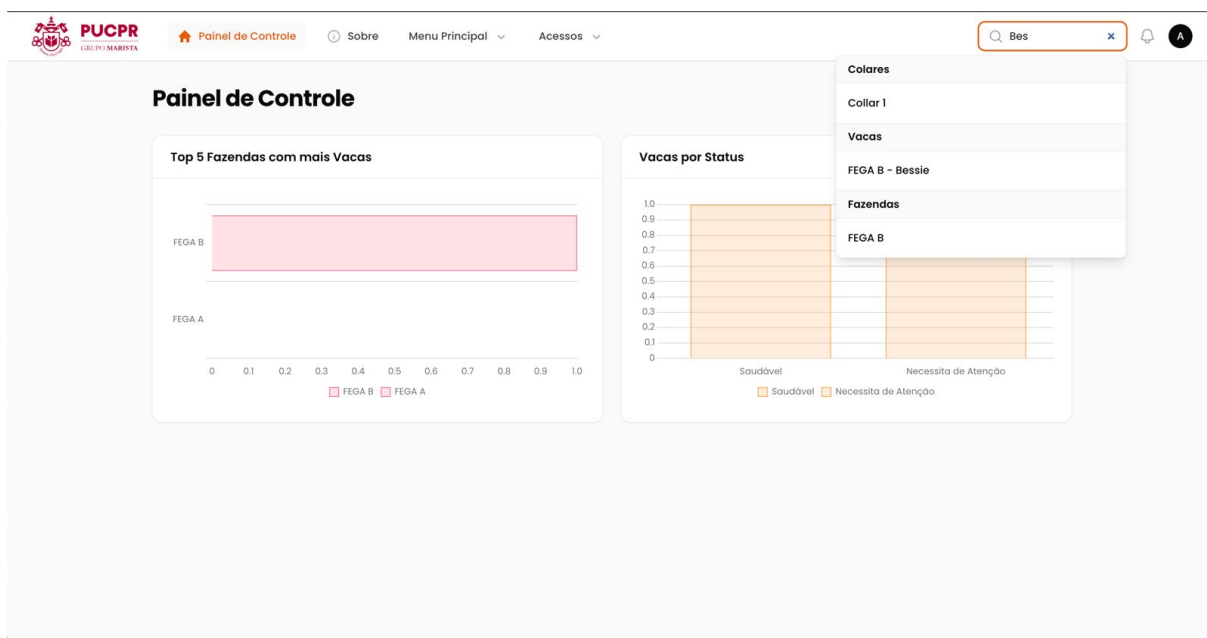
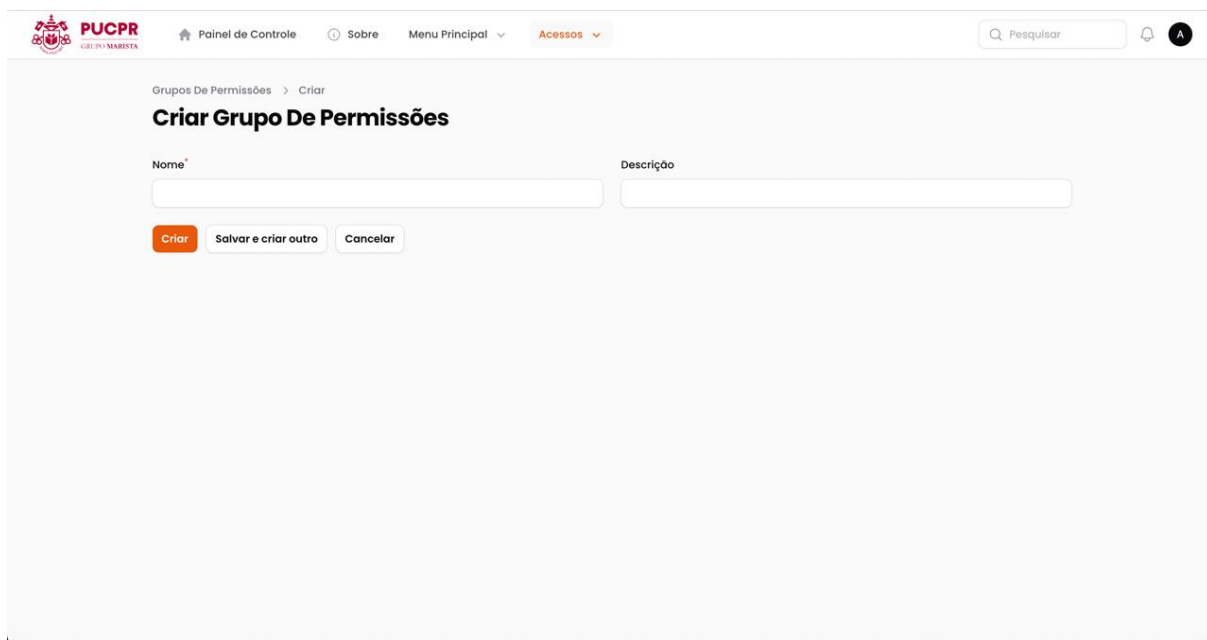
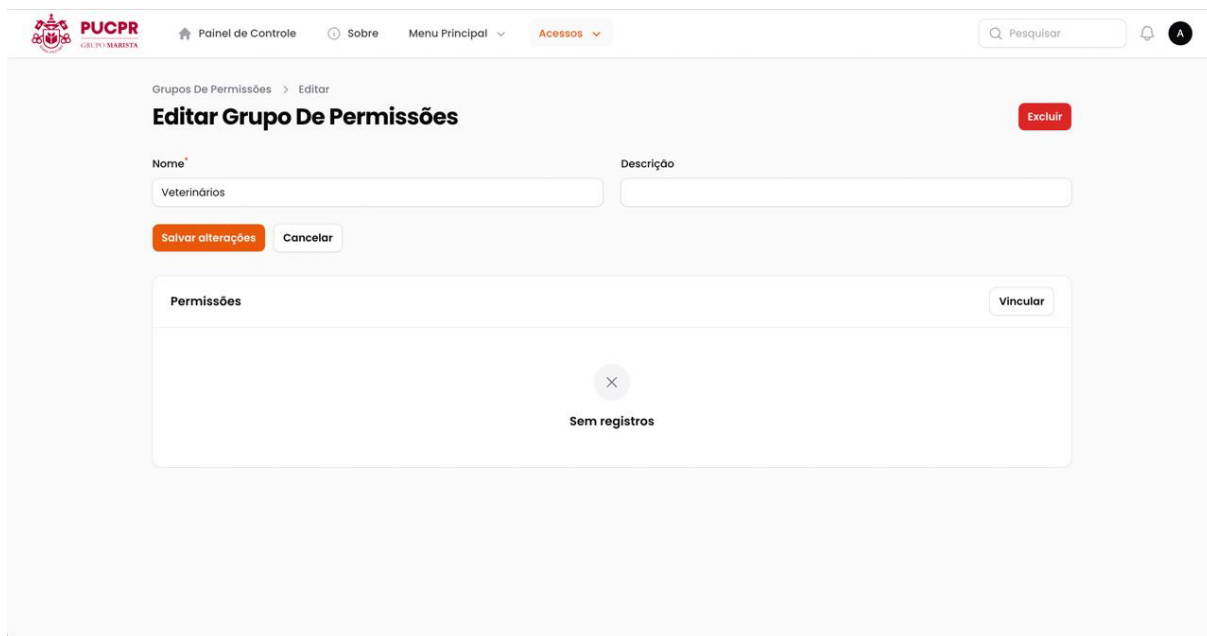


Figura 8. Busca global

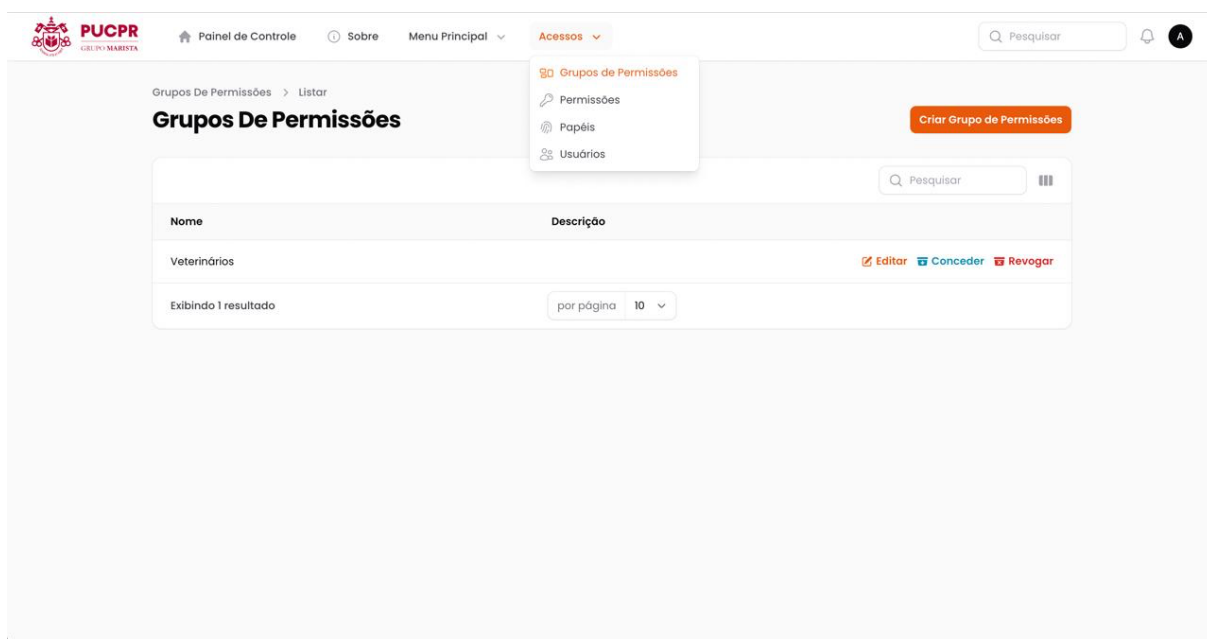
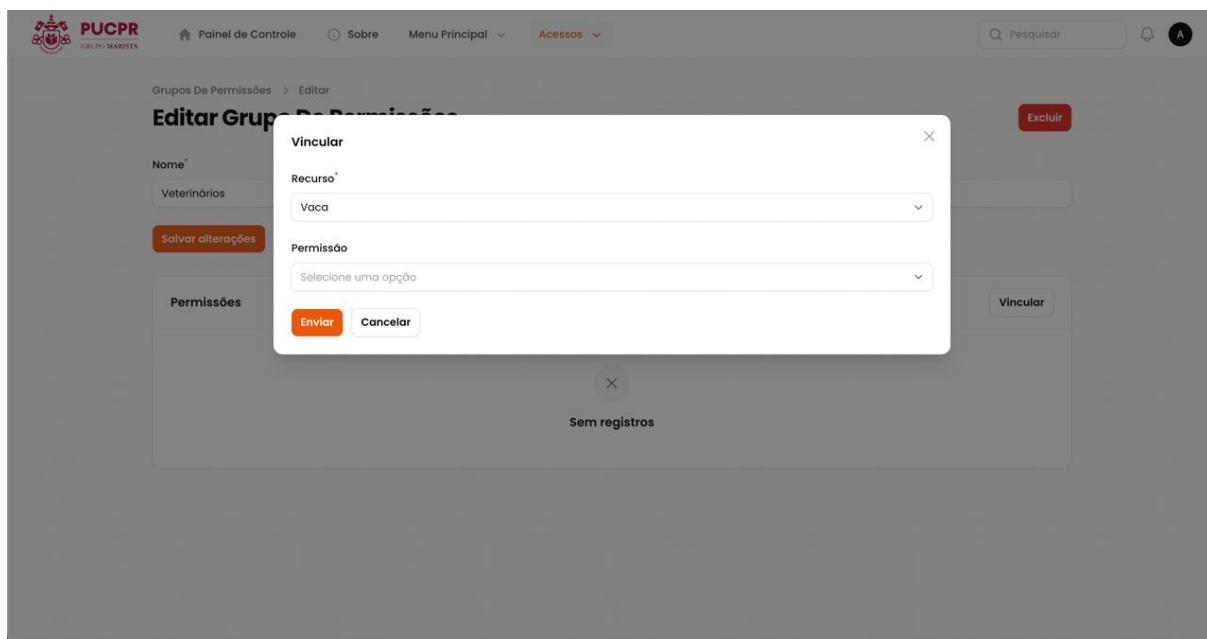
Grupo das Telas Grupo Permissões

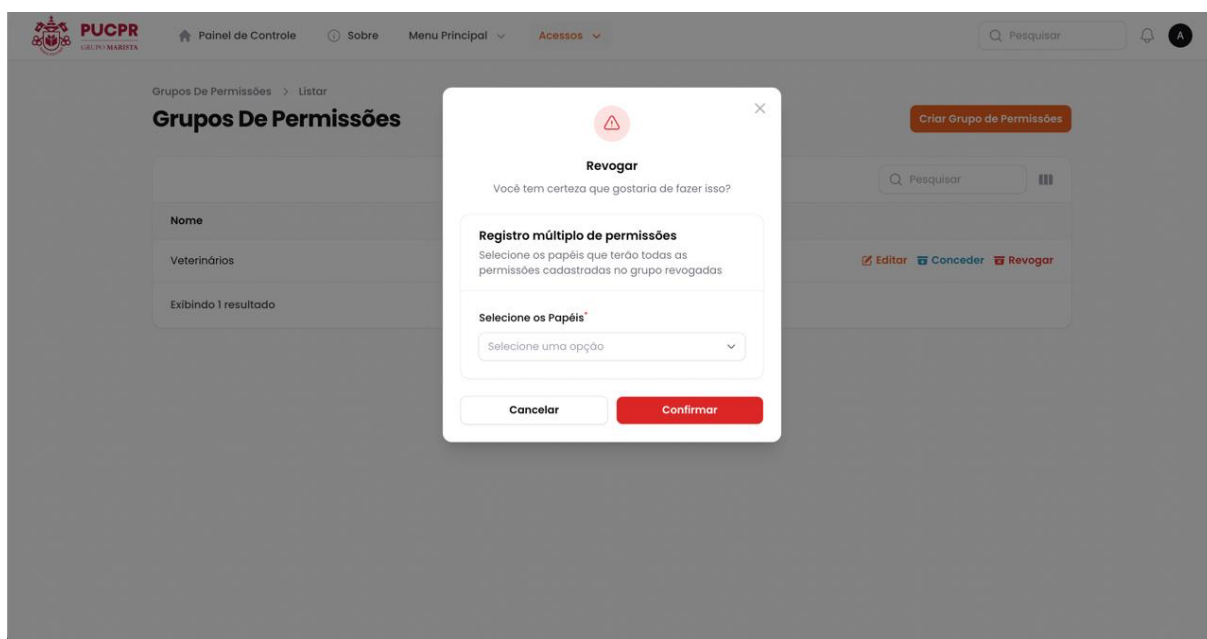
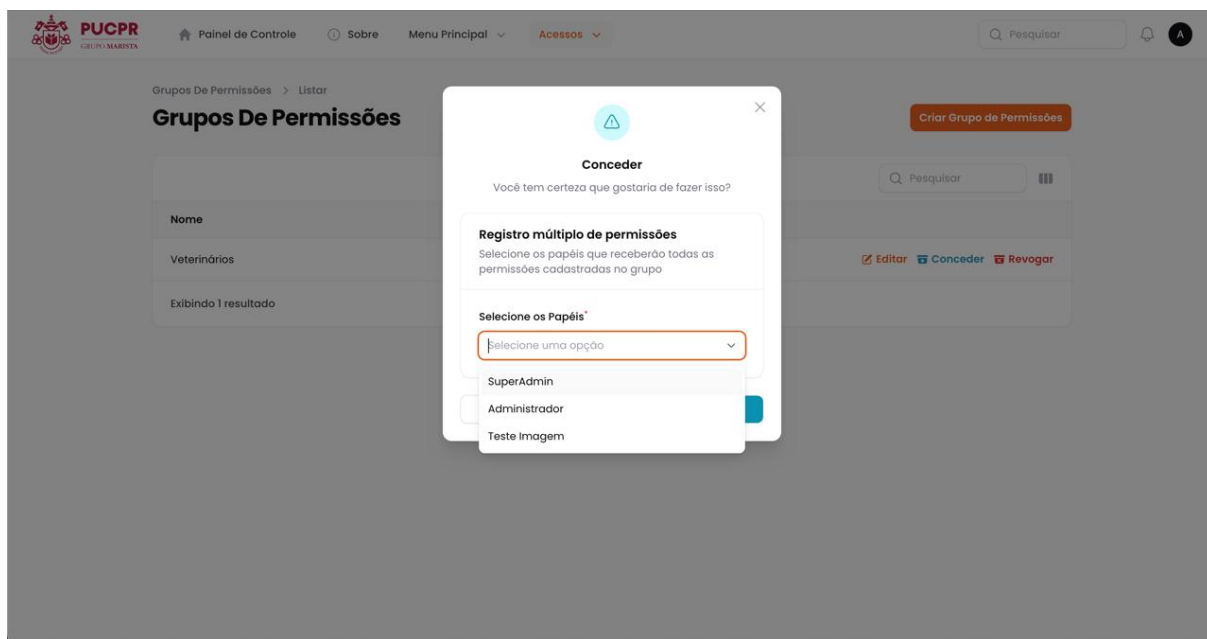


The screenshot shows the 'Criar Grupo De Permissões' (Create Permission Group) form. The header includes the PUCPR logo, navigation links (Painel de Controle, Sobre, Menu Principal, Acessos), a search bar, and a user profile icon. The form title is 'Criar Grupo De Permissões'. It contains two input fields: 'Nome*' (Name) and 'Descrição' (Description). Below the fields are three buttons: 'Criar' (Create), 'Salvar e criar outro' (Save and create another), and 'Cancelar' (Cancel).

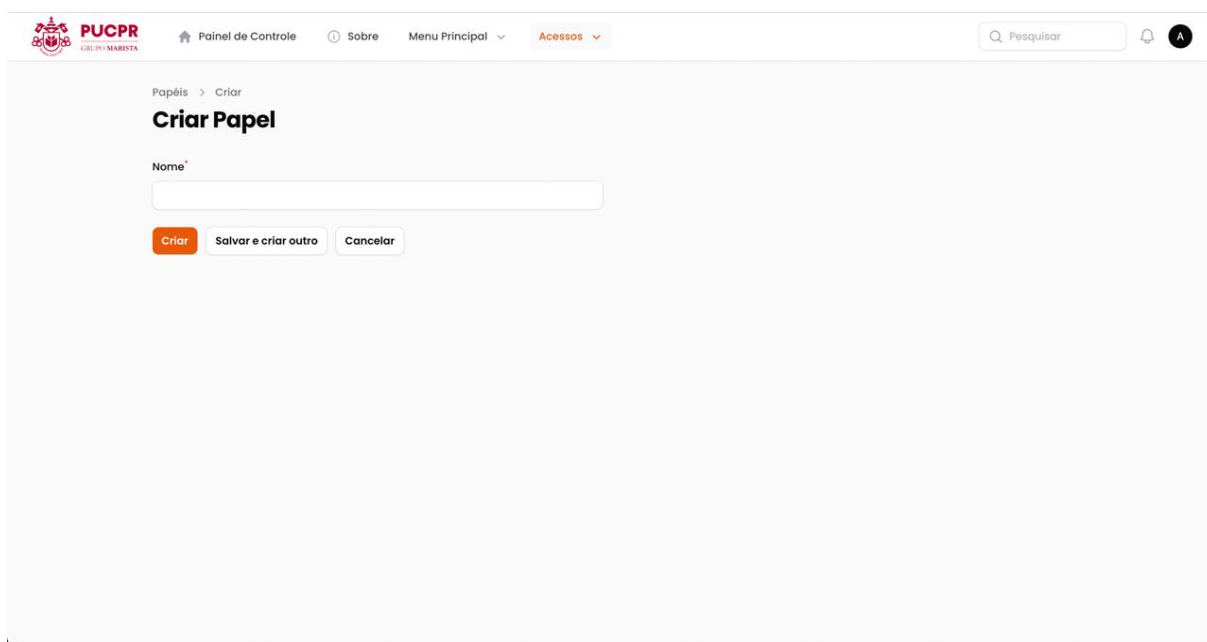


The screenshot shows the 'Editar Grupo De Permissões' (Edit Permission Group) form. The header is identical to the previous form. The form title is 'Editar Grupo De Permissões'. It contains two input fields: 'Nome*' (Name) with the value 'Veterinários' and 'Descrição' (Description). Below the fields are two buttons: 'Salvar alterações' (Save changes) and 'Cancelar' (Cancel). To the right of the form title is an 'Excluir' (Delete) button. Below the input fields is a section titled 'Permissões' (Permissions) with a 'Vincular' (Link) button. The section is currently empty, displaying 'Sem registros' (No records).

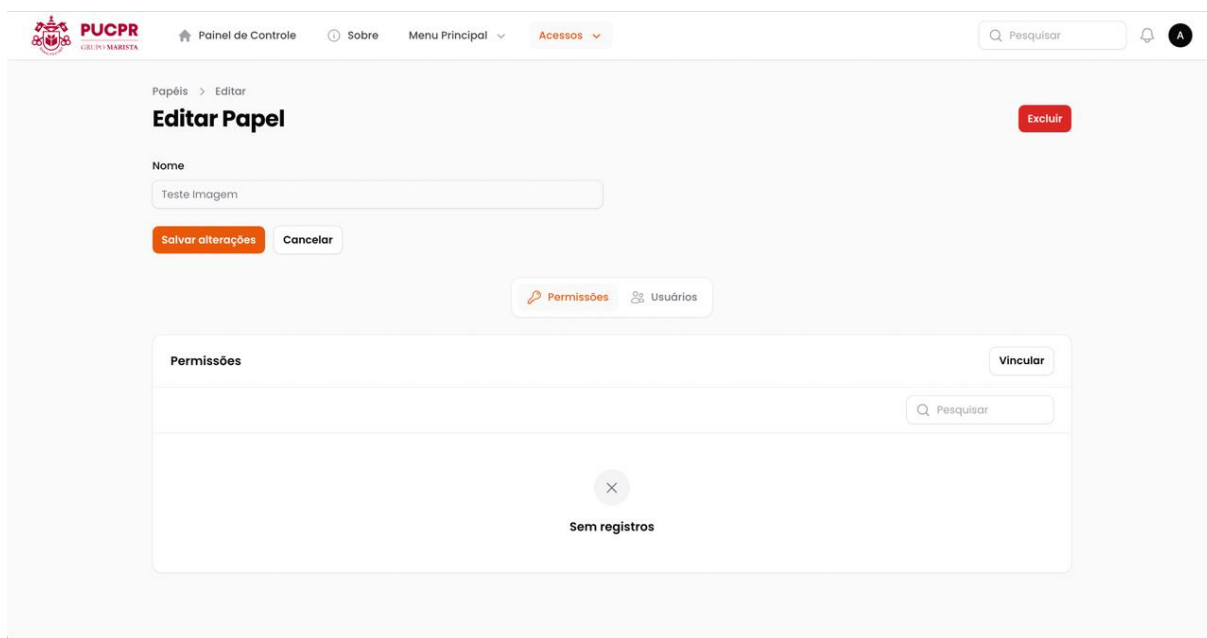




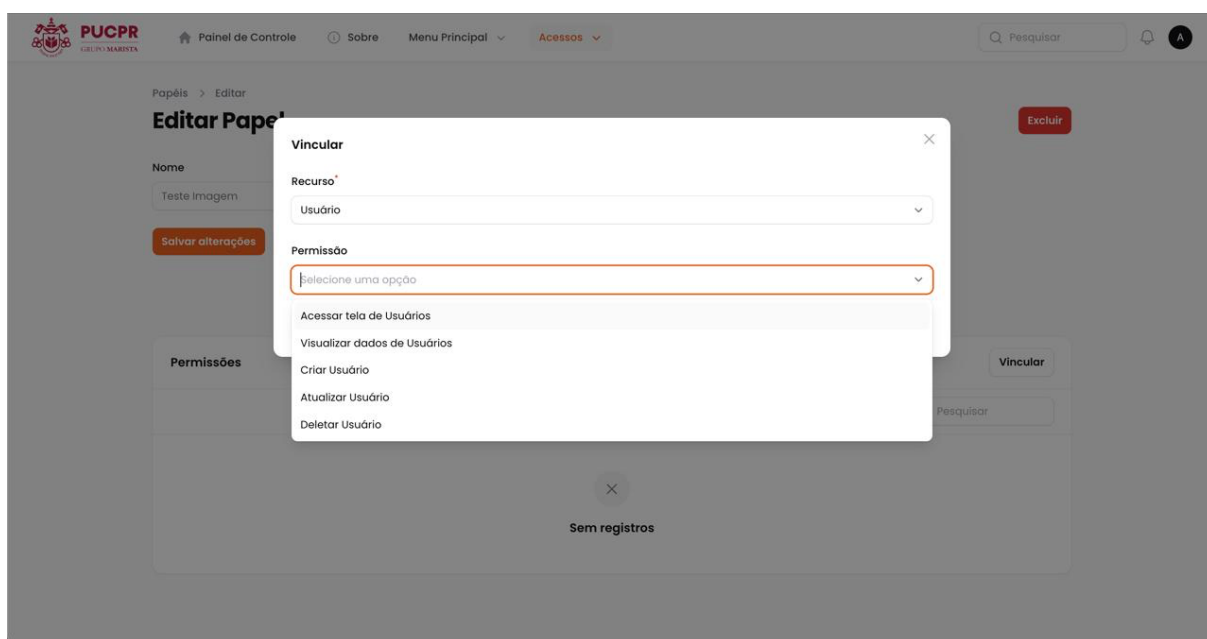
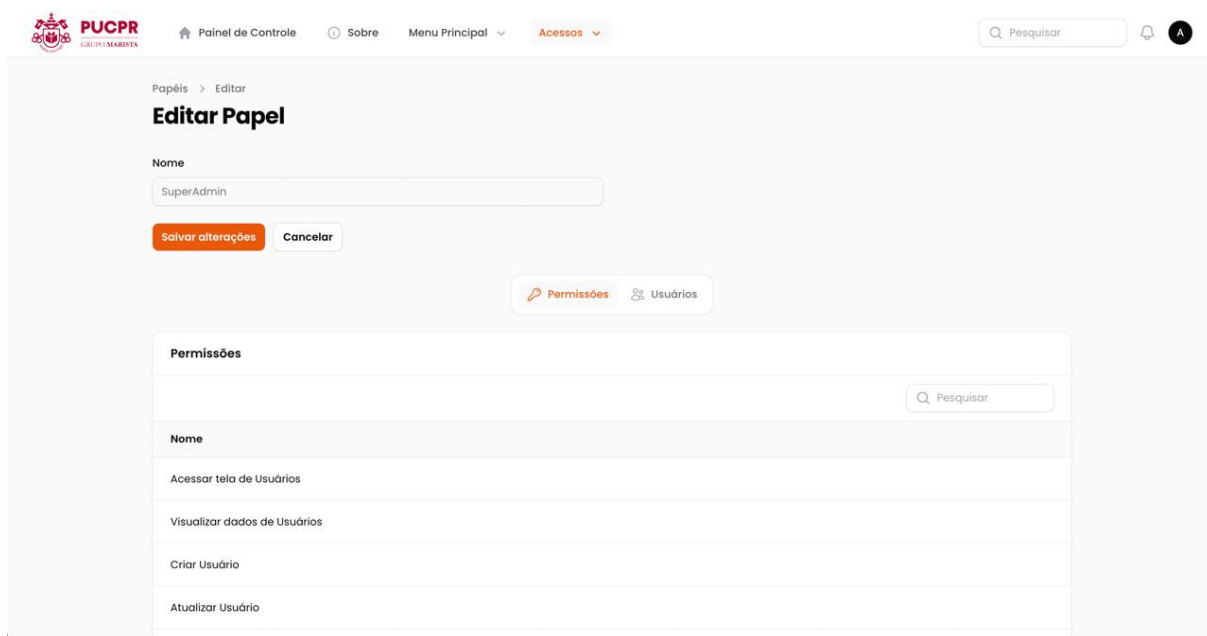
Grupo das Telas Papéis

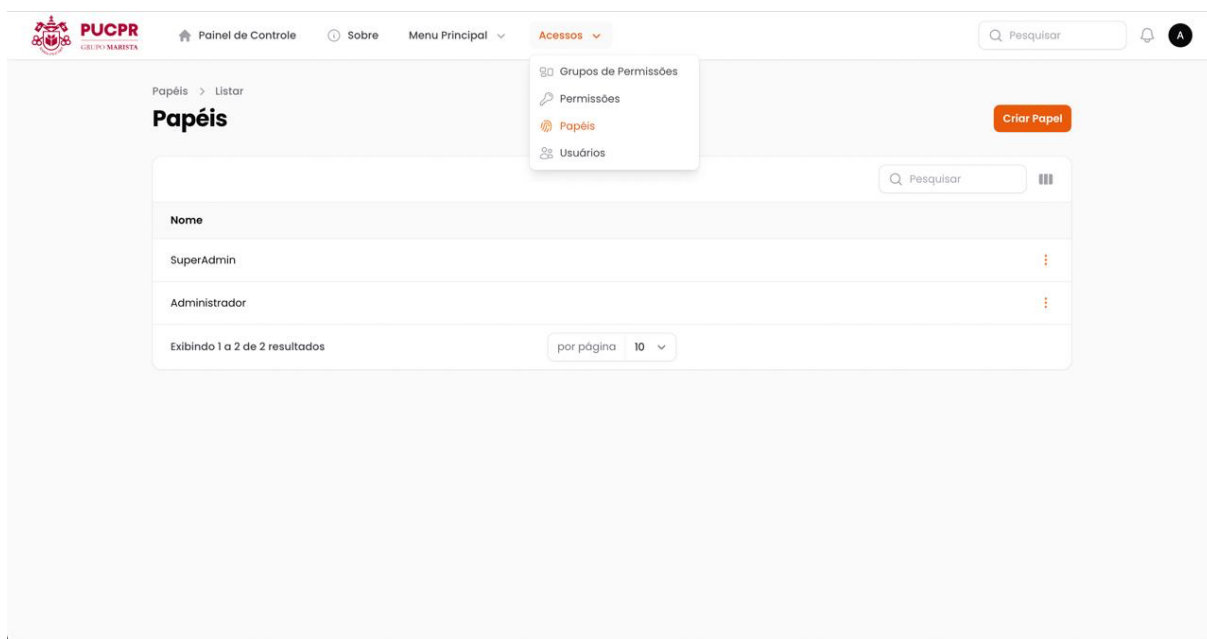
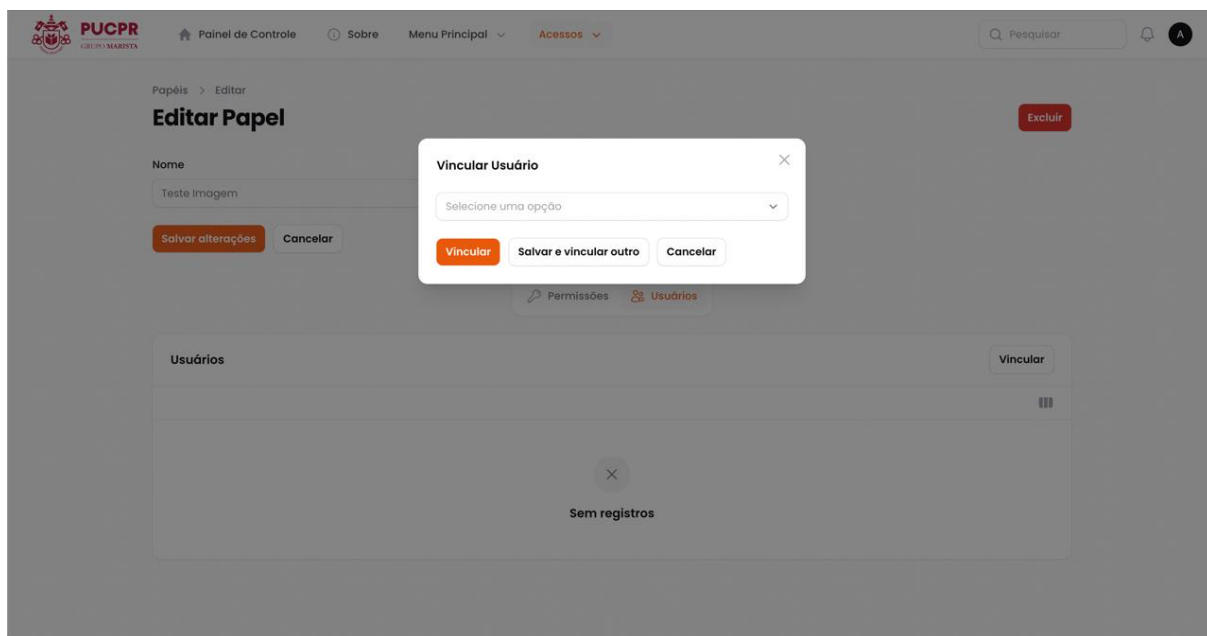


The screenshot shows the 'Criar Papel' (Create Role) form. The header includes the PUCPR logo, navigation links (Painel de Controle, Sobre, Menu Principal, Acessos), a search bar, and a user profile icon. The main content area has a breadcrumb 'Papéis > Criar' and the title 'Criar Papel'. Below the title is a form with a 'Nome' label and a text input field. At the bottom of the form are three buttons: 'Criar' (orange), 'Salvar e criar outro' (gray), and 'Cancelar' (gray).

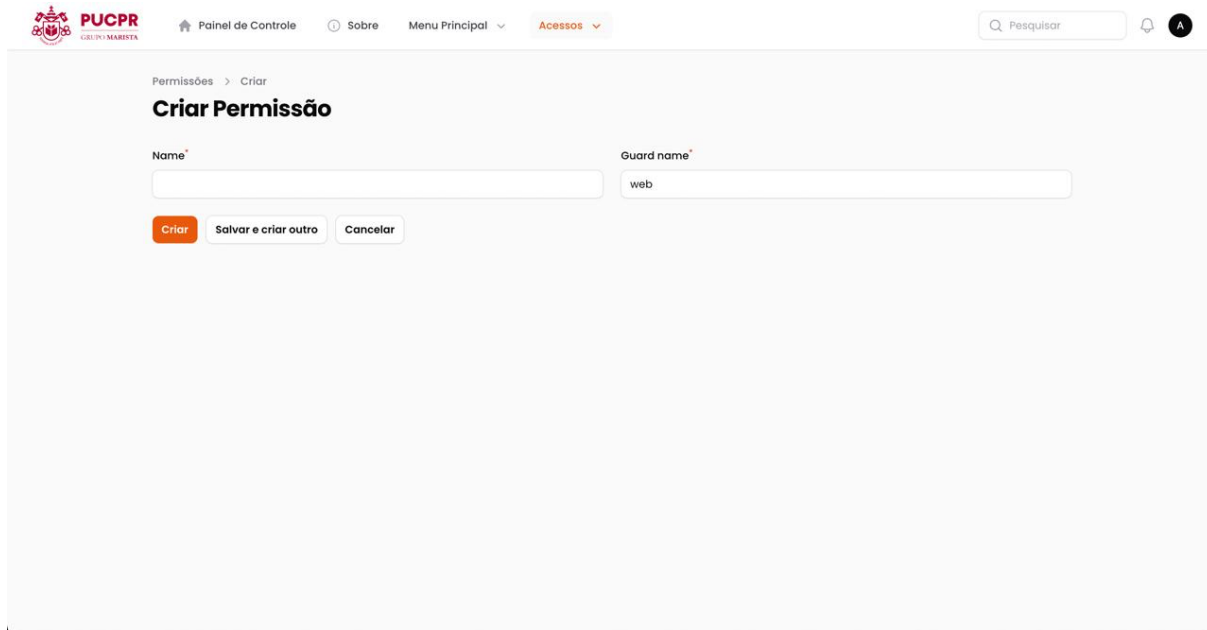


The screenshot shows the 'Editar Papel' (Edit Role) form. The header is identical to the previous form. The main content area has a breadcrumb 'Papéis > Editar' and the title 'Editar Papel'. Below the title is a form with a 'Nome' label and a text input field containing 'Teste Imagem'. To the right of the title is a red 'Excluir' button. Below the form are two buttons: 'Salvar alterações' (orange) and 'Cancelar' (gray). Below these buttons are two tabs: 'Permissões' (selected) and 'Usuários'. The 'Permissões' tab shows a table with a search bar and a 'Vincular' button. The table is currently empty, displaying 'Sem registros' (No records).

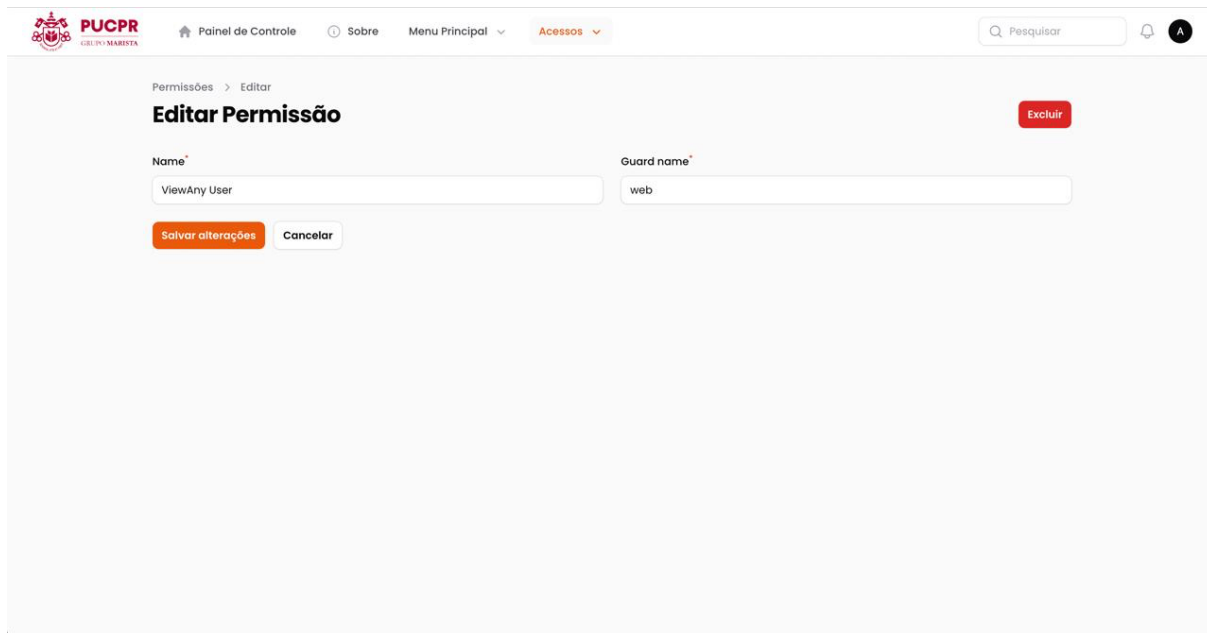




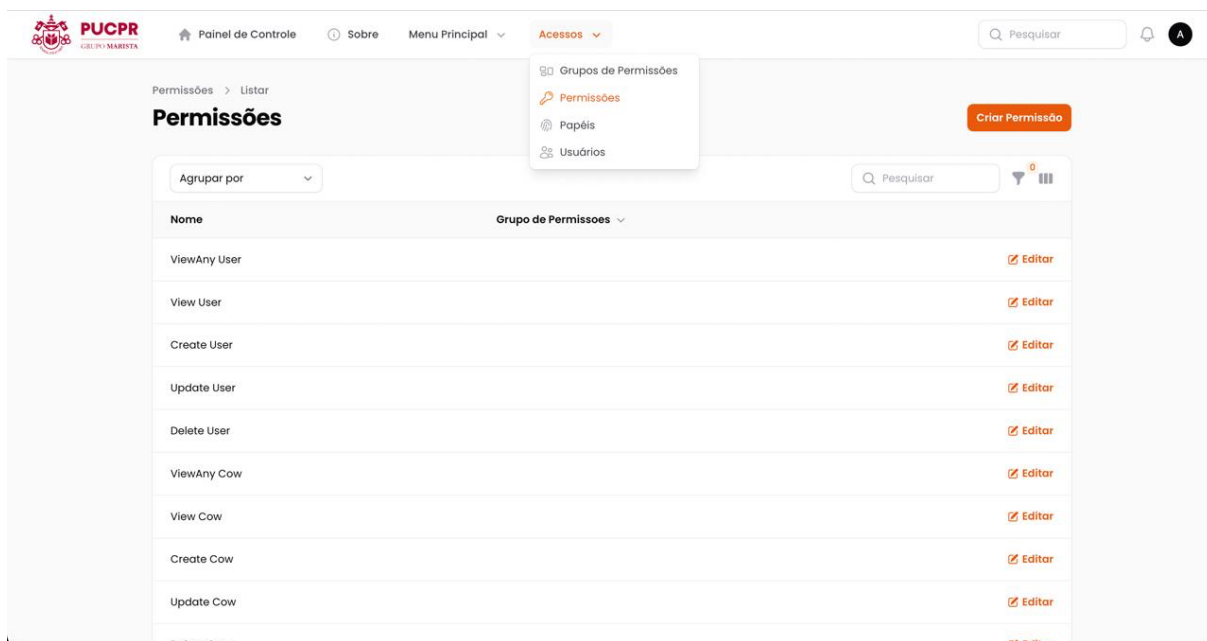
Grupo das Telas Permissões



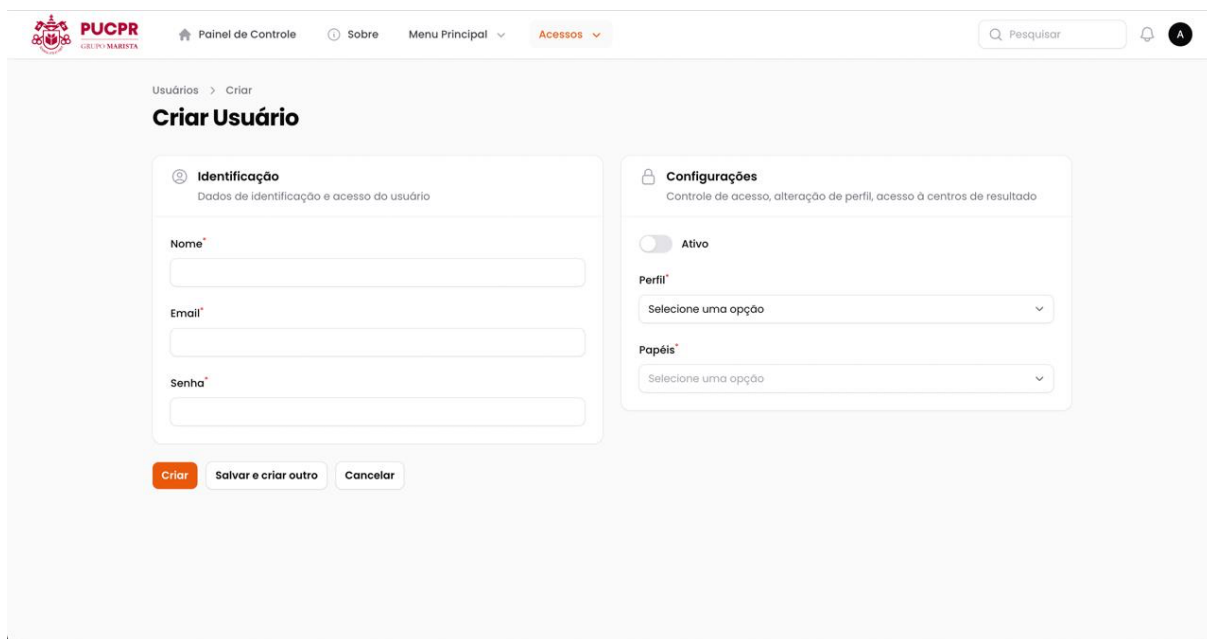
The screenshot shows the 'Criar Permissão' (Create Permission) form. The header includes the PUCPR logo, navigation links (Painel de Controle, Sobre, Menu Principal, Acessos), a search bar, and a user profile icon. The form title is 'Criar Permissão'. It contains two input fields: 'Name' (empty) and 'Guard name' (containing 'web'). Below the fields are three buttons: 'Criar' (orange), 'Salvar e criar outro' (gray), and 'Cancelar' (gray).



The screenshot shows the 'Editar Permissão' (Edit Permission) form. The header is identical to the previous form. The form title is 'Editar Permissão'. It contains two input fields: 'Name' (containing 'ViewAny User') and 'Guard name' (containing 'web'). Below the fields are two buttons: 'Salvar alterações' (orange) and 'Cancelar' (gray). An 'Excluir' (Delete) button is located in the top right corner of the form area.



Grupo das Telas Usuários



 **PUCPR** GRUPO MARISTA

[Painel de Controle](#) [Sobre](#) [Menu Principal](#) [Acessos](#)

Usuários > admin > Editar

Editar admin

[Excluir](#)

Identificação

Dados de identificação e acesso do usuário

Nome*

Email

Senha

[Salvar alterações](#) [Cancelar](#)

Configurações

Controle de acesso, alteração de perfil, acesso à centros de resultado

☒ Ativo

Perfil*

Administrador

Papéis*

SuperAdmin x

Selecione uma opção

 **PUCPR** GRUPO MARISTA

[Painel de Controle](#) [Sobre](#) [Menu Principal](#) [Acessos](#)

Usuários > Listar

Usuários

[Criar Usuário](#)

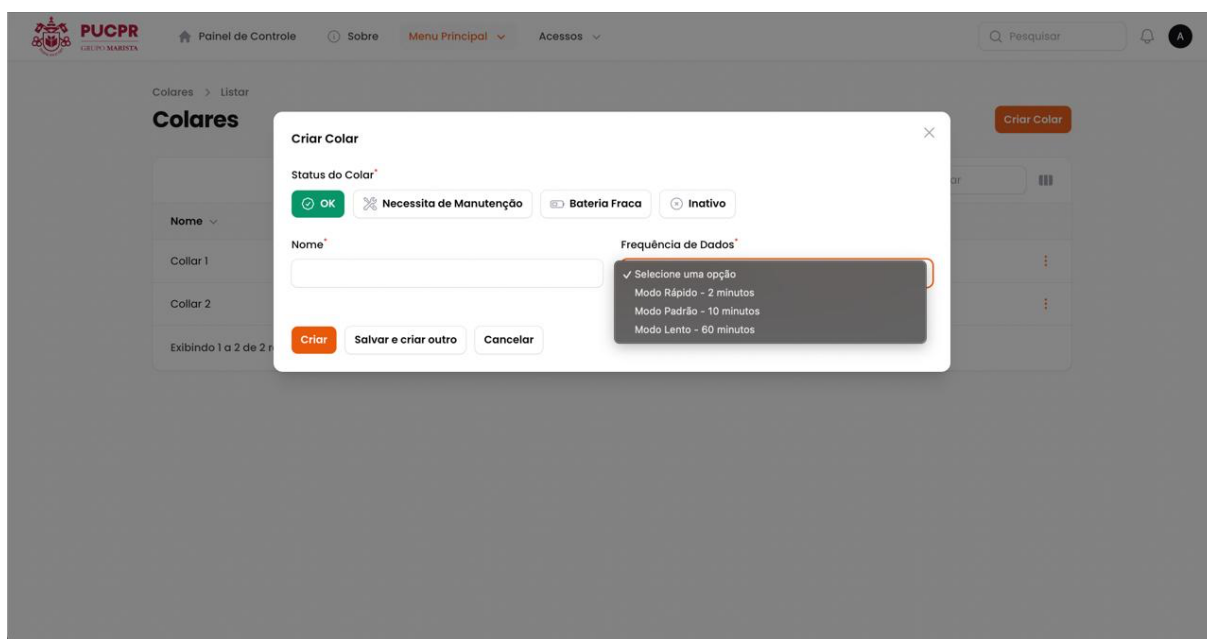
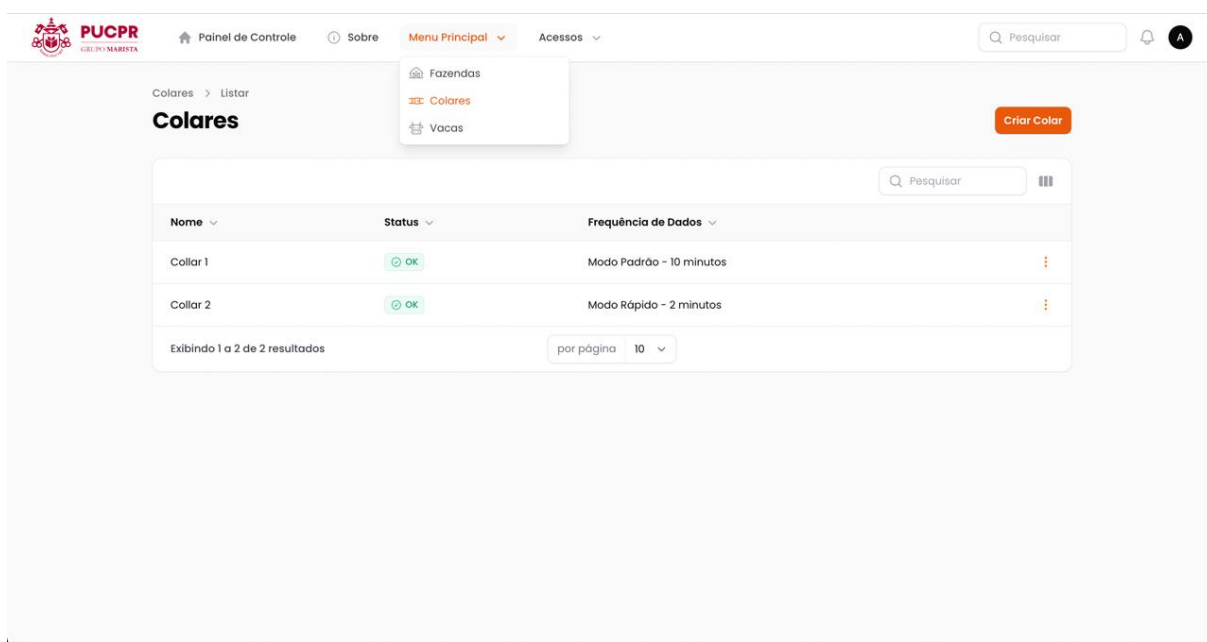
[Grupos de Permissões](#)
[Permissões](#)
[Papéis](#)
[Usuários](#)

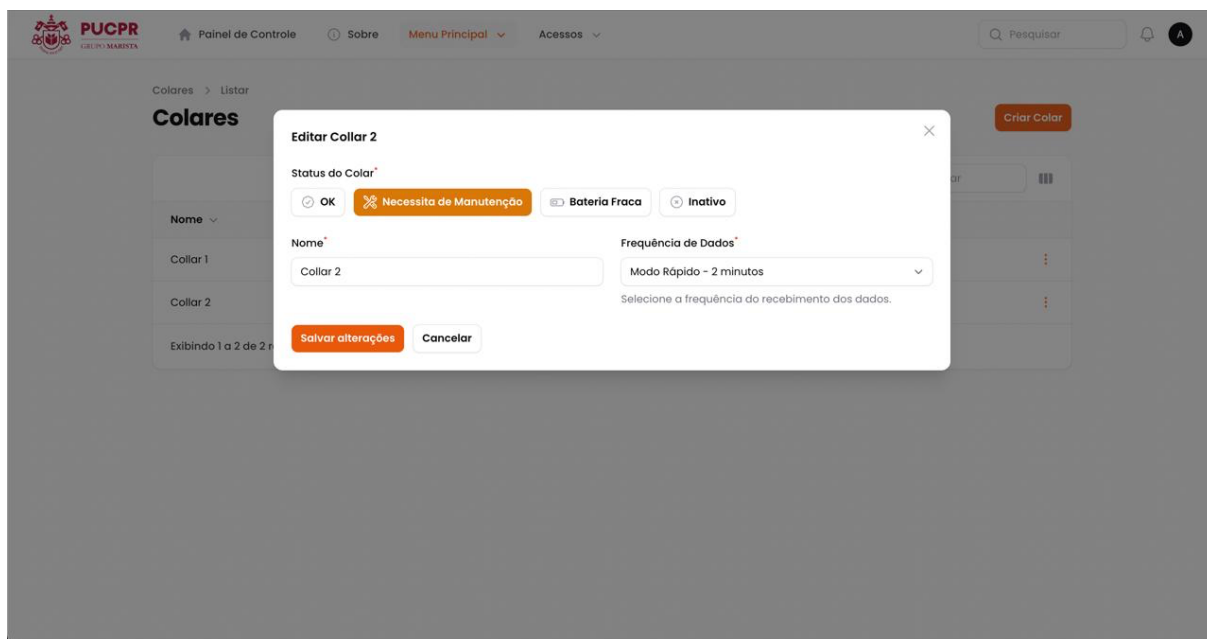
☐	Perfil	Papéis	Nome	Email	Acesso	
☐	Administrador	SuperAdmin	admin	admin@admin.com		Editar

Exibindo 1 resultado

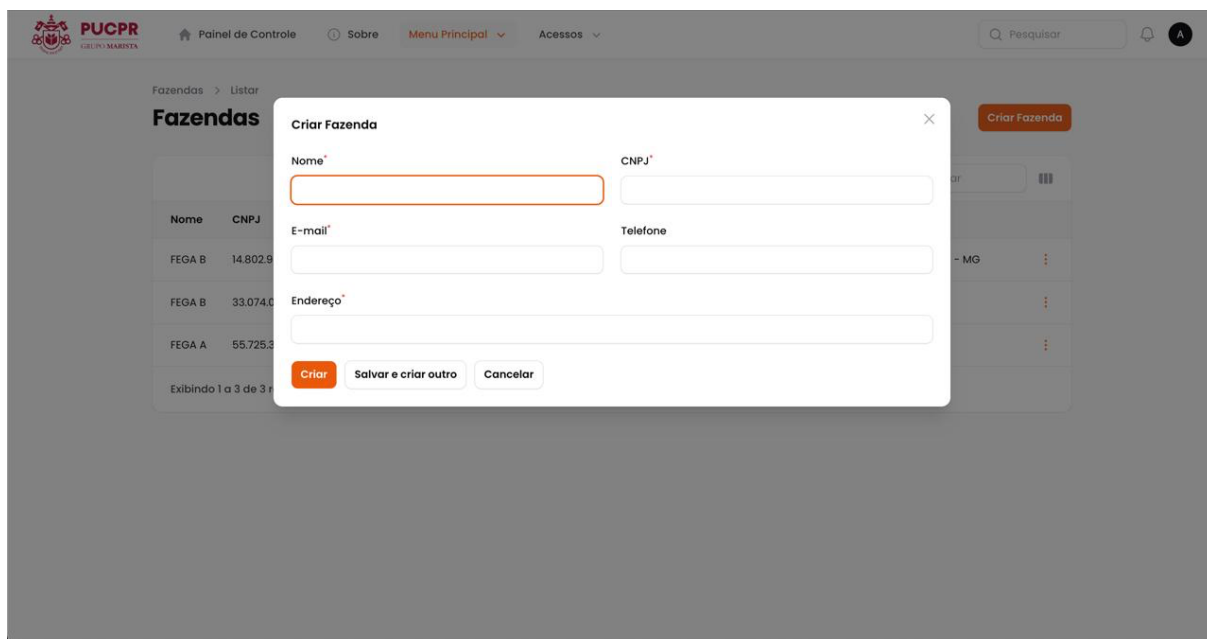
por página 10

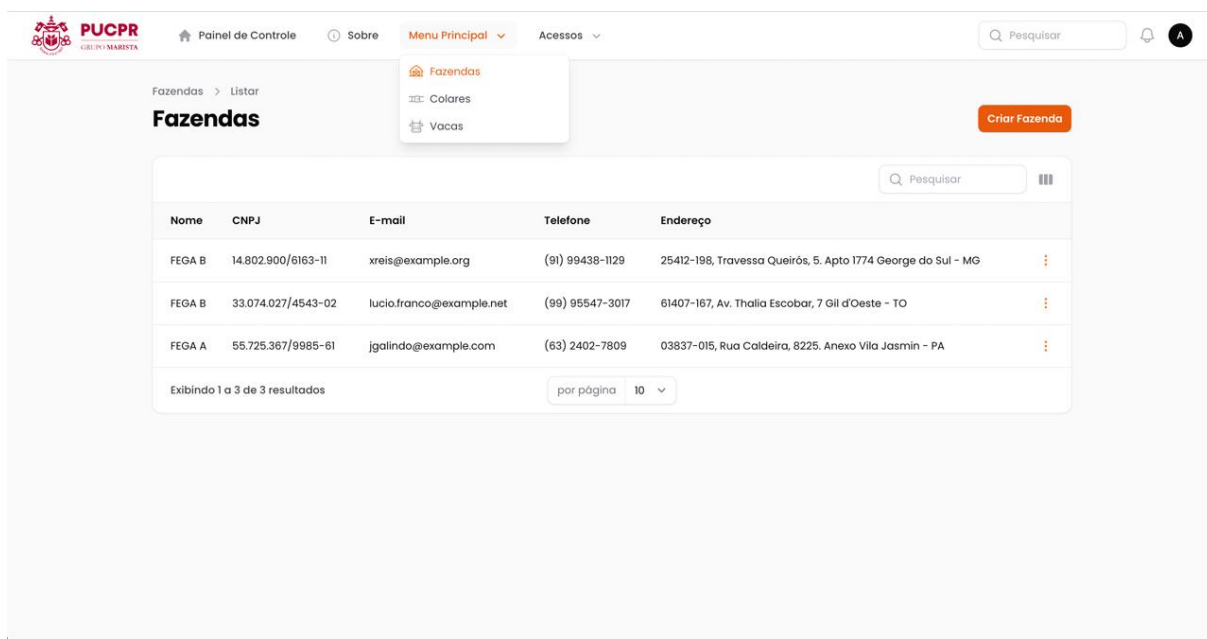
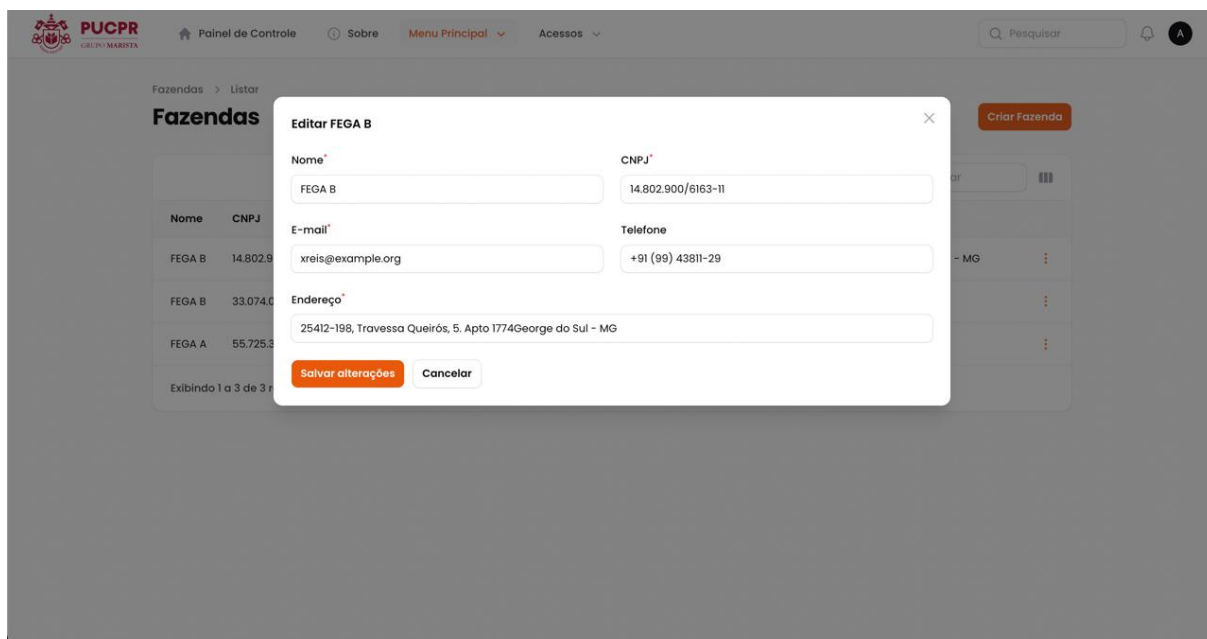
Grupo das Telas Colares



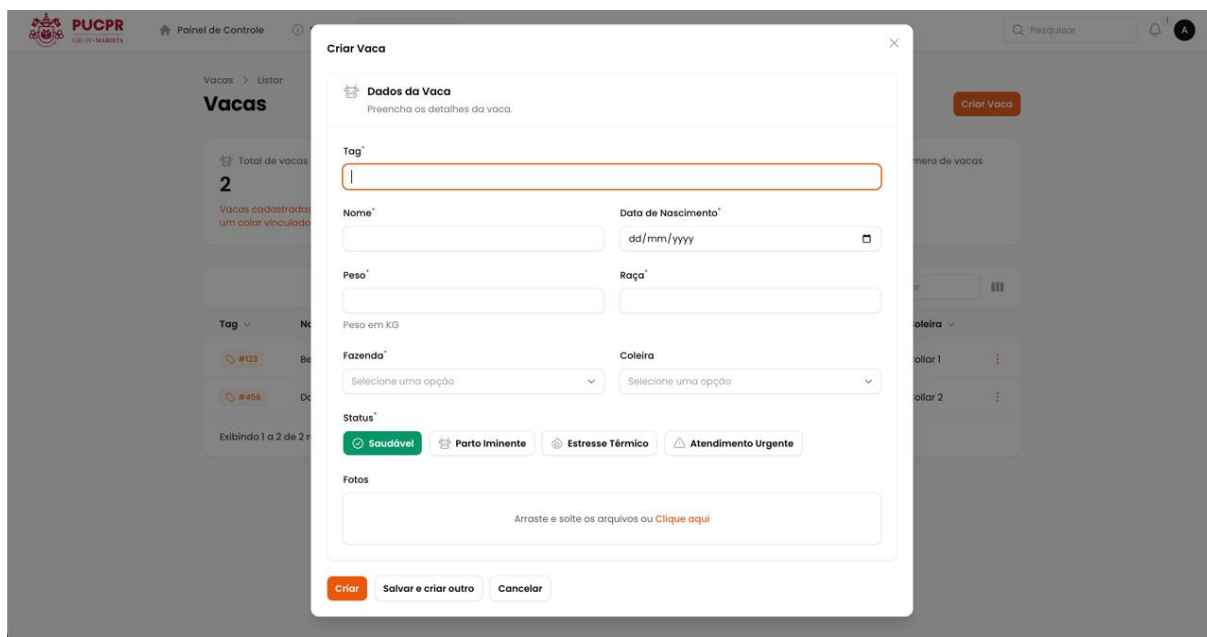


Grupo das Telas Fazendas





Grupo das Telas Vacas



Criar Vaca

Dados da Vaca
Preencha os detalhes da vaca.

Tag*

Nome*

Data de Nascimento*

Peso*

Raça*

Fazenda*

Coleira*

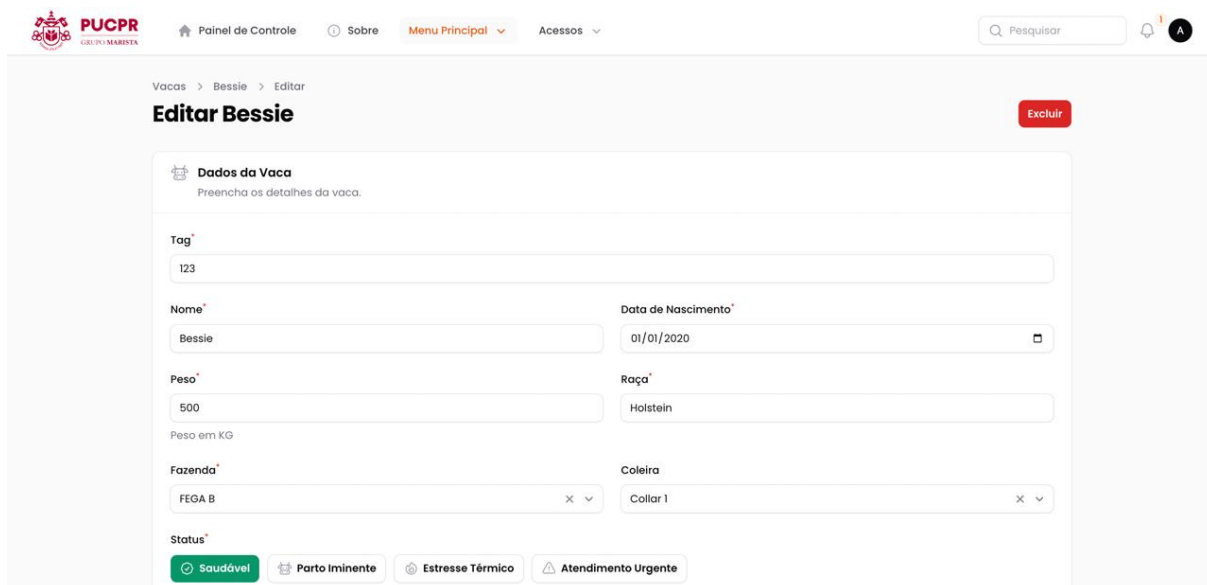
Status*

☒ Saúdável ☐ Parto Iminente ☐ Estresse Térmico ☐ Atendimento Urgente

Fotos

Arraste e solte os arquivos ou [Clique aqui](#)

Criar **Salvar e criar outro** **Cancelar**



Editar Bessie

Dados da Vaca
Preencha os detalhes da vaca.

Tag*

Nome*

Data de Nascimento*

Peso*

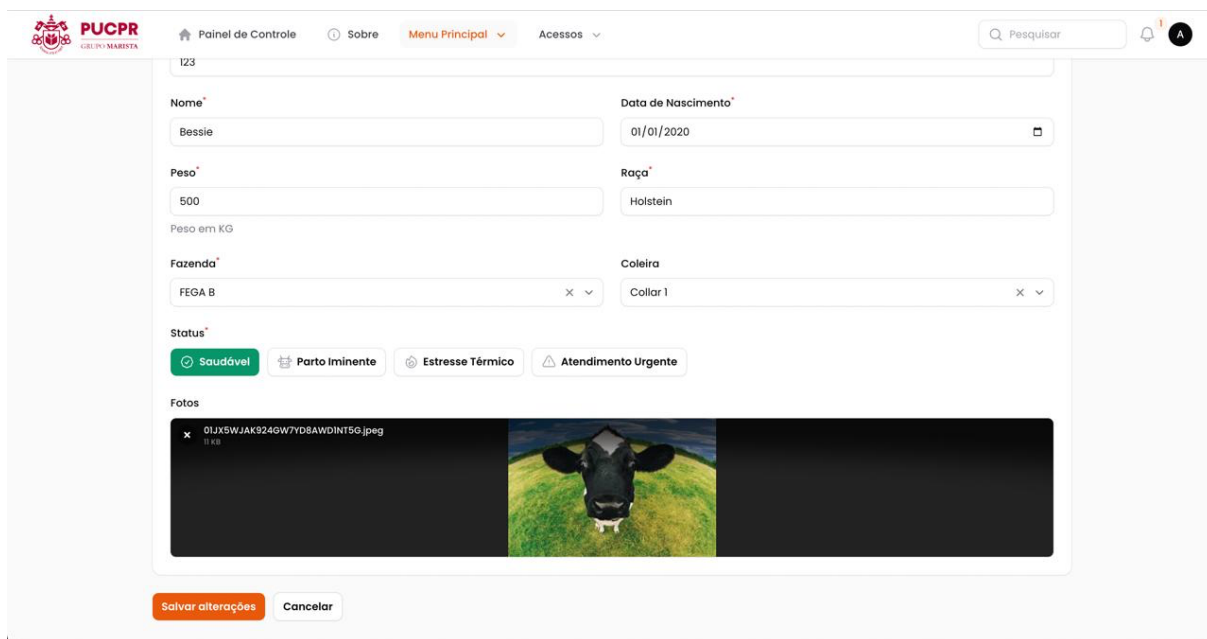
Raça*

Fazenda*

Coleira*

Status*

☒ Saúdável ☐ Parto Iminente ☐ Estresse Térmico ☐ Atendimento Urgente



Painel de Controle **Sobre** **Menu Principal** **Acessos** 

123

Nome* **Data de Nascimento***

Peso* **Raça***

Peso em KG

Fazenda* **Coleira**

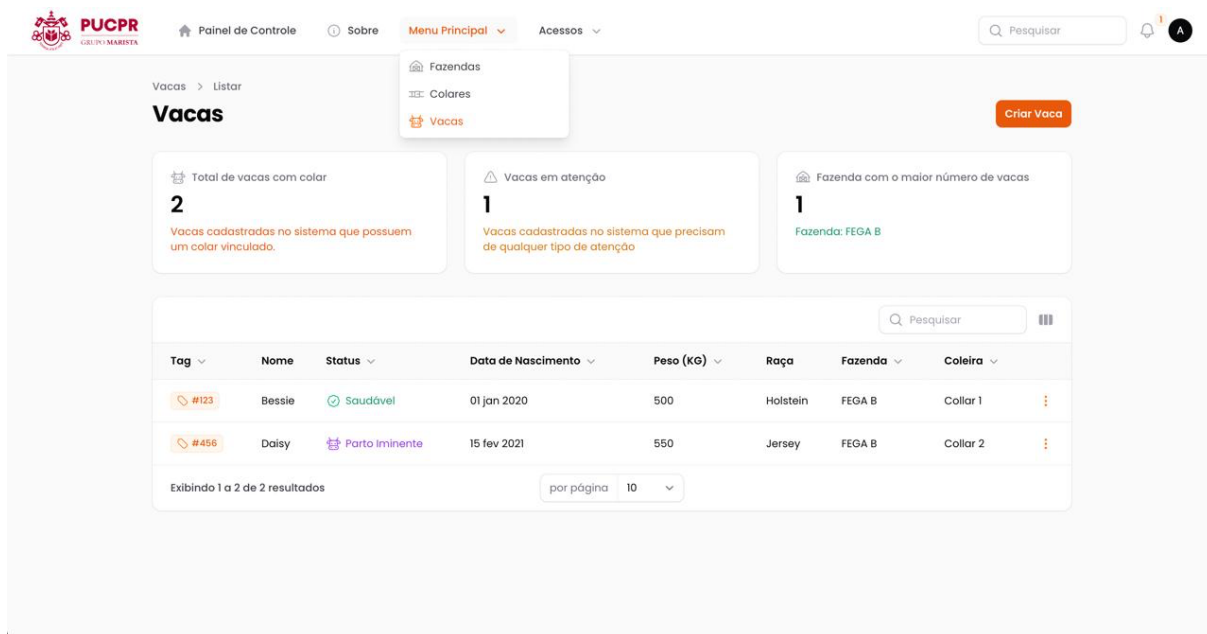
Status*

☒ Saudável ☐ Parto Iminente ☐ Estresse Térmico ☐ Atendimento Urgente

Fotos

☒ 01JXSWJAK924GW7YD8AWDINT5G.jpeg 

Salvar alterações **Cancelar**



Painel de Controle **Sobre** **Menu Principal** **Acessos** 

Vacas > Listar

Vacas

Fazendas
Colares
Vacas

Criar Vaca

Total de vacas com colar
2
Vacas cadastradas no sistema que possuem um colar vinculado.

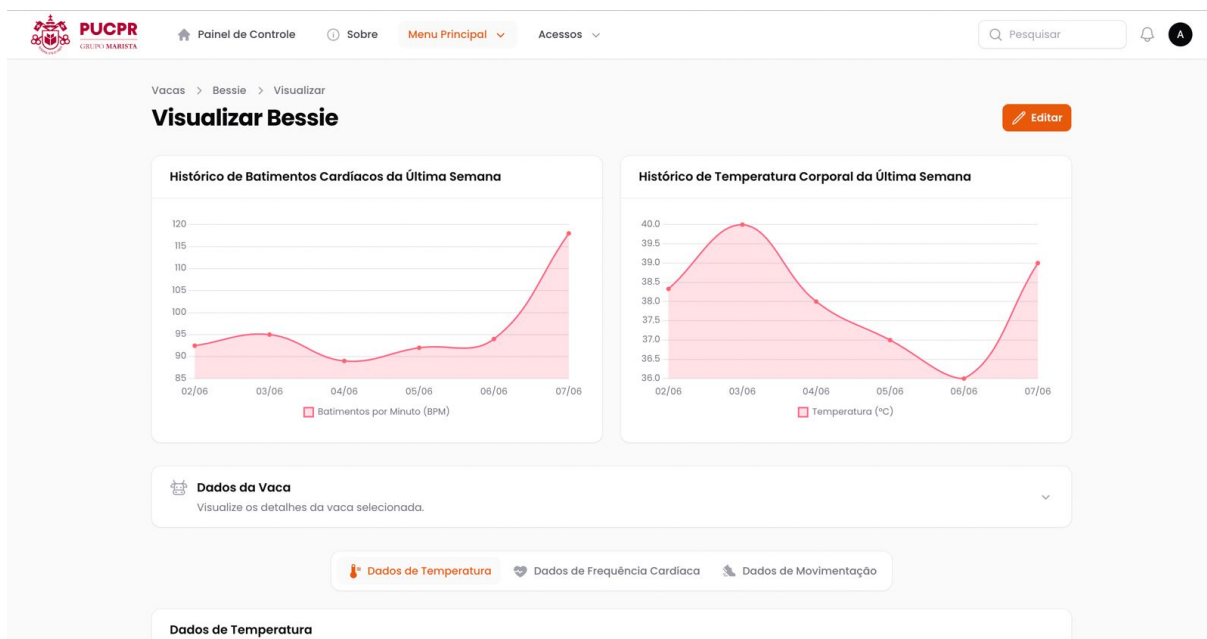
Vacas em atenção
1
Vacas cadastradas no sistema que precisam de qualquer tipo de atenção

Fazenda com o maior número de vacas
1
Fazenda: FEGA B

Tag	Nome	Status	Data de Nascimento	Peso (KG)	Raça	Fazenda	Coleira
#123	Bessie	Saudável	01 jan 2020	500	Holstein	FEGA B	Collar 1
#456	Daisy	Parto Iminente	15 fev 2021	550	Jersey	FEGA B	Collar 2

Exibindo 1 a 2 de 2 resultados

por página 10



Dados da Vaca
Visualize os detalhes da vaca selecionada.

Nome
Bessie

Data de Nascimento
01/01/2020

Peso
500
Peso em KG

Raça
Holstein

Fazenda
FEGA B

Coleira
Collar 1

Status
Saudável
Parto Iminente
Estresse Térmico
Atendimento Urgente

Fotos
Arraste e solte os arquivos ou [Clique aqui](#)

01JXSWJAK924QW7YD8AWDINT5Q.jpeg
11 KB



PUCPR GRUPO MARISTA

Painel de Controle Sobre Menu Principal Acessos

Pesquisar

Dados de Temperatura Dados de Frequência Cardíaca Dados de Movimentação

Dados de Frequência Cardíaca

Frequência Cardíaca	Recebido Em	Atualizado Em
112 BPM	02 jun 2025 - 00:39	02 jun 2025 - 00:39
68 BPM	02 jun 2025 - 00:39	02 jun 2025 - 00:39
97 BPM	02 jun 2025 - 00:39	02 jun 2025 - 00:39
100 BPM	02 jun 2025 - 00:39	02 jun 2025 - 00:39
91 BPM	02 jun 2025 - 00:39	02 jun 2025 - 00:39
66 BPM	02 jun 2025 - 00:39	02 jun 2025 - 00:39
114 BPM	02 jun 2025 - 00:39	02 jun 2025 - 00:39
78 BPM	02 jun 2025 - 00:39	02 jun 2025 - 00:39
92 BPM	05 jun 2025 - 00:39	02 jun 2025 - 00:39
95 BPM	03 jun 2025 - 00:39	02 jun 2025 - 00:39

Exibindo 1 a 10 de 56 resultados

por página 10

1 2 3 4 5 6 >

PUCPR GRUPO MARISTA

Painel de Controle Sobre Menu Principal Acessos

Pesquisar

Dados de Temperatura Dados de Frequência Cardíaca Dados de Movimentação

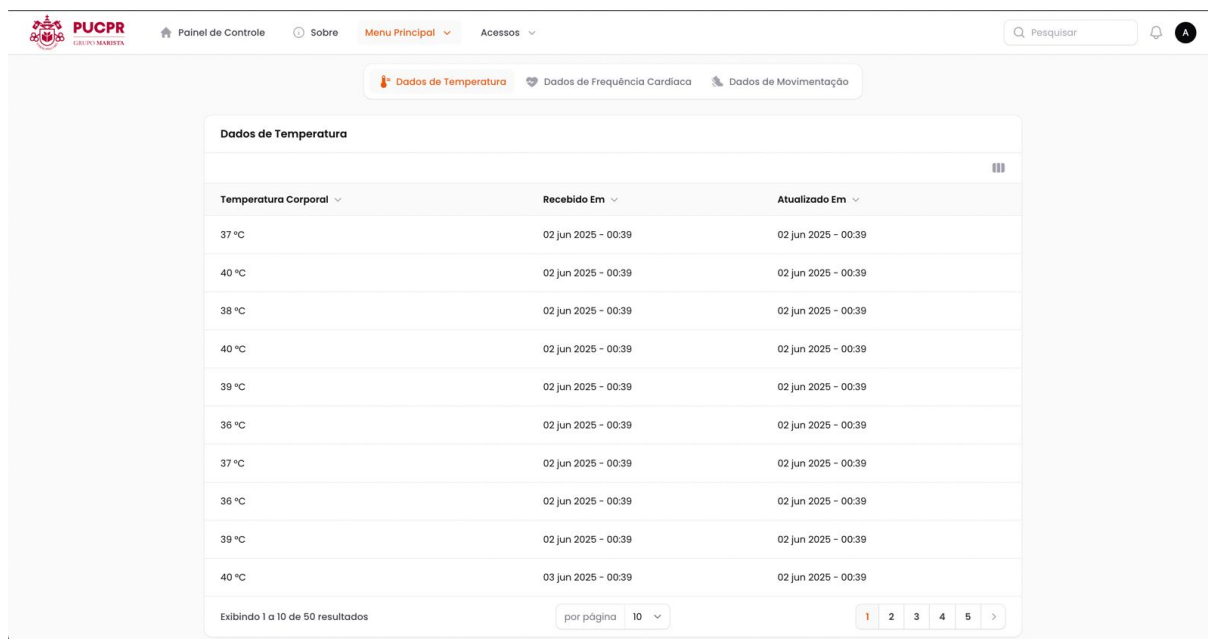
Dados de Movimentação

Giro X	Giro Y	Giro Z	Acel X	Acel Y	Acel Z	Recebido Em	Atualizado Em
2,06	-8,04	9,87	9,39	6,86	5,24	02 jun 2025 - 00:39	02 jun 2025 - 00:39
-8,42	-2,66	8,33	-4,68	3,33	5,58	02 jun 2025 - 00:39	02 jun 2025 - 00:39
-1,38	-7,94	-2,59	2,11	-5,28	-8,11	02 jun 2025 - 00:39	02 jun 2025 - 00:39
-1,61	-0,68	-4,91	-5,88	-5,78	-1,5	02 jun 2025 - 00:39	02 jun 2025 - 00:39
-5,71	-4,54	0,57	-8,12	7,61	-3,93	02 jun 2025 - 00:39	02 jun 2025 - 00:39
8,21	-0,93	-3,61	1,01	-5,99	1,56	02 jun 2025 - 00:39	02 jun 2025 - 00:39
2,5	-2,58	-4,74	1,39	5,33	-4,49	02 jun 2025 - 00:39	02 jun 2025 - 00:39
-5,78	5,59	7,54	9,31	0,38	-7,44	02 jun 2025 - 00:39	02 jun 2025 - 00:39
5,2	0,09	-7,76	3,35	-0,74	9,99	02 jun 2025 - 00:39	02 jun 2025 - 00:39
1,8	-3,51	-8,69	5,86	8,81	0,06	02 jun 2025 - 00:39	02 jun 2025 - 00:39

Exibindo 1 a 10 de 48 resultados

por página 10

1 2 3 4 5 >



The screenshot displays the 'Dados de Temperatura' (Temperature Data) section of the CowHealth AI web application. The interface includes a header with the PUCPR logo, navigation links (Painel de Controle, Sobre, Menu Principal, Acessos), a search bar, and a notification icon. Below the header, there are tabs for 'Dados de Temperatura', 'Dados de Frequência Cardíaca', and 'Dados de Movimentação'. The main content area shows a table with the following data:

Temperatura Corporal	Recebido Em	Atualizado Em
37 °C	02 jun 2025 - 00:39	02 jun 2025 - 00:39
40 °C	02 jun 2025 - 00:39	02 jun 2025 - 00:39
38 °C	02 jun 2025 - 00:39	02 jun 2025 - 00:39
40 °C	02 jun 2025 - 00:39	02 jun 2025 - 00:39
39 °C	02 jun 2025 - 00:39	02 jun 2025 - 00:39
36 °C	02 jun 2025 - 00:39	02 jun 2025 - 00:39
37 °C	02 jun 2025 - 00:39	02 jun 2025 - 00:39
36 °C	02 jun 2025 - 00:39	02 jun 2025 - 00:39
39 °C	02 jun 2025 - 00:39	02 jun 2025 - 00:39
40 °C	03 jun 2025 - 00:39	02 jun 2025 - 00:39

At the bottom of the table, it indicates 'Exibindo 1 a 10 de 50 resultados' (Showing 1 to 10 of 50 results) and a pagination control set to 'por página 10'.

7. ARQUITETURA DO BANCO DE DADOS DO SISTEMA COWHEALTH AI

7.1. Estrutura do Banco de Dados

O banco de dados do projeto CowHealth AI foi projetado para sustentar todas as operações da plataforma de monitoramento de saúde bovina, integrando dados provenientes de sensores embarcados (como acelerômetros, sensores térmicos e de frequência cardíaca) com os módulos de controle, visualização e gestão de usuários na plataforma web. A modelagem segue princípios relacionais, utilizando o SGBD MySQL 8.0 com engine InnoDB, garantindo suporte à integridade referencial, escalabilidade e desempenho sob múltiplas conexões simultâneas.

7.1.1. Tabelas e Finalidades

Tabela	Finalidade
accelerometer_data	Registra os dados do acelerômetro e giroscópio (eixos X, Y, Z), associados a cada vaca.
cache	Armazena dados temporários utilizados pelo framework Laravel para caching.
cache_locks	Controle de concorrência sobre operações de cache distribuído.
collars	Identifica as coleiras eletrônicas vinculadas a vacas específicas.
cows	Tabela central que armazena informações das vacas monitoradas, incluindo dados biométricos e estado de saúde.
failed_jobs	Registra jobs Laravel que falharam durante o processamento assíncrono.
farms	Contém os registros das propriedades rurais onde os dispositivos estão instalados.
heart_rate_data	Armazena batimentos cardíacos (bpm) coletados pelos sensores MAX30102.
job_batches	Gerencia o agrupamento de jobs assíncronos para controle de execução em lote.
jobs	Fila de execução de tarefas Laravel – como análise de saúde ou recepção de dados via MQTT.
migrations	Controle de versionamento do schema do banco de dados Laravel.
model_has_permissions	Associa permissões específicas a instâncias de modelos (polimórfico).
model_has_roles	Associa usuários ou entidades a cargos (roles) específicos.
notifications	Registra alertas e notificações geradas no sistema (ex.: saúde crítica de uma vaca).
password_reset_tokens	Gerencia tokens de recuperação de senha por e-mail.
permission_group_permission	Associação entre permissões e grupos de permissões.
permission_groups	Agrupamento hierárquico de permissões para modularidade e governança.
permissions	Conjunto de permissões específicas atribuíveis a usuários ou cargos.
role_has_permissions	Associa cargos (roles) a permissões, conforme o modelo RBAC.

roles	Define os papéis possíveis no sistema (produtor, veterinário, administrador).
sessions	Gerencia sessões ativas dos usuários logados.
temperature_data	Armazena leituras de temperatura capturadas pelos sensores MLX90614.
users	Tabela de usuários do sistema, incluindo autenticação, perfil e identificação pessoal.

7.1.2. Modelagem Conceitual e Relacionamentos

Esta seção descreve, em formato narrativo e técnico, a modelagem conceitual do banco de dados do sistema CowHealth AI. São apresentadas as tabelas, seus campos (colunas), as chaves primárias (PK) e as chaves estrangeiras (FK), descrevendo como se relacionam entre si. A estrutura foi extraída diretamente do script *cow_health_app_mysql.sql* e complementada pela interpretação do diagrama entidade-relacionamento fornecido pela equipe do projeto.

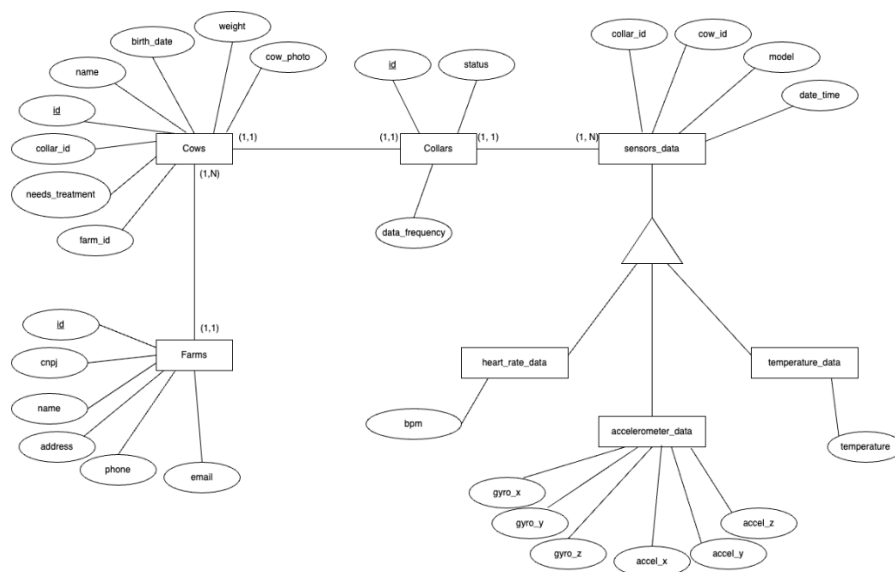


Figura 9. Modelo Conceitual do BD

- **Tabela: accelerometer_data**

Colunas:

- accelerometer_data
- id
- cow_id
- gyro_x
- gyro_y
- gyro_z
- accel_x
- accel_y
- accel_z
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: cache**

Colunas:

- cache
- key
- value
- expiration

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: cache_locks**

Colunas:

- cache_locks
- key
- owner
- expiration

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: collars**

Colunas:

- collars
- id
- name
- status
- data_frequency
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: cows**

Colunas:

- cows
- id
- name
- birth_date
- weight
- race
- cow_photos
- status
- tag
- farm_id
- collar_id
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: failed_jobs**

Colunas:

- failed_jobs
- id

- uuid
- connection
- queue
- payload
- exception
- failed_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: farms**

Colunas:

- farms
- id
- cnpj
- name
- address
- phone
- email
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: heart_rate_data**

Colunas:

- heart_rate_data
- id
- cow_id
- bpm
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: jobs**

Colunas:

- jobs
- id
- queue
- payload
- attempts
- reserved_at
- available_at
- created_at

Chave Primária: *(não definida explicitamente)*

Chaves Estrangeiras: *nenhuma definida*

- **Tabela: job_batches**

Colunas:

- job_batches
- id
- name
- total_jobs
- pending_jobs
- failed_jobs
- failed_job_ids
- options
- cancelled_at
- created_at
- finished_at

Chave Primária: *(não definida explicitamente)*

Chaves Estrangeiras: *nenhuma definida*

- **Tabela: migrations**

Colunas:

- migrations
- id
- migration
- batch

Chave Primária: *(não definida explicitamente)*

Chaves Estrangeiras: *nenhuma definida*

- Tabela: model_has_permissions

Colunas:

- model_has_permissions
- permission_id
- model_type
- model_id

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- Tabela: model_has_roles

Colunas:

- model_has_roles
- role_id
- model_type
- model_id

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- Tabela: notifications

Colunas:

- notifications
- id
- type
- notifiable_type
- notifiable_id
- data
- read_at
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- Tabela: password_reset_tokens

Colunas:

- password_reset_tokens
- email
- token
- created_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- Tabela: permissions

Colunas:

- permissions
- id
- name
- guard_name
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- Tabela: permission_groups

Colunas:

- permission_groups
- id
- nome
- descricao
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- Tabela: permission_group_permission

Colunas:

- permission_group_permission
- id

- permission_group_id
- permission_id
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: roles**

Colunas:

- roles
- id
- name
- guard_name
- created_at
- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: role_has_permissions**

Colunas:

- role_has_permissions
- permission_id
- role_id

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: sessions**

Colunas:

- sessions
- id
- user_id
- ip_address
- user_agent
- payload

- last_activity

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: temperature_data**

Colunas:

- temperature_data

- id

- cow_id

- temperature

- created_at

- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

- **Tabela: users**

Colunas:

- users

- id

- name

- ativo

- perfil

- email

- email_verified_at

- password

- remember_token

- created_at

- updated_at

Chave Primária: (não definida explicitamente)

Chaves Estrangeiras: nenhuma definida

7.1.3. Diagrama ER

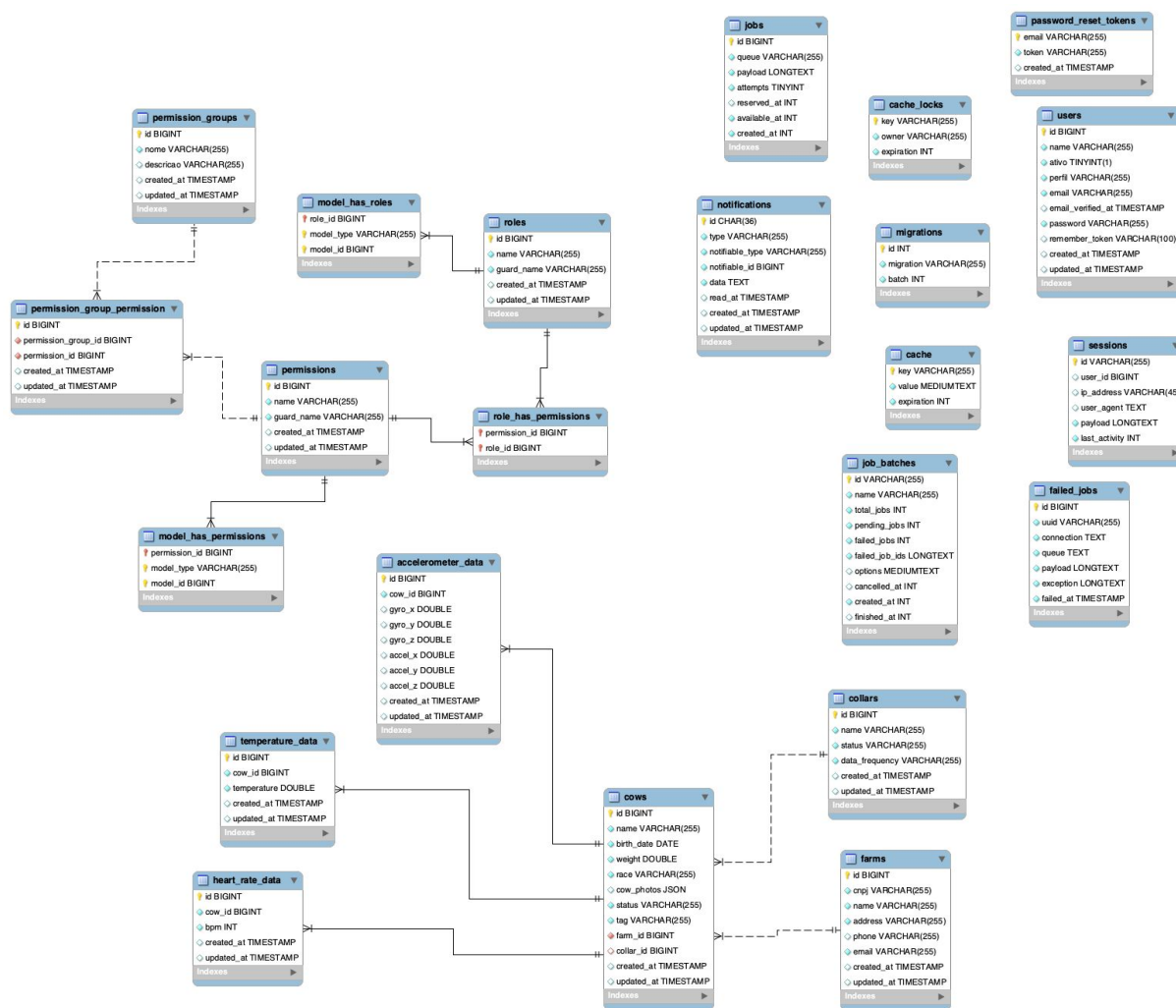


Figura 10. Diagrama ER

7.2. Correlação entre Banco de Dados e Resultados de Aprendizagem

De acordo com o Plano de Ensino da disciplina, os estudantes devem ser capazes de estruturar soluções computacionais orientadas a dados, desenvolvendo aplicações multicamadas que integrem bases relacionais, sistemas embarcados e interfaces web funcionais e seguras.

A modelagem e implementação do banco de dados do projeto CowHealth AI demonstra o atendimento direto a esses objetivos pedagógicos, articulando conceitos fundamentais de engenharia de dados, normalização, integridade referencial, autenticação multiusuário e interoperabilidade.

A estrutura relacional implementada não apenas organiza os dados clínicos e operacionais do sistema, mas também viabiliza análises em tempo real, inferências clínicas baseadas em heurísticas, controle de acesso por perfis e papéis e exportação de informações (ainda não implementados), contribuindo para uma solução completa, escalável e academicamente rigorosa.

RA2 – Desenvolver aplicações com arquitetura multicamadas, conectando frontend, backend e base de dados

Temas relacionados:

- **TE4:** Arquitetura MVC e organização modular de sistemas web
- **TE6:** Integração com bancos de dados relacionais
- **TE7:** Segurança e controle de acesso em sistemas de informação

Contribuições do Projeto CowHealthAI

O banco de dados está diretamente conectado à camada de backend Laravel e à camada de apresentação via Filament. Cada entidade do domínio (ex.: Cow, Collar, User, HeartRateData) possui uma tabela dedicada com chaves estrangeiras e relacionamentos normalizados. A arquitetura segue o padrão de separação de camadas: a base de dados persiste, o backend processa e a interface exibe. A estrutura de permissões (roles, permissions, model_has_roles) garante governança dos dados em múltiplos perfis de usuários.

Indicadores Atendidos

- **ID2.1:** Estruturação de sistemas com separação clara de camadas
- **ID4.3:** Desenvolvimento de sistemas com tolerância a falhas

RA3 – Analisar, implementar e validar soluções computacionais orientadas a dados

Temas relacionados:

- **TE6:** Integração com bancos de dados relacionais
- **TE9:** Interoperabilidade, exportação/importação de dados

Contribuições do Projeto CowHealthAI

O banco de dados armazena milhões de pontos de dados fisiológicos organizados por animal, sensor, data e condição clínica. Os dados são utilizados para análise de padrões, inferência

heurística e geração de notificações. A estrutura foi modelada para suportar tanto leitura intensa quanto escrita assíncrona proveniente dos dispositivos. Tabelas como `temperature_data`, `accelerometer_data` e `heart_rate_data` formam a base de análises comparativas.

Indicadores Atendidos

- **ID3.1:** Coleta e análise de dados reais
- **ID3.2:** Apresentação de dados com visualização interativa

RA4 – Projetar sistemas que integrem software embarcado, redes de sensores e interface web

Temas relacionados:

- **TE8:** IoT e sistemas conectados com protocolos de comunicação
- **TE9:** Interoperabilidade, exportação/importação de dados

Contribuições do Projeto CowHealthAI

O banco de dados foi projetado para receber dados via MQTT por jobs que escutam os dispositivos embarcados (coleiras inteligentes). Cada entrada de sensor contém um `cow_id`, timestamp e valores padronizados. Isso permite que o backend realize inferências de saúde, armazenando resultados no banco para exibição e alertas. Essa integração fecha o ciclo entre

o mundo físico (dados) e digital (decisões), concretizando o conceito de sistema ciberfísico completo.

Indicadores Atendidos

- **ID4.1:** Integração com dispositivos e redes de sensores
- **ID4.3:** Desenvolvimento de sistemas com tolerância a falhas

8. CRONOGRAMA DE IMPLEMENTAÇÃO

Vide Anexo I – Cronograma

9. DIAGRAMAS

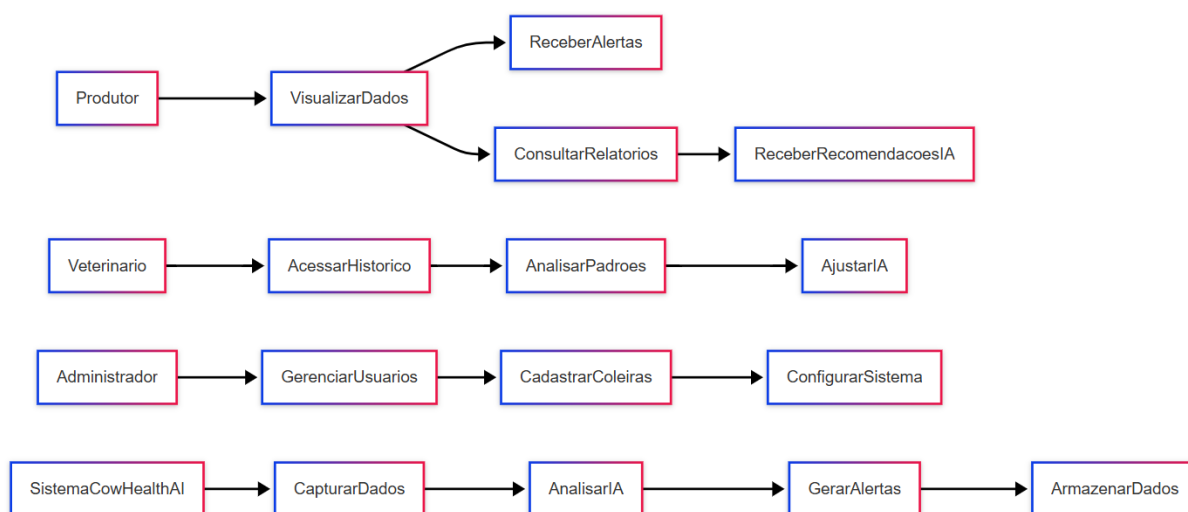


Figura 11 - Casos de Uso

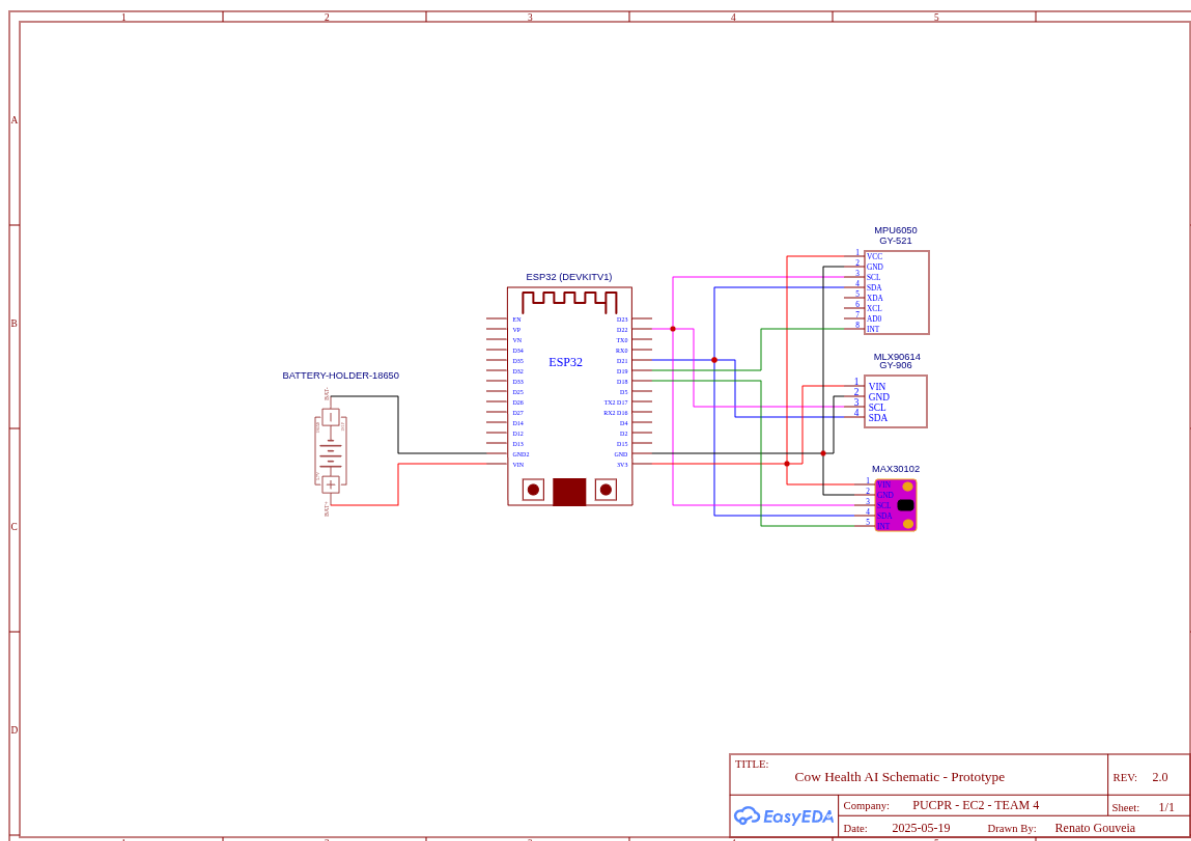


Figura 12 - Desenho esquemático do protótipo (Sujeito a alterações / melhorias)

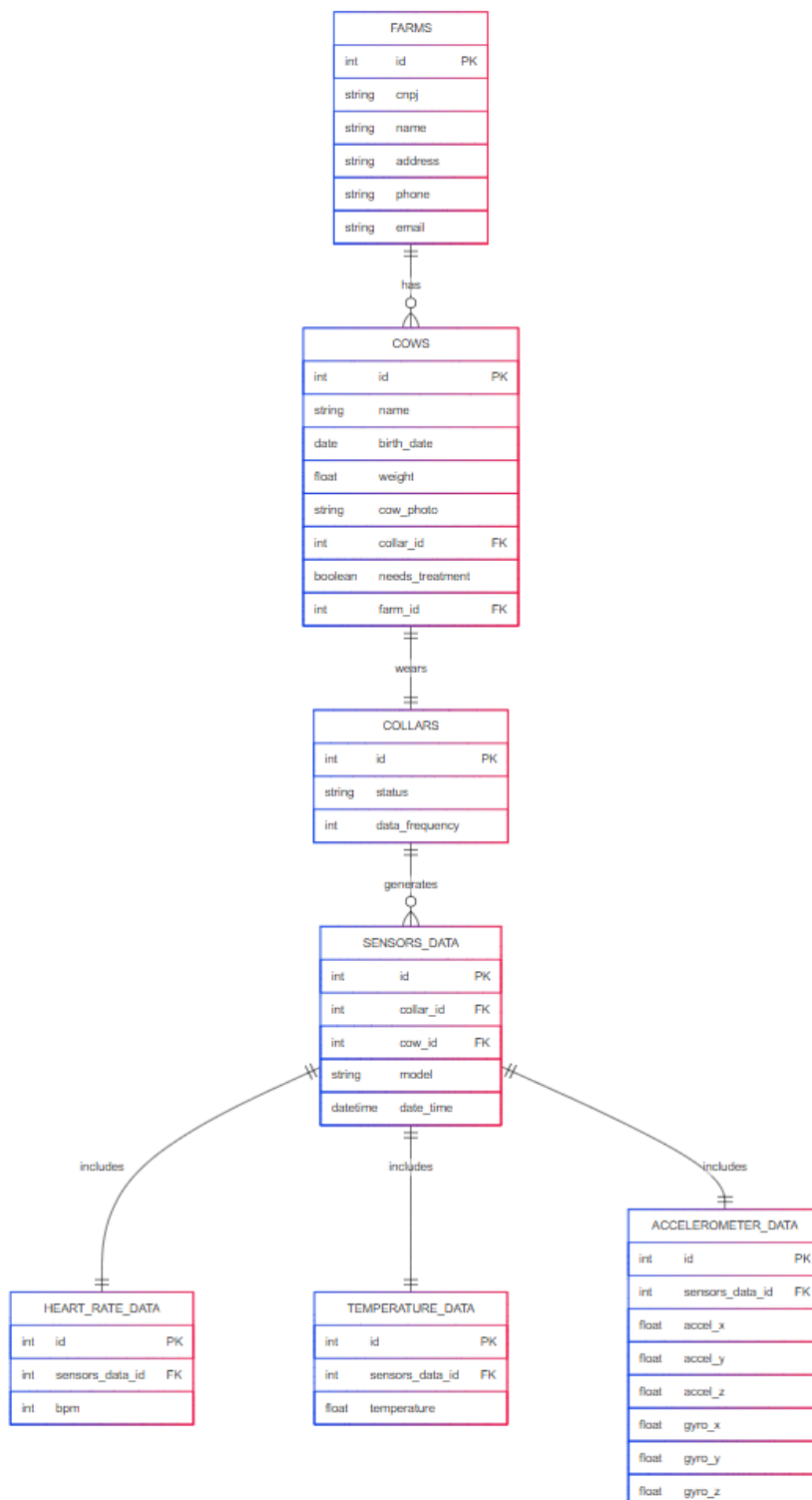


Figura 13 - Diagrama Entidade-Relacionamento

10. ANEXOS

Anexo II – Estudo de Consumo de Energia por componentes

Anexo III – *Datasheet* dos componentes

Anexo IV – Ação de Campo (Questionário/Diagnóstico FEGA)

Anexo V – Arquivos de Mídia (Fotos, vídeos e layouts prévios)

Anexo VI – Relatórios de Fabricação (Diversos)

Anexo VII – Modelos Heurísticos de Diagnósticos

11. REFERÊNCIAS

SOMMERVILLE, Ian. *Engenharia de Software*. 9. ed. São Paulo: **Pearson Education do Brasil**, 2011.

RAJAPAKSE, T *et al* IoT-Enabled Cattle Health and Location Monitoring System. **International Journal of Applied Sciences & Development**, v. 1, n. 2945-0454, p. 64–74, 31 dez. 2022.

ARSHAD, J *et al*. *Deployment of Wireless Sensor Network and IoT Platform to Implement an Intelligent Animal Monitoring System*. **MDPI Sustainability**, v. 14, n. 6249, p. 2–22, 20 maio 2022.

RAHMAWATI, D. *et al* Design of a Real Time Cow Smart Collar Health and Position Monitoring System. **9th Information Technology International Seminar (ITIS)**, v. 1, n. 59651, p. 201–207, 18 out. 2023.

KOLLURI, R. *et al* IoT-Based Smart Collar For Automatic Continuous Vital And Activity Monitoring Of A Cattle. **Industrial Engineering Journal**, v. 54, n. 1, p. 94–103, 10 jan. 2025.

CELERITAS. *How Livestock Monitoring Sensors Enhance Animal Health and Productivity*. 2025. Disponível em: <https://www.celeritasdigital.com/how-livestock-monitoring-sensors-enhance-animal-health-and-productivity/>. Acesso em: 12 abr. 2025.

CELERITAS. *The Rise of Wearable Devices in Monitoring Animal Health*. 2025. Disponível em: <https://www.celeritasdigital.com/the-rise-of-wearable-devices-in-monitoring-animal-health/>. Acesso em: 12 abr. 2025.

OLIVEIRA, P. C. S. *et al*. Tecnologias aplicadas ao monitoramento de parâmetros fisiológicos na produção de ruminantes. **Revista Agrária Acadêmica**, v. 6, n. 2, p. 27–37, 2023. DOI: 10.32406.